

A Ground-Up, Exam-Focused Treatment

Computer Vision

Synthesised from the complete posted course material

Topic order. Every topic below appears in the chronological order in which the lecturer posted it, from the oldest post (Lazebnik reference lectures) to the newest (Camera Calibration). Where a later post revisits a topic introduced earlier, the two treatments are merged at the point of first appearance and cross-referenced forward, so that nothing is repeated and nothing is lost.

Prerequisite assumed. None. Every term — convolution kernel, spatial frequency, homography, eigenvector, Lagrange multiplier, Jacobian — is built from a physical analogy before a single symbol is written.

Every topic follows the same three-part structure:

1. Intuitive theory and definition (with diagrams)
2. Mathematical formulation with every intermediate step written out
3. A fully solved numerical example

Contents

Notation Used Throughout	vii
I Foundations: Images, Filtering and Early Vision	1
1 What Computer Vision Is, and What an Image Really Is	2
1.1 Intuitive Theory and Definition	2
1.1.1 The four pillars of the course	2
1.1.2 An image is a function	3
1.2 Mathematical Formulation	3
1.3 Solved Numerical Example	4
2 Point Processing: Changing the Range	6
2.1 Intuitive Theory and Definition	6
2.2 Mathematical Formulation	7
2.2.1 Deriving contrast stretching step by step	7
2.3 Solved Numerical Example	8
3 Sampling, Aliasing and the Nyquist–Shannon Theorem	10
3.1 Intuitive Theory and Definition	10
3.2 Mathematical Formulation and Derivation	11
3.2.1 Why the factor of two? A step-by-step argument	11
3.2.2 Anti-aliasing: the only cure	12
3.3 Solved Numerical Example	12
4 Resampling and Interpolation	14
4.1 Intuitive Theory and Definition	14
4.2 Mathematical Formulation and Derivation	14
4.2.1 The basis-function view	15
4.3 Solved Numerical Example	16
5 Linear Filtering: Correlation and Convolution	18
5.1 Intuitive Theory and Definition	18
5.2 Mathematical Formulation	18
5.2.1 Correlation versus convolution	18
5.2.2 Properties, proved	19

5.2.3	Boundary handling	20
5.2.4	The standard kernel catalogue	20
5.2.5	Deriving the sharpening filter	20
5.3	Solved Numerical Example	21
6	Gaussian Filtering and Separability	23
6.1	Intuitive Theory and Definition	23
6.2	Mathematical Formulation and Derivation	24
6.2.1	Separability — the property that makes Gaussians practical	24
6.2.2	Other properties worth memorising	24
6.3	Solved Numerical Example	25
7	Non-Linear Filtering: Median and Bilateral	27
7.1	Intuitive Theory and Definition	27
7.1.1	Bilateral filtering: blur, but not across edges	27
7.2	Mathematical Formulation	28
7.3	Solved Numerical Example	29
8	Edge Detection	31
8.1	Intuitive Theory and Definition	31
8.2	Mathematical Formulation and Derivation	32
8.2.1	The image gradient	32
8.2.2	Derivative of Gaussian: the crucial trick	32
8.2.3	The Canny edge detector	33
8.3	Solved Numerical Example	34
9	Corner Detection: The Harris Detector	37
9.1	Intuitive Theory and Definition	37
9.2	Mathematical Formulation and Derivation	37
9.2.1	From “appearance change” to a matrix	37
9.2.2	Interpreting the eigenvalues	38
9.2.3	The Harris response function	39
9.2.4	The full algorithm and its invariances	39
9.3	Solved Numerical Example	40
II	Geometric Transformations of Images	43
10	Image Manipulation: Range versus Domain	44
10.1	Intuitive Theory and Definition	44
10.2	Mathematical Formulation	45
10.3	Solved Numerical Example	45

11	2×2 Linear Transformations	47
11.1	Intuitive Theory and Definition	47
11.2	Mathematical Formulation	47
11.2.1	The four canonical 2×2 matrices	47
11.2.2	What the determinant tells you	48
11.2.3	The decisive limitation	49
11.3	Solved Numerical Example	49
12	Homogeneous Coordinates	51
12.1	Intuitive Theory and Definition	51
12.2	Mathematical Formulation and Derivation	51
12.2.1	Translation becomes a matrix	52
12.2.2	Embedding the linear transformations	52
12.3	Solved Numerical Example	53
13	The Transformation Hierarchy	55
13.1	Intuitive Theory and Definition	55
13.2	Mathematical Formulation	55
13.2.1	Level 1: Translation (2 DoF)	55
13.2.2	Level 2: Rigid / Euclidean (3 DoF)	55
13.2.3	Level 3: Similarity (4 DoF)	56
13.2.4	Level 4: Affine (6 DoF)	56
13.2.5	Level 5: Projective / homography (8 DoF)	56
13.3	Solved Numerical Example	57
14	Projective Transformations and the Homography	59
14.1	Intuitive Theory and Definition	59
14.2	Mathematical Formulation and Derivation	59
14.2.1	Why eight degrees of freedom, not nine	60
14.2.2	Expanding the division	60
14.2.3	Vanishing points from the homogeneous algebra	60
14.2.4	Composition of homographies	61
14.3	Solved Numerical Example	61
15	Forward and Backward Image Warping	63
15.1	Intuitive Theory and Definition	63
15.2	Mathematical Formulation	64
15.3	Solved Numerical Example	65

III	Image Formation and the Camera Model	67
16	Image Formation and Perspective Projection	68
16.1	Intuitive Theory and Definition	68
16.2	Mathematical Formulation and Derivation	68
16.2.1	The perspective projection equations	68
16.2.2	Magnification	69
16.2.3	Vanishing points, derived properly	69
16.3	Solved Numerical Example	70
17	The Linear Camera Model	73
17.1	Intuitive Theory and Definition	73
17.2	Mathematical Formulation and Derivation	73
17.2.1	Stage 3 first: image plane to sensor	73
17.2.2	Removing the division with homogeneous coordinates	74
17.2.3	Stage 1: world to camera (extrinsics)	74
17.2.4	The complete projection matrix	75
17.3	Solved Numerical Example	76
IV	The Optimisation Toolkit	79
18	Constrained Optimisation and Lagrange Multipliers	80
18.1	Intuitive Theory and Definition	80
18.2	Mathematical Formulation and Derivation	81
18.2.1	Deriving the condition	81
18.2.2	The vector form used throughout the rest of the course	82
18.3	Solved Numerical Examples	82
V	Fitting: Lines, Curves and Robustness	88
19	Least-Squares Fitting of Lines and Curves	89
19.1	Intuitive Theory and Definition	89
19.2	Mathematical Formulation and Derivation	89
19.2.1	Line fitting by ordinary least squares	89
19.2.2	The matrix formulation — the version that generalises	91
19.2.3	Parabola and polynomial fitting	91
19.3	Solved Numerical Example	92
20	Total Least Squares	94
20.1	Intuitive Theory and Definition	94
20.2	Mathematical Formulation and Derivation	94

20.2.1	A line parameterisation that admits vertical lines	94
20.2.2	Eliminating d	95
20.2.3	The eigenvalue solution	95
20.3	Solved Numerical Example	96
20.3.1	Fitting curves to edge points: the general over-determined system . . .	98
21	Robust Fitting and RANSAC	100
21.1	Intuitive Theory and Definition	100
21.2	Mathematical Formulation	100
21.2.1	The three parameters	101
21.2.2	Deriving the number of iterations	101
21.2.3	Robust loss functions — the alternative	102
21.3	Solved Numerical Example	102
22	The Hough Transform	106
22.1	Intuitive Theory and Definition	106
22.2	Mathematical Formulation	106
22.2.1	Why not (m, b) space	106
22.2.2	The polar (normal) parameterisation	107
22.2.3	The generalised Hough transform	107
22.3	Solved Numerical Example	108
VI	Image Alignment	111
23	Image Alignment	112
23.1	Intuitive Theory and Definition	112
23.2	Mathematical Formulation and Derivation	112
23.2.1	Fitting an affine transformation	112
23.2.2	Fitting a homography — and why it is harder	113
23.2.3	Getting the correspondences: descriptors and matching	114
23.3	Solved Numerical Examples	115
VII	Camera Calibration	121
24	Camera Calibration	122
24.1	Intuitive Theory and Definition	122
24.2	Mathematical Formulation and Derivation	122
24.2.1	Setting up the equations	122
24.2.2	How many points?	123
24.2.3	Fixing the scale: two options	124

24.2.4	Decomposing $\mathbf{P}$ into $\mathbf{K}$, $\mathbf{R}$ and $\mathbf{t}$	124
24.2.5	What the linear method leaves out	125
24.3	Solved Numerical Example	126

VIII Appendices 130

A Linear Algebra Refresher 131

A.1	Eigenvalues and eigenvectors	131
A.2	The singular value decomposition	132
A.3	The pseudo-inverse	132
A.4	Matrix calculus identities	132
A.5	Quick 2×2 and 3×3 formulas	133

B Master Formula Sheet 134

B.1	Image processing	134
B.2	Geometric transformations	135
B.3	Camera geometry	135
B.4	Fitting and robustness	136
B.5	The one decision tree you will need	136

C All Exam Pitfalls, Collected 137

Notation Used Throughout

Mathematics in computer vision is unforgiving about the difference between a point, a vector of unknowns, and a matrix. This guide uses one convention everywhere.

Symbol	Meaning
$f(x, y)$	A continuous image: a function mapping position to intensity.
$I[i, j]$	A <i>digital</i> image: intensity stored at row i , column j .
$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$	Bold lowercase is a column vector (a point, a parameter vector).
$\tilde{\mathbf{x}} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$	A tilde marks a point written in <i>homogeneous</i> coordinates.
$\mathbf{A}, \mathbf{H}, \mathbf{K}$	Bold uppercase is a matrix.
$\mathbf{A}^\top$	Transpose.
$\mathbf{A}^{-1}$	Inverse.
$\ \mathbf{x}\ $	Euclidean (ℓ_2) length, $\sqrt{x^2 + y^2}$.
∇f	Gradient, $\begin{bmatrix} \partial f / \partial x & \partial f / \partial y \end{bmatrix}^\top$.
$h * f$	h convolved with f .
λ_1, λ_2	Eigenvalues, ordered $\lambda_1 \geq \lambda_2$ unless stated.
$\mathbf{P}_w$	A point in <i>world</i> coordinates (3D).
$\mathbf{P}_c$	The same point in <i>camera</i> coordinates (3D).
(u, v)	Pixel coordinates in the image sensor.
$\sim$	“Equal up to a non-zero scale factor” (used for homogeneous quantities).

EXAM PITFALL: Row-column versus x - y

An image array is indexed $I[\text{row}, \text{column}] = I[i, j]$, but geometry is written (x, y) with x horizontal and y vertical. So x corresponds to the *column* index j and y to the *row* index i . Mixing these up transposes your image and is the single most common source of “my warp came out sideways” in a lab exam. Whenever you write a transformation matrix, state which convention you are using.

KEY DEFINITION: How to read the callout boxes

Blue boxes carry definitions you should be able to recite. **Gold boxes** carry formulae worth memorising verbatim. **Red boxes** carry the specific mistakes that cost marks. **Green boxes** carry the physical analogy that makes the mathematics stick. **Outlined boxes** carry a complete numerical example with every arithmetic step shown.

PART I

Foundations: Images, Filtering and Early Vision

What Computer Vision Is, and What an Image Really Is

1.1 Intuitive Theory and Definition

Computer vision asks a deceptively simple question: given an array of numbers produced by a camera, can a machine recover *what is out there in the world*? Graphics runs the arrow one way — from a 3D scene to a 2D picture. Vision runs it backwards — from a 2D picture back to the 3D scene. That backwards direction is the whole difficulty, because the forward direction throws information away. A camera flattens three dimensions into two, and once depth is gone, infinitely many different scenes could have produced exactly the same picture.

PHYSICAL ANALOGY: Vision as inverse graphics

Think of a shadow puppet show. Making the shadow from your hand is easy and deterministic: that is graphics. Looking only at the shadow on the wall and reconstructing the exact shape, distance and orientation of the hand is hard and ambiguous: that is vision. A rabbit-shaped shadow could come from a hand, from a cardboard cutout, or from an actual rabbit. The shadow does not tell you which.

KEY DEFINITION: Computer vision

The construction of explicit, meaningful descriptions of physical objects from images. “Explicit” means a machine-usable description — *there is a cup at (x, y, z) rotated by θ* — not merely a pretty picture.

Images are **fundamentally ambiguous**. The same pixel intensities can be explained by a bright surface in shadow or a dark surface in light; by a small nearby object or a large distant one. Vision algorithms therefore never simply “read off” the answer. They impose extra assumptions — smoothness, rigidity, a known calibration object, a known transformation family — to cut the infinite set of explanations down to one. Almost every technique in this guide is, at bottom, a way of adding an assumption and then solving the resulting constrained problem.

1.1.1 The four pillars of the course

The introductory lecture organises everything that follows into four blocks. Keep this map in your head; it tells you where any given topic sits.

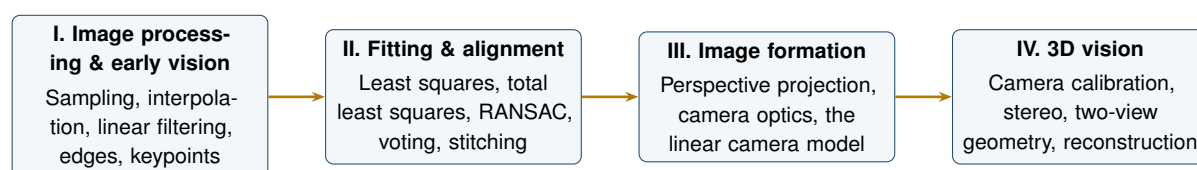


Figure 1.1: The four topic blocks of the course. Blocks I and II are “2D” — they act on pixels and on point sets in the image plane. Blocks III and IV re-introduce the third dimension. Your midterm material spans I, II and the entry point into III/IV (calibration).

1.1.2 An image is a function

The single most useful mental shift in this course: stop thinking of an image as a picture and start thinking of it as a *function*.

KEY DEFINITION: Image as a function

A grayscale image is a function

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad (x, y) \mapsto f(x, y),$$

mapping a spatial position (x, y) to an intensity (brightness) value. A colour image is a vector-valued function $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$, returning (R, G, B) at each position. A *digital* image is this function *sampled* on a finite grid and *quantised* to a finite set of values, typically the integers 0–255 for 8-bit storage.

PHYSICAL ANALOGY: A landscape of brightness

Picture a grayscale image as a hilly terrain seen from above: bright pixels are mountain peaks, dark pixels are valleys. Blurring flattens the terrain. Edge detection finds the cliffs. Corner detection finds the isolated peaks and saddle points. Once you see the image as terrain, “take the derivative of the image” stops being strange — it is just asking how steep the hillside is.

Practical consequences of the sampled-function view, straight from the Nayar material: a camera sensor is a rectangular array of photosites; each photosite integrates light over a small area and reports one number; that number is the sample. Colour cameras do not measure three numbers per photosite. They place a **Bayer mosaic** of red, green and blue filters over the sensor so each photosite sees only one colour band, then **demosaic** — interpolate the two missing channels at every pixel from its neighbours. Demosaicing is therefore the very first interpolation problem a vision system solves, before any “vision” has happened at all.

[INSERT IMAGE HERE: Reference lec02_image_processing, slides 4–8 — “Images as sampled functions” and “Digital color images”]

The slides show a photograph, a zoomed crop of the same photograph revealing the individual pixel squares, and the same crop overlaid with the raw integer intensity values (e.g. 157, 159, 159, 104, ...). A companion figure shows the Bayer RGGB filter mosaic laid over the photosite grid, with the missing colour samples marked as question marks to motivate demosaicing.

1.2 Mathematical Formulation

Let the continuous scene radiance be $f(x, y)$. A digital camera produces

$$I[i, j] = Q\left(\iint_{A_{ij}} f(x, y) \, dx \, dy\right), \quad i = 0, \dots, M-1, \, j = 0, \dots, N-1,$$

where A_{ij} is the physical area of photosite (i, j) and $Q(\cdot)$ is the quantiser that rounds the collected energy to an integer in $\{0, 1, \dots, 255\}$.

Two distinct operations are hidden in that one line, and the whole of image processing splits along the seam between them:

IMPORTANT FORMULA: Range versus domain

Every image operation changes either the *range* of f or its *domain*.

$$\text{Range change (intensities):} \quad g(x, y) = T_r(f(x, y)) \quad (1.1)$$

$$\text{Domain change (positions):} \quad g(x, y) = f(T_d(x, y)) \quad (1.2)$$

The first family is *point processing and filtering*. The second family is *warping and geometric transformation*. Topics 2–9 of this guide live in the first family; Topics 10–15 live in the second.

Source deck: 1__2D_Transformations, slides 1–3

1.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Storage cost and quantisation error

Problem. A camera produces a 1920×1080 colour image with 8 bits per channel. (a) How many bytes does the raw uncompressed image occupy? (b) The sensor at one photosite collects a normalised energy of 0.7137 on a scale where 1.0 is full well capacity. What integer is stored, and what is the quantisation error?

Step 1: Count the samples.

Each pixel stores three channels. The number of stored values is

$$N_{\text{vals}} = 1920 \times 1080 \times 3 = 2\,073\,600 \times 3 = 6\,220\,800.$$

Step 2: Convert to bytes.

At 8 bits (= 1 byte) per value,

$$N_{\text{bytes}} = 6\,220\,800 \text{ bytes} = \frac{6\,220\,800}{1024^2} \text{ MiB} \approx 5.93 \text{ MiB}.$$

Step 3: Quantise the sample.

An 8-bit quantiser maps $[0, 1]$ onto $\{0, \dots, 255\}$ with

$$Q(e) = \text{round}(255e).$$

Here $255 \times 0.7137 = 181.9935$, so $Q = \text{round}(181.9935) = 182$.

Step 4: Quantisation error.

The stored value corresponds to a reconstructed energy of $182/255 = 0.713725\dots$, so

$$\varepsilon = |0.7137 - 0.713725| = 2.5 \times 10^{-5}.$$

Expressed on the integer scale, $|181.9935 - 182| = 0.0065$ levels. The worst possible error of any 8-bit quantiser is half a level, $0.5/255 \approx 1.96 \times 10^{-3}$.

Answer.

5.93 MiB; the stored value is 182 with error 2.5×10^{-5} , comfortably under the 1.96×10^{-3} worst case.

EXAM PITFALL: “The image is just data”

Never treat pixel values as exact. They are area-integrated, quantised and noisy. This is precisely why later topics (least squares, total least squares, RANSAC) exist: every measurement you extract from an image carries error, so you fit models rather than solve exact equations.

Point Processing: Changing the Range

2.1 Intuitive Theory and Definition

Point processing is the simplest possible image operation: look at one pixel, compute a new value from *that pixel alone*, write it out. No neighbour is consulted. Brightening, darkening, inverting, contrast stretching and gamma correction are all point processes.

PHYSICAL ANALOGY: A lookup table on a dial

Imagine a dial that takes any input brightness and returns an output brightness — a machine with one knob and no memory. Feed it 100, it always returns the same number, no matter where in the image that 100 came from. Because the machine has no memory of neighbours, a point process can never blur, sharpen or detect an edge. It can only re-map the grey levels.

KEY DEFINITION: Point process

An operation of the form $g(x, y) = T_r(f(x, y))$ where $T_r : \mathbb{R} \rightarrow \mathbb{R}$ depends only on the intensity value, not on the position (x, y) or on any neighbouring value.

The Nayar slides demonstrate exactly three canonical examples: *darken* ($f - 128$), *low contrast* ($f/2$), and the *negative* ($255 - f$). The shape of the mapping curve tells you at a glance what the operation does.

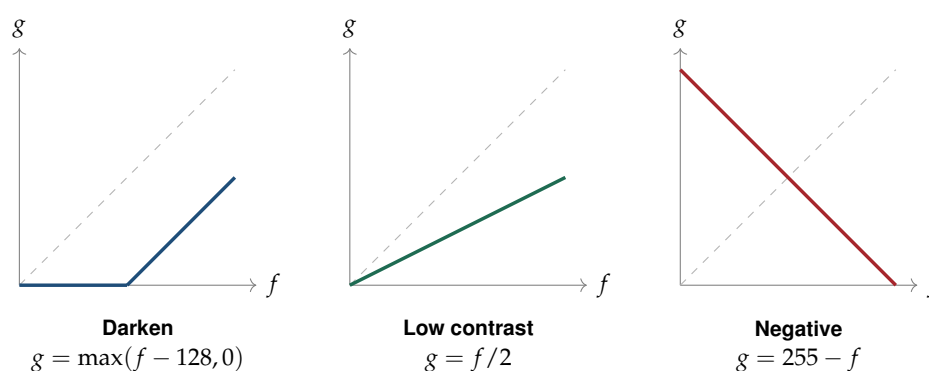


Figure 2.1: Transfer curves for three point processes. The dashed 45° line is the identity. Anything *below* it darkens; anything *flatter* than it reduces contrast; a *negative* slope inverts.

Source deck: *s0_0_Image_Processing_intro*, slides 7–9

2.2 Mathematical Formulation

IMPORTANT FORMULA: The four standard point transforms

$$\text{Brightness shift: } g = f + b, \quad b \in \mathbb{R} \quad (2.1)$$

$$\text{Contrast scale: } g = a f, \quad a > 0 \quad (2.2)$$

$$\text{Affine (both): } g = a f + b \quad (2.3)$$

$$\text{Gamma correction: } g = 255 \left(\frac{f}{255} \right)^\gamma \quad (2.4)$$

Every result must finally be **clipped** to the displayable range: $g \leftarrow \min(255, \max(0, g))$.

2.2.1 Deriving contrast stretching step by step

Suppose an image only uses grey levels between $f_{\min}$ and $f_{\max}$ and looks washed out. We want a linear map that sends $f_{\min} \mapsto 0$ and $f_{\max} \mapsto 255$.

Step 1: Write the general linear form.

$$g = a f + b.$$

Step 2: Impose the two endpoint conditions.

$$a f_{\min} + b = 0 \quad (2.5)$$

$$a f_{\max} + b = 255 \quad (2.6)$$

Step 3: Subtract the first equation from the second to eliminate b .

$$a(f_{\max} - f_{\min}) = 255 \implies a = \frac{255}{f_{\max} - f_{\min}}.$$

Step 4: Back-substitute into the first equation.

$$b = -a f_{\min} = -\frac{255 f_{\min}}{f_{\max} - f_{\min}}.$$

Step 5: Assemble.

IMPORTANT FORMULA: Contrast stretch

$$g(x, y) = 255 \cdot \frac{f(x, y) - f_{\min}}{f_{\max} - f_{\min}}$$

Note what this derivation demonstrates in miniature, because you will repeat the pattern for the rest of the course: *write the model with unknown parameters, impose one equation per known correspondence, solve the linear system*. Affine fitting, homography fitting and camera calibration are all this same move at larger scale.

2.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Contrast stretch and gamma on a 3×3 patch

Problem. A 3×3 patch has intensities

$$I = \begin{bmatrix} 100 & 110 & 105 \\ 120 & 130 & 118 \\ 102 & 140 & 125 \end{bmatrix}.$$

(a) Apply a full contrast stretch. (b) Apply gamma correction with $\gamma = 0.5$ to the centre pixel of the *original* patch.

Step 1: Find the range.

Scanning all nine values: $f_{\min} = 100$, $f_{\max} = 140$, so $f_{\max} - f_{\min} = 40$.

Step 2: Build the mapping.

$$a = \frac{255}{40} = 6.375, \quad b = -6.375 \times 100 = -637.5, \quad g = 6.375f - 637.5.$$

Step 3: Apply to each entry.

$$f = 100 \rightarrow 6.375(100) - 637.5 = 637.5 - 637.5 = 0 \quad (2.7)$$

$$f = 110 \rightarrow 701.25 - 637.5 = 63.75 \rightarrow 64 \quad (2.8)$$

$$f = 105 \rightarrow 669.375 - 637.5 = 31.875 \rightarrow 32 \quad (2.9)$$

$$f = 120 \rightarrow 765 - 637.5 = 127.5 \rightarrow 128 \quad (2.10)$$

$$f = 130 \rightarrow 828.75 - 637.5 = 191.25 \rightarrow 191 \quad (2.11)$$

$$f = 118 \rightarrow 752.25 - 637.5 = 114.75 \rightarrow 115 \quad (2.12)$$

$$f = 102 \rightarrow 650.25 - 637.5 = 12.75 \rightarrow 13 \quad (2.13)$$

$$f = 140 \rightarrow 892.5 - 637.5 = 255 \quad (2.14)$$

$$f = 125 \rightarrow 796.875 - 637.5 = 159.375 \rightarrow 159 \quad (2.15)$$

Step 4: Assemble the output.

$$g = \begin{bmatrix} 0 & 64 & 32 \\ 128 & 191 & 115 \\ 13 & 255 & 159 \end{bmatrix}.$$

The patch now spans the full range: contrast has increased by the factor 6.375.

Step 5: Gamma on the centre pixel.

The centre of the original patch is $f = 130$. With $\gamma = 0.5$,

$$g = 255 \left(\frac{130}{255} \right)^{0.5} = 255 \sqrt{0.50980} = 255 \times 0.71401 = 182.07 \rightarrow 182.$$

Answer.

The stretched patch is shown above; gamma-0.5 maps $130 \mapsto 182$, i.e. $\gamma < 1$ *brightens* midtones.

EXAM PITFALL: Clipping is not optional, and it is not invertible

Students routinely forget the min / max clip and report intensities such as -37 or 291 . Worse, once a value clips, the original is gone: $\max(f - 128, 0)$ maps every input below 128 onto 0, so darkening then re-brightening does *not* recover the original image. Point processes are only invertible on the unclipped interior.

Sampling, Aliasing and the Nyquist–Shannon Theorem

3.1 Intuitive Theory and Definition

A continuous scene contains detail at all scales. A sensor has a finite number of photosites. Sampling is the act of measuring the continuous function only at grid points, and it is where information is irretrievably destroyed — not by noise, but by geometry.

PHYSICAL ANALOGY: The wagon wheel in an old film

In a Western, the stagecoach speeds up and the wheel spokes appear to slow, stop, then spin *backwards*. Nothing is wrong with the wheel. The camera samples 24 frames per second; when the spokes rotate almost one full spoke-spacing between frames, the eye interprets the short residual motion as a slow reverse rotation. The high-frequency truth has *masqueraded* as a low-frequency lie. That masquerade is aliasing, and it happens in space exactly as it happens in time.

KEY DEFINITION: Spatial frequency

How rapidly intensity changes with position, measured in cycles per pixel or cycles per image width. A smooth sky is low frequency. A brick wall, a striped shirt, or a sharp edge is high frequency. An *edge* is not a single frequency: an ideal step contains energy at *all* frequencies, which is why edges are the first thing to break under bad sampling.

KEY DEFINITION: Aliasing

The phenomenon in which a signal component with frequency above half the sampling rate is indistinguishable, after sampling, from a lower-frequency component. High frequencies take on a false (“alias”) low-frequency identity. In images this appears as moiré patterns on fabric, jagged staircase edges, and spurious ring patterns on brickwork.

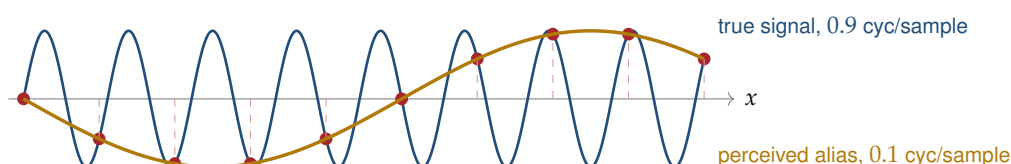


Figure 3.1: Aliasing in one dimension. The blue signal oscillates at 0.9 cycles per sample. Sampling at integer positions (red dots) produces exactly the same dot pattern that a 0.1 cycles-per-sample signal (gold) would produce. After sampling, the two are mathematically indistinguishable, so the reconstruction picks the slow one.

Source deck: *lec02_image_processing*, slides 10–23

[INSERT IMAGE HERE: Reference lec02_image_processing, slides 18–21 —
“Aliasing in images” and “Aliasing in neural networks”]

Photographic examples: a checkerboard receding toward the horizon which degenerates into a grey mush plus spurious diagonal moiré bands in the far field; a striped shirt on a television broadcast showing rainbow interference patterns; and a pair of CNN feature maps showing that the same aliasing artefacts propagate into learned representations when a network downsamples without an anti-aliasing filter.

3.2 Mathematical Formulation and Derivation

IMPORTANT FORMULA: Nyquist–Shannon sampling theorem

Let $f(x)$ be band-limited, meaning its Fourier transform satisfies $F(\omega) = 0$ for all $|\omega| > \omega_{\max}$. Then f can be reconstructed *exactly* from its samples if and only if the sampling frequency satisfies

$$\omega_s > 2\omega_{\max}.$$

The threshold $2\omega_{\max}$ is the **Nyquist rate**; the quantity $\omega_s/2$ is the **Nyquist frequency** or *folding frequency*.

3.2.1 Why the factor of two? A step-by-step argument

Step 1: Sampling is multiplication by an impulse train.

Sampling at spacing Δ means multiplying $f(x)$ by $\text{III}(x) = \sum_{n=-\infty}^{\infty} \delta(x - n\Delta)$.

Step 2: Multiplication in space is convolution in frequency.

By the convolution theorem, if $f \leftrightarrow F$ and $\text{III} \leftrightarrow \text{III}$ (an impulse train transforms to an impulse train with spacing $\omega_s = 2\pi/\Delta$), then

$$f(x) \text{III}(x) \longleftrightarrow \frac{1}{\Delta} F(\omega) * \sum_k \delta(\omega - k\omega_s) = \frac{1}{\Delta} \sum_{k=-\infty}^{\infty} F(\omega - k\omega_s).$$

Step 3: Read the consequence.

The spectrum of the sampled signal is the original spectrum *replicated* at every multiple of ω_s . Each copy occupies the band $[k\omega_s - \omega_{\max}, k\omega_s + \omega_{\max}]$.

Step 4: Demand that the copies do not touch.

The $k = 0$ copy ends at $\omega_{\max}$; the $k = 1$ copy begins at $\omega_s - \omega_{\max}$. Non-overlap requires

$$\omega_s - \omega_{\max} > \omega_{\max} \implies \omega_s > 2\omega_{\max}.$$

Step 5: Interpret the failure case.

If $\omega_s < 2\omega_{\max}$ the replicas overlap. Energy from the $k = 1$ copy lands *inside* the baseband and is, after sampling, arithmetically identical to genuine low-frequency content. No reconstruction filter can separate them again. ■

IMPORTANT FORMULA: The alias frequency

If a signal of frequency ω_0 is sampled at ω_s with $\omega_s/2 < \omega_0 < \omega_s$, the observed (aliased) frequency is

$$\omega_{\text{alias}} = |\omega_0 - \omega_s|.$$

More generally, $\omega_{\text{alias}} = \min_k |\omega_0 - k\omega_s|$.

3.2.2 Anti-aliasing: the only cure

You cannot undo aliasing after the fact. The cure is to **remove the high frequencies before you sample**, by blurring. This is why the correct downsampling pipeline is *always* low-pass filter *then* discard pixels, never the reverse.

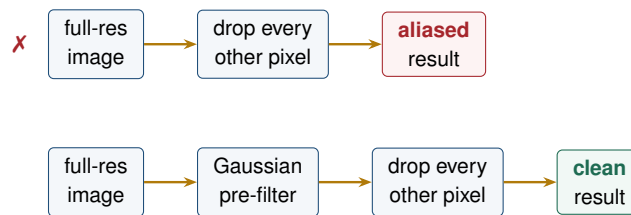


Figure 3.2: The two downsampling pipelines. Subsampling *without* pre-filtering (top) folds high frequencies into the baseband. Pre-filtering (bottom) deletes those frequencies first, so nothing is left to fold.

Source deck: *lec02_image_processing*, slides 25–29

3.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Predicting the moiré pattern on a striped shirt

Problem. A shirt has vertical stripes with a period of 4 mm. It is photographed so that 1 mm of shirt maps onto 0.6 pixels. (a) What is the stripe frequency in cycles per pixel? (b) Does aliasing occur? (c) If so, what apparent stripe period, in pixels, will appear in the photograph? (d) What Gaussian pre-filter would prevent it?

Step 1: Convert the stripe period to pixels.

One stripe period is 4 mm of shirt, and $1\text{ mm} \rightarrow 0.6\text{ pixels}$, so

$$\text{period} = 4 \times 0.6 = 2.4\text{ pixels per cycle.}$$

Step 2: Convert to frequency.

$$\omega_0 = \frac{1}{2.4} = 0.41\bar{6}\text{ cycles per pixel.}$$

Step 3: Compare with the Nyquist frequency.

Sampling is one sample per pixel, so $\omega_s = 1.0$ cycles per pixel and the Nyquist frequency is

$$\frac{\omega_s}{2} = 0.5\text{ cycles per pixel.}$$

Since $0.4167 < 0.5$, the *fundamental* stripe frequency is safely below Nyquist. **So far, no aliasing.**

Step 4: Do not stop at the fundamental — check the harmonics.

Real stripes are square waves, not sinusoids. A square wave of fundamental ω_0 contains odd harmonics at $3\omega_0, 5\omega_0, \dots$

$$3\omega_0 = 1.25\text{ cyc/px}, \quad 5\omega_0 = 2.083\text{ cyc/px.}$$

Both exceed 0.5, so both alias.

Step 5: Compute the alias of the third harmonic.

$$\omega_{\text{alias}} = \min_k |1.25 - k(1.0)| = |1.25 - 1| = 0.25\text{ cycles per pixel.}$$

Step 6: Convert the alias back to a visible period.

$$\text{apparent period} = \frac{1}{0.25} = 4 \text{ pixels.}$$

A broad 4-pixel banding will appear over the true 2.4-pixel stripes — the moiré.

Step 7: Design the pre-filter.

We must attenuate everything above 0.5 cyc/px while preserving the 0.4167 fundamental. A Gaussian of spatial standard deviation σ has frequency response $\exp(-2\pi^2\sigma^2\omega^2)$. Requiring the third harmonic to be attenuated to 1%:

$$\exp(-2\pi^2\sigma^2(1.25)^2) = 0.01 \quad (3.1)$$

$$-2\pi^2\sigma^2(1.5625) = \ln 0.01 = -4.605 \quad (3.2)$$

$$\sigma^2 = \frac{4.605}{2\pi^2(1.5625)} = \frac{4.605}{30.843} = 0.1493 \quad (3.3)$$

$$\sigma = 0.386 \text{ pixels.} \quad (3.4)$$

Check the fundamental survives: $\exp(-2\pi^2(0.1493)(0.4167)^2) = \exp(-0.512) = 0.60$, i.e. 60% of the stripe contrast is retained.

Answer.

$\omega_0 = 0.417$ cyc/px (safe), but the third harmonic at 1.25 cyc/px aliases down to 0.25 cyc/px, producing false 4-pixel bands. A Gaussian with $\sigma \approx 0.39$ px applied *before* sampling suppresses the harmonic to 1% while retaining 60% of the true stripe contrast.

EXAM PITFALL: Nyquist is about the *maximum* frequency present

Two classic errors. First, comparing the sampling rate to the *dominant* frequency instead of the *highest* frequency — harmonics and edges are what actually alias. Second, stating the condition as $\omega_s > \omega_{\max}$ instead of $\omega_s > 2\omega_{\max}$. Remember the replica argument: you need room for a full mirrored copy on each side.

Resampling and Interpolation

4.1 Intuitive Theory and Definition

Downsampling asks which samples to throw away. **Upsampling** asks the harder question: what value belongs at a position where you never measured anything? The answer is *interpolation* — inventing a continuous function that agrees with your samples, then evaluating it wherever you like.

PHYSICAL ANALOGY: Reading between the fence posts

You measure the height of a hillside at every fence post, one metre apart. Someone asks how high the ground is 0.3 m past the fourth post. You did not measure there. Nearest neighbour says: use post 4's height, full stop — fast, but the hillside becomes a staircase. Linear interpolation says: draw a straight line between posts 4 and 5 and read off the value at 0.3 — smooth, and almost always what you want. Bilinear interpolation is the same idea when the fence posts form a *grid* rather than a line.

KEY DEFINITION: Interpolation

Reconstructing a continuous function from discrete samples by weighting nearby samples. Formally, $f(x) = \sum_k I[k] h(x - k)$, where h is an **interpolation kernel** or basis function. Different choices of h give nearest-neighbour (h a box), linear (h a triangle), or bicubic (h a cubic spline) interpolation.

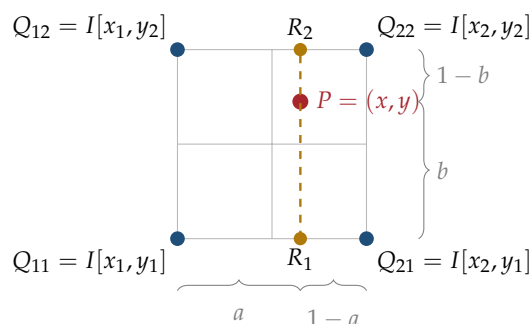


Figure 4.1: Bilinear interpolation. Interpolate horizontally along the bottom edge to get R_1 and along the top edge to get R_2 , then interpolate vertically between R_1 and R_2 . The order does not matter — doing it vertically first gives the same answer.

Source deck: *lec02_image_processing*, slides 33–39

4.2 Mathematical Formulation and Derivation

Let the query point $P = (x, y)$ lie inside the unit cell with corners $(x_1, y_1), (x_2, y_1), (x_1, y_2), (x_2, y_2)$ where $x_2 = x_1 + 1$ and $y_2 = y_1 + 1$. Define the fractional offsets

$$a = x - x_1 \in [0, 1], \quad b = y - y_1 \in [0, 1].$$

Step 1: Interpolate along the bottom edge ($y = y_1$).

Linear interpolation between Q_{11} at $a = 0$ and Q_{21} at $a = 1$:

$$R_1 = (1 - a) Q_{11} + a Q_{21}.$$

Step 2: Interpolate along the top edge ($y = y_2$).

$$R_2 = (1 - a) Q_{12} + a Q_{22}.$$

Step 3: Interpolate vertically between R_1 and R_2 .

$$P = (1 - b) R_1 + b R_2.$$

Step 4: Substitute Steps 1 and 2 into Step 3.

$$P = (1 - b) [(1 - a) Q_{11} + a Q_{21}] + b [(1 - a) Q_{12} + a Q_{22}] \quad (4.1)$$

$$= (1 - a)(1 - b) Q_{11} + a(1 - b) Q_{21} + (1 - a)b Q_{12} + ab Q_{22}. \quad (4.2)$$

IMPORTANT FORMULA: Bilinear interpolation

$$f(x, y) = (1 - a)(1 - b) Q_{11} + a(1 - b) Q_{21} + (1 - a)b Q_{12} + ab Q_{22}$$

with $a = x - \lfloor x \rfloor$, $b = y - \lfloor y \rfloor$. The four weights are non-negative and sum to one:

$$(1 - a)(1 - b) + a(1 - b) + (1 - a)b + ab = (1 - b) [(1 - a) + a] + b [(1 - a) + a] \quad (4.3)$$

$$= (1 - b) + b = 1. \quad (4.4)$$

Each weight equals the *area of the sub-rectangle diagonally opposite its corner* — a useful sanity check.

Step 5: Note the matrix form (useful for exams).

$$f(x, y) = \begin{bmatrix} 1 - a & a \end{bmatrix} \begin{bmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \end{bmatrix} \begin{bmatrix} 1 - b \\ b \end{bmatrix}.$$

4.2.1 The basis-function view

Expanding the product shows bilinear interpolation is separable: it is the tensor product of two 1-D triangular (“hat”) kernels,

$$h(t) = \begin{cases} 1 - |t|, & |t| \leq 1 \\ 0, & \text{otherwise} \end{cases} \implies f(x, y) = \sum_i \sum_j I[i, j] h(x - i) h(y - j).$$

Replacing h with a wider, smoother kernel gives bicubic or Lanczos interpolation: more accurate, more expensive, and capable of **overshoot**.

EXAM PITFALL: Interpolation and function extrema

Bilinear interpolation can never produce a value outside the range of the four corner samples, because the weights are non-negative and sum to one. **Bicubic can.** If you bicubically upsample an image with a sharp edge you will see values below 0 or above

255 — ringing. A related exam point: the interpolated surface's maximum generally does *not* sit at a sample location, which is exactly why sub-pixel peak localisation (used in corner and edge detection) works at all.

4.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Bilinear resampling of a 2×2 neighbourhood

Problem. An image has the values

$$I[0,0] = 10, \quad I[1,0] = 20, \quad I[0,1] = 30, \quad I[1,1] = 60$$

(using $I[x,y]$, x horizontal). Find the bilinearly interpolated intensity at $(x,y) = (0.25, 0.75)$. Then find it at $(0.5, 0.5)$ and compare with nearest-neighbour.

Step 1: Identify the cell and the offsets.

The query lies in the cell with $x_1 = 0, y_1 = 0$, so

$$a = 0.25 - 0 = 0.25, \quad b = 0.75 - 0 = 0.75.$$

The corner values in the notation of the formula are

$$Q_{11} = I[0,0] = 10, \quad Q_{21} = I[1,0] = 20, \quad Q_{12} = I[0,1] = 30, \quad Q_{22} = I[1,1] = 60.$$

Step 2: Compute the four weights.

$$w_{11} = (1 - a)(1 - b) = (0.75)(0.25) = 0.1875 \quad (4.5)$$

$$w_{21} = a(1 - b) = (0.25)(0.25) = 0.0625 \quad (4.6)$$

$$w_{12} = (1 - a)b = (0.75)(0.75) = 0.5625 \quad (4.7)$$

$$w_{22} = ab = (0.25)(0.75) = 0.1875 \quad (4.8)$$

Check: $0.1875 + 0.0625 + 0.5625 + 0.1875 = 1.0000$. ✓

Step 3: Weighted sum.

$$f(0.25, 0.75) = 0.1875(10) + 0.0625(20) + 0.5625(30) + 0.1875(60) \quad (4.9)$$

$$= 1.875 + 1.25 + 16.875 + 11.25 \quad (4.10)$$

$$= 31.25. \quad (4.11)$$

Step 4: Verify by the two-stage route.

Bottom edge: $R_1 = (1 - 0.25)(10) + 0.25(20) = 7.5 + 5 = 12.5$. Top edge: $R_2 = (1 - 0.25)(30) + 0.25(60) = 22.5 + 15 = 37.5$. Vertical: $f = (1 - 0.75)(12.5) + 0.75(37.5) = 3.125 + 28.125 = 31.25$. ✓

Step 5: The centre point $(0.5, 0.5)$.

All four weights equal 0.25, so

$$f(0.5, 0.5) = \frac{1}{4}(10 + 20 + 30 + 60) = \frac{120}{4} = 30.$$

Bilinear at the exact centre is simply the average of the four corners.

Step 6: Compare with nearest neighbour.

At $(0.25, 0.75)$ the nearest sample is $I[0, 1] = 30$ (rounding $0.25 \rightarrow 0$ and $0.75 \rightarrow 1$), versus the bilinear 31.25. At $(0.5, 0.5)$ nearest-neighbour is ambiguous and typically rounds to $I[0, 0] = 10$ or $I[1, 1] = 60$ depending on tie-breaking — an error of up to 30 grey levels against the bilinear answer of 30.

Answer.

$f(0.25, 0.75) = 31.25$ and $f(0.5, 0.5) = 30$. Nearest-neighbour errs by 1.25 and by up to 30 levels respectively, which is why staircase artefacts appear on nearest-neighbour upsampling.

Linear Filtering: Correlation and Convolution

5.1 Intuitive Theory and Definition

A point process could not blur or sharpen, because it never looked at a pixel's neighbours. Linear filtering fixes that. Slide a small fixed-size window over the image; at every position, form a *weighted sum* of the pixels under the window; write that sum to the output. The pattern of weights is called the **kernel**, **mask** or **filter**, and it entirely determines what the operation does.

PHYSICAL ANALOGY: A stencil of weights

Picture a small square of transparent plastic with a number written in each of its nine cells. Lay it over the image so its centre sits on a pixel. Multiply each image value by the number printed above it, add up all nine products, and that total is the new value of the centre pixel. Slide the plastic one step right and repeat. If every printed number is $1/9$, you have computed a local average — blurring. If the numbers are $-1, -1, -1$ on one side and $+1, +1, +1$ on the other, you have computed a difference — edge detection. The *same machine* does both; only the printed numbers change.

KEY DEFINITION: Convolution kernel

A small matrix of weights $h[u, v]$ defining a linear, shift-invariant image operation. Its support (typically $3 \times 3, 5 \times 5, \dots$) is its *size*; the sum of its entries determines how the operation affects overall brightness. A kernel summing to 1 preserves mean brightness; a kernel summing to 0 produces a response centred on zero and is a *derivative-like* filter.

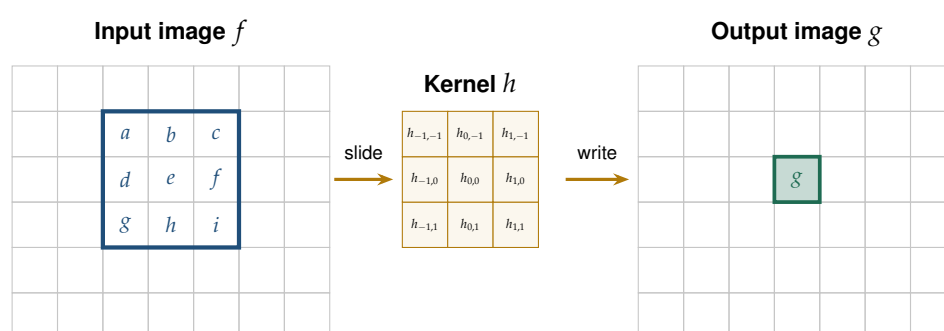


Figure 5.1: The sliding-window machine. Nine input values under the window are combined with nine kernel weights to produce a *single* output value at the corresponding centre position. The window then slides and the process repeats for every pixel.

Source deck: `lec03_filter`, slides 3–12

5.2 Mathematical Formulation

5.2.1 Correlation versus convolution

Two definitions differ only by a sign, and confusing them is a classic exam trap.

IMPORTANT FORMULA: Cross-correlation and convolution

$$\text{Cross-correlation: } g[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] f[i + u, j + v] \quad (5.1)$$

$$\text{Convolution: } g[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k h[u, v] f[i - u, j - v] \quad (5.2)$$

Convolution is correlation with the kernel **flipped through** 180° (flipped both left–right and top–bottom). If h is symmetric under that flip — as box, Gaussian and Laplacian kernels are — the two give identical results, which is why the distinction is often glossed over.

EXAM PITFALL: When the flip actually matters

The flip matters for *asymmetric* kernels, above all derivative filters. Correlating with $[-1 \ 0 \ 1]$ and convolving with $[-1 \ 0 \ 1]$ give answers of *opposite sign*. In an exam, state which operation you are performing. Deep-learning frameworks call their operation “convolution” but implement cross-correlation — harmless there because the weights are learned, fatal here because your weights are fixed by you.

5.2.2 Properties, proved

Linearity.

For scalars α, β and images f_1, f_2 :

$$h * (\alpha f_1 + \beta f_2)[i, j] = \sum_{u,v} h[u, v] (\alpha f_1 + \beta f_2)[i - u, j - v] \quad (5.3)$$

$$= \alpha \sum_{u,v} h[u, v] f_1[i - u, j - v] + \beta \sum_{u,v} h[u, v] f_2[i - u, j - v] \quad (5.4)$$

$$= \alpha (h * f_1)[i, j] + \beta (h * f_2)[i, j]. \quad \blacksquare \quad (5.5)$$

Shift invariance.

Let $f'[i, j] = f[i - i_0, j - j_0]$ be the image shifted by (i_0, j_0) . Then

$$(h * f')[i, j] = \sum_{u,v} h[u, v] f[i - i_0 - u, j - j_0 - v] = (h * f)[i - i_0, j - j_0]. \quad \blacksquare$$

Filtering a shifted image gives a shifted filtered image. An operation that is both linear and shift-invariant *must* be a convolution — this is why convolution is not one choice among many but *the* canonical local image operation.

Further algebraic properties.

Property	Statement	Why you care
Commutative	$f * h = h * f$	Kernel and image play symmetric roles
Associative	$(f * h_1) * h_2 = f * (h_1 * h_2)$	Pre-combine two blurs into one
Distributive	$f * (h_1 + h_2) = f * h_1 + f * h_2$	Used to derive sharpening
Identity	$f * \delta = f$	δ is the unit-impulse kernel
Differentiation	$\frac{\partial}{\partial x}(f * h) = f * \frac{\partial h}{\partial x}$	Basis of derivative-of-Gaussian

5.2.3 Boundary handling

At the image border the window hangs over the edge. Five standard remedies:

Method	Rule for missing pixels	Artefact
Zero padding	Treat outside as 0	Dark vignette at the border
Clip / valid	Do not compute; output shrinks	Output smaller than input
Wrap (circular)	Index modulo image size	Left edge bleeds into right
Clamp / replicate	Repeat the nearest border pixel	Mild streaking; usually fine
Mirror / reflect	Reflect across the border	Smoothest; standard in practice

Source deck: *lec03_filter*, slides 13–15

5.2.4 The standard kernel catalogue

$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$
Identity no change	Shift image moves left 1 px	Box blur $\times \frac{1}{9}$ local average	Sharpen boosts detail	Sobel x vertical edges

Figure 5.2: The five kernels every exam expects you to recognise on sight. Note the sums: identity, shift, normalised box blur and sharpen all sum to 1 and preserve mean brightness; Sobel sums to 0 and returns zero on any constant region.

Source deck: *lec03_filter*, slides 21–34; *s0_1_Spatial_Filtering*, slides 8–10

5.2.5 Deriving the sharpening filter

Sharpening is not an arbitrary set of numbers; it is derived, and the derivation is a favourite exam question.

Step 1: Define the detail.

Blur the image and subtract:

$$\text{detail} = f - (f * h_{\text{blur}}).$$

What survives is precisely what the blur destroyed: the high frequencies.

Step 2: Add the detail back, amplified by α .

$$g = f + \alpha(f - f * h_{\text{blur}}).$$

Step 3: Express as a single convolution using linearity and the identity kernel.

Since $f = f * \delta$,

$$g = f * \delta + \alpha(f * \delta - f * h_{\text{blur}}) = f * \left[(1 + \alpha) \delta - \alpha h_{\text{blur}} \right].$$

IMPORTANT FORMULA: Unsharp masking

$$h_{\text{sharpen}} = (1 + \alpha) \delta - \alpha h_{\text{blur}}$$

Taking $\alpha = 1$ and h_{blur} the four-neighbour average gives

$$2 \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \longrightarrow \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix},$$

the standard sharpening kernel, which sums to 1 and therefore preserves mean brightness.

EXAM PITFALL: Sharpening amplifies noise

Sharpening is a high-pass operation, and noise is high frequency. Pushing α too high produces a gritty image with bright/dark halos on either side of strong edges (overshoot). If asked “why does my sharpened image look noisy?”, this is the answer.

5.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Convolving a 5×5 patch with three kernels

Problem. Given the patch

$$f = \begin{bmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 20 & 30 & 20 & 10 \\ 10 & 30 & 90 & 30 & 10 \\ 10 & 20 & 30 & 20 & 10 \\ 10 & 10 & 10 & 10 & 10 \end{bmatrix},$$

compute the output at the centre pixel (value 90) for (a) the 3×3 box blur, (b) the sharpening kernel, (c) the Sobel- x kernel, and interpret each result.

Step 1: Extract the 3×3 neighbourhood of the centre.

$$N = \begin{bmatrix} 20 & 30 & 20 \\ 30 & 90 & 30 \\ 20 & 30 & 20 \end{bmatrix}.$$

Step 2 (a): Box blur.

The kernel $\frac{1}{9}\mathbf{1}_{3 \times 3}$ is symmetric, so correlation equals convolution.

$$\sum N = 20 + 30 + 20 + 30 + 90 + 30 + 20 + 30 + 20 = 290, \quad g_{\text{blur}} = \frac{290}{9} = 32.\bar{2} \approx 32.$$

The bright spike of 90 has been pulled down to 32: the blur has *smoothed* the peak.

Step 3 (b): Sharpening.

The kernel $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ is also symmetric under the 180° flip:

$$g_{\text{sharp}} = 0(20) + (-1)(30) + 0(20) \tag{5.6}$$

$$+ (-1)(30) + 5(90) + (-1)(30) \tag{5.7}$$

$$+ 0(20) + (-1)(30) + 0(20) \tag{5.8}$$

$$= -30 - 30 + 450 - 30 - 30 = 330. \tag{5.9}$$

This exceeds 255, so the displayed value clips to 255. The peak has been *exaggerated*, and the clipping is the overshoot warned about above.

Step 4 (c): Sobel- x , done carefully.

$$h_{\text{Sobel-}x} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

is *antisymmetric* in x , so the flip matters. True convolution requires flipping h by 180° first:

$$\tilde{h} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}.$$

Now form the sum of products with N :

$$g_{\text{Sobel}} = 1(20) + 0(30) + (-1)(20) \quad (5.10)$$

$$+ 2(30) + 0(90) + (-2)(30) \quad (5.11)$$

$$+ 1(20) + 0(30) + (-1)(20) \quad (5.12)$$

$$= (20 - 20) + (60 - 60) + (20 - 20) = 0. \quad (5.13)$$

Step 5: Interpret the zero.

Zero is the *correct* answer, not an error. The neighbourhood is left–right symmetric about its centre column, so there is no horizontal intensity gradient there, and Sobel- x correctly reports none. Evaluating one pixel to the left instead — neighbourhood centred on the value 30, i.e. $\begin{bmatrix} 10 & 20 & 30 \\ 10 & 30 & 90 \\ 10 & 20 & 30 \end{bmatrix}$ — gives

$$1(10) - 1(30) + 2(10) - 2(90) + 1(10) - 1(30) = 10 - 30 + 20 - 180 + 10 - 30 = -200,$$

a strong negative response because intensity climbs steeply to the right at that location.

Answer.

Blur $\rightarrow$ 32 (peak suppressed); sharpen $\rightarrow$ 330, clipped to 255 (peak amplified); Sobel- x $\rightarrow$ 0 at the symmetric centre but -200 one pixel left, where a real gradient exists.

Gaussian Filtering and Separability

6.1 Intuitive Theory and Definition

The box blur averages every neighbour equally, including the far corner pixel that has almost nothing to do with the centre. The result looks wrong: boxy, with horizontal and vertical streaks, because the kernel itself has sharp square edges whose high-frequency content leaks into the output. The Gaussian kernel fixes this by weighting neighbours according to distance — nearby pixels count more, distant pixels fade smoothly to zero.

PHYSICAL ANALOGY: A drop of ink

Put a drop of ink on wet paper. It spreads as a soft round blob, darkest at the centre and fading gradually — never as a hard-edged square. That blob *is* a Gaussian, and convolving an image with a Gaussian is mathematically identical to letting every pixel diffuse into its neighbourhood for a fixed time. The standard deviation σ is the “how long you waited” knob.

KEY DEFINITION: Gaussian filter

The kernel

$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{u^2 + v^2}{2\sigma^2}\right),$$

where σ (in pixels) sets the amount of smoothing. The constant $1/(2\pi\sigma^2)$ normalises the continuous integral to 1; in the discrete case you instead divide by the sum of the sampled weights.

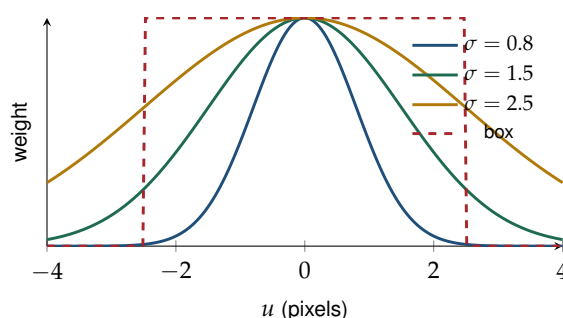


Figure 6.1: Gaussian profiles versus a box profile of comparable width. The box kernel’s vertical walls create ringing and blockiness; the Gaussian’s smooth decay avoids them.

Source deck: *lec03_filter*, slides 36–47

[INSERT IMAGE HERE: Reference lec03_filter, slides 38–43 — “Gaussian vs. box filtering” and “Applying Gaussian filters”]

Photographic comparison: the same portrait blurred by a 7×7 box filter (visible square blocking and faint cross-shaped streaks radiating from bright highlights) versus a Gaussian of equivalent width (a smooth, optically plausible defocus). A second strip shows one image blurred with $\sigma = 1, 2, 4, 8$, progressively removing texture, then edges, then object identity.

6.2 Mathematical Formulation and Derivation

6.2.1 Separability — the property that makes Gaussians practical

Step 1: Split the exponent.

$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{u^2 + v^2}{2\sigma^2}\right) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{u^2}{2\sigma^2}\right) \exp\left(-\frac{v^2}{2\sigma^2}\right).$$

Step 2: Factor the normalising constant.

$$\frac{1}{2\pi\sigma^2} = \frac{1}{\sqrt{2\pi}\sigma} \cdot \frac{1}{\sqrt{2\pi}\sigma}.$$

Step 3: Recognise the product of two 1-D Gaussians.

$$h_{\sigma}(u, v) = \underbrace{\frac{1}{\sqrt{2\pi}\sigma} e^{-u^2/2\sigma^2}}_{g_{\sigma}(u)} \cdot \underbrace{\frac{1}{\sqrt{2\pi}\sigma} e^{-v^2/2\sigma^2}}_{g_{\sigma}(v)} = g_{\sigma}(u) g_{\sigma}(v).$$

Step 4: Push the factorisation through the convolution sum.

$$(h * f)[i, j] = \sum_u \sum_v g_{\sigma}(u) g_{\sigma}(v) f[i - u, j - v] \quad (6.1)$$

$$= \sum_u g_{\sigma}(u) \underbrace{\left[\sum_v g_{\sigma}(v) f[i - u, j - v] \right]}_{\text{1-D blur along } j}. \quad (6.2)$$

The inner bracket is a horizontal 1-D convolution; the outer sum is a vertical 1-D convolution of the result.

IMPORTANT FORMULA: Separability and its cost saving

A 2-D Gaussian convolution equals a 1-D horizontal convolution followed by a 1-D vertical convolution. For a $k \times k$ kernel this reduces the work per pixel from

$$k^2 \text{ multiply-adds} \longrightarrow 2k \text{ multiply-adds,}$$

a speed-up of $k/2$. For $k = 11$ that is $121 \rightarrow 22$, a factor of 5.5.

6.2.2 Other properties worth memorising

1. **The Gaussian is the only kernel that is simultaneously separable, circularly symmetric and strictly positive.** Circular symmetry means it treats all edge orientations equally — no directional bias.

2. **Cascade property.** Convolving with g_{σ_1} then g_{σ_2} equals convolving once with

$$\sigma_{\text{total}} = \sqrt{\sigma_1^2 + \sigma_2^2}.$$

Variances add, not standard deviations. Blurring twice with $\sigma = 3$ gives $\sigma = \sqrt{9+9} = 3\sqrt{2} \approx 4.24$, not 6.

3. **The Fourier transform of a Gaussian is a Gaussian** of reciprocal width: a narrow spatial Gaussian is a wide frequency Gaussian. Hence a large- σ blur is an aggressive low-pass filter and a small- σ blur barely filters at all.
4. **Choosing the kernel size.** The Gaussian never truly reaches zero, so it must be truncated. The standard rule is

$$k = 2\lceil 3\sigma \rceil + 1,$$

which captures 99.7% of the mass and guarantees an odd size with a well-defined centre.

EXAM PITFALL: The three Gaussian mistakes

(1) Adding standard deviations instead of variances when cascading. (2) Choosing σ without choosing a matching kernel size — a 3×3 window with $\sigma = 5$ is effectively a box filter, because truncation discards the informative tails and leaves nine nearly equal weights. (3) Forgetting to re-normalise after truncating: the sampled weights no longer sum to 1, so the whole image darkens.

6.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Building a 3×3 Gaussian with $\sigma = 1$ and applying it

Problem. (a) Construct the 3×3 Gaussian kernel for $\sigma = 1$, normalised to sum to 1. (b) Verify separability. (c) Apply it to the centre of $N = \begin{bmatrix} 20 & 30 & 20 \\ 30 & 90 & 30 \\ 20 & 30 & 20 \end{bmatrix}$. (d) If this blur is applied twice, what single σ is equivalent?

Step 1: Evaluate the unnormalised weight at each offset.

With $\sigma = 1$ the unnormalised weight at (u, v) is $\exp(-(u^2 + v^2)/2)$:

$$(0, 0) : e^0 = 1.00000 \quad (6.3)$$

$$(\pm 1, 0), (0, \pm 1) : e^{-1/2} = 0.60653 \quad (6.4)$$

$$(\pm 1, \pm 1) : e^{-2/2} = 0.36788 \quad (6.5)$$

Step 2: Lay these out as a matrix.

$$\tilde{h} = \begin{bmatrix} 0.36788 & 0.60653 & 0.36788 \\ 0.60653 & 1.00000 & 0.60653 \\ 0.36788 & 0.60653 & 0.36788 \end{bmatrix}.$$

Step 3: Sum the entries.

$$S = 4(0.36788) + 4(0.60653) + 1.00000 = 1.47152 + 2.42612 + 1.00000 = 4.8976.$$

Step 4: Normalise.

$$h = \frac{1}{4.89764} \tilde{h} = \begin{bmatrix} 0.0751 & 0.1238 & 0.0751 \\ 0.1238 & 0.2042 & 0.1238 \\ 0.0751 & 0.1238 & 0.0751 \end{bmatrix}.$$

Check: $4(0.0751) + 4(0.1238) + 0.2042 = 0.3004 + 0.4952 + 0.2042 = 0.9998 \approx 1$. ✓ This is why the integer approximation $\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$ is used in practice: compare $1/16 = 0.0625$, $2/16 = 0.1250$, $4/16 = 0.2500$ against the exact values.

Step 5 (b): Verify separability.

The 1-D kernel is $\begin{bmatrix} e^{-0.5} & 1 & e^{-0.5} \end{bmatrix} = \begin{bmatrix} 0.60653 & 1 & 0.60653 \end{bmatrix}$ with sum $s = 2.21306$, so normalised

$$g = \begin{bmatrix} 0.27407 & 0.45186 & 0.27407 \end{bmatrix}.$$

Outer product entries:

$$\text{corner: } 0.27407 \times 0.27407 = 0.07511 \quad (6.6)$$

$$\text{edge: } 0.27407 \times 0.45186 = 0.12384 \quad (6.7)$$

$$\text{centre: } 0.45186 \times 0.45186 = 0.20418 \quad (6.8)$$

These match h entry for entry. ✓ Note also $s^2 = 2.21306^2 = 4.89764 = S$, so the normalisers factor as well.

Step 6 (c): Apply to N , grouping by weight class.

$$\text{corners: } 0.0751 (20 + 20 + 20 + 20) = 0.0751 \times 80 = 6.008 \quad (6.9)$$

$$\text{edges: } 0.1238 (30 + 30 + 30 + 30) = 0.1238 \times 120 = 14.856 \quad (6.10)$$

$$\text{centre: } 0.2042 \times 90 = 18.378 \quad (6.11)$$

$$g_{\text{Gauss}} = 6.008 + 14.856 + 18.378 = 39.24 \approx 39.$$

Step 7: Compare with the box blur.

The box filter gave 32.2 on this same patch; the Gaussian gives 39.2. The Gaussian preserves more of the central peak because it weights the centre 0.2042 rather than 0.1111. That is the quantitative content of “Gaussian blur is gentler on structure”.

Step 8 (d): Two applications.

$$\sigma_{\text{total}} = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414 \quad (\text{not } 2).$$

Answer.

h as in Step 4; separability verified; filtered centre = $39.24 \approx 39$; two passes $\equiv$ one pass with $\sigma \approx 1.41$.

Non-Linear Filtering: Median and Bilateral

7.1 Intuitive Theory and Definition

Linear filtering has a structural weakness: because the output is a weighted *sum*, one wildly wrong pixel contaminates every output it touches. Two kinds of noise expose this.

KEY DEFINITION: Gaussian noise versus salt-and-pepper noise

Gaussian noise perturbs every pixel by a small random amount drawn from a normal distribution — the image looks grainy. **Salt-and-pepper (impulse) noise** leaves most pixels untouched but sets a scattered few to pure white (255) or pure black (0) — the image looks speckled with confetti.

Averaging is the right response to Gaussian noise: independent zero-mean errors cancel. Averaging is the *wrong* response to salt-and-pepper noise. A single 255 dragged into a 3×3 average shifts the result by roughly $255/9 \approx 28$ grey levels, so the filter does not remove the speck — it *smears* it into a grey blob nine pixels wide.

PHYSICAL ANALOGY: Mean versus median salary

Nine people are in a room; eight earn about \$50k and one is a billionaire. The *mean* salary is over \$100 million — a number describing nobody. The *median* salary is \$50k — exactly right. Salt-and-pepper pixels are the billionaire. The median throws the outlier out by construction, because it depends only on *rank order*, never on magnitude.

KEY DEFINITION: Median filter

Replace each pixel by the median of the values in its neighbourhood. It is **non-linear**, so it is not a convolution and has no kernel.

7.1.1 Bilateral filtering: blur, but not across edges

Gaussian blurring removes noise and edges together, because it weights neighbours purely by *spatial* distance. The bilateral filter adds a second weight based on *intensity* distance: a neighbour contributes only if it is both nearby *and* similar in brightness.

PHYSICAL ANALOGY: Only listen to people who agree with you

Standard blurring polls everyone within earshot and averages their opinions, so a person on the far side of a wall (an edge) still drags the average across. Bilateral filtering polls everyone within earshot but ignores anyone whose opinion is wildly different from yours. Noise (small disagreements) still gets averaged away; the wall (a huge disagreement) is never crossed, so the edge stays sharp.

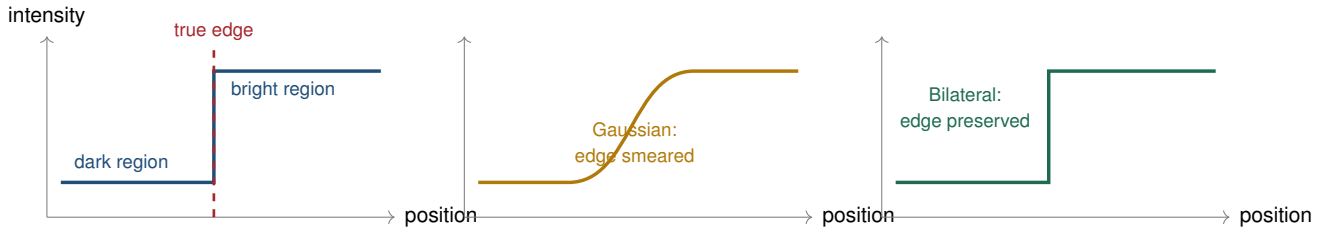


Figure 7.1: A step edge (noise omitted for clarity). Gaussian smoothing (centre) removes noise but rounds the step. Bilateral filtering (right) removes noise *within* each flat region while refusing to average across the intensity jump.

Source deck: *lec03_filter*, slides 49–64

7.2 Mathematical Formulation

IMPORTANT FORMULA: Median filter

$$g[i, j] = \text{median} \{ f[i + u, j + v] : (u, v) \in W \}$$

where W is the window. For a $k \times k$ window with k odd, sort the k^2 values and take element number $\lceil k^2/2 \rceil$.

IMPORTANT FORMULA: Bilateral filter

$$g[i, j] = \frac{1}{W_{ij}} \sum_{(u,v) \in W} \underbrace{G_{\sigma_s}(\|(u, v)\|)}_{\text{spatial weight}} \underbrace{G_{\sigma_r}(|f[i + u, j + v] - f[i, j]|)}_{\text{range weight}} f[i + u, j + v]$$

with

$$W_{ij} = \sum_{(u,v) \in W} G_{\sigma_s}(\|(u, v)\|) G_{\sigma_r}(|f[i + u, j + v] - f[i, j]|).$$

Here σ_s controls spatial reach and σ_r controls intensity tolerance. As $\sigma_r \rightarrow \infty$ the range weight becomes constant and the bilateral filter degenerates to an ordinary Gaussian blur.

Why the bilateral filter is non-linear.

The range weight depends on the image values themselves, so the effective “kernel” changes at every pixel. There is no fixed h , hence no convolution, hence no shift-invariance theorem and no separability shortcut.

Proving the median filter is non-linear.

One counterexample suffices. Take the 1×3 signals

$$f_1 = [0, 0, 9], \quad f_2 = [0, 9, 0].$$

Then $\text{med}(f_1) = 0$ and $\text{med}(f_2) = 0$, so their sum is 0. But $f_1 + f_2 = [0, 9, 9]$, whose median is $9 \neq 0$. ■

7.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Median versus mean versus bilateral on impulse noise

Problem. A 3×3 neighbourhood of an otherwise flat region has been corrupted by one “salt” pixel:

$$N = \begin{bmatrix} 52 & 55 & 50 \\ 54 & 255 & 53 \\ 51 & 56 & 52 \end{bmatrix}.$$

The true uncorrupted centre value is 53. Compute (a) the box-blur output, (b) the median output, (c) the bilateral output with $\sigma_s = 1$, $\sigma_r = 10$, and compare errors.

Step 1 (a): Box blur.

$$\sum N = 52 + 55 + 50 + 54 + 255 + 53 + 51 + 56 + 52 = 678, \quad g_{\text{mean}} = \frac{678}{9} = 75.33.$$

Error against truth: $|75.33 - 53| = 22.33$ grey levels. The speck was not removed but *spread*, and it will contaminate the eight neighbouring outputs as well.

Step 2 (b): Median. Sort all nine values.

$$50, 51, 52, 52, \underline{53}, 54, 55, 56, 255.$$

The 5th of nine is the median, so $g_{\text{med}} = 53$ and the error is exactly 0. Rank order is blind to how extreme the outlier is.

Step 3 (c): Bilateral — spatial weights ($\sigma_s = 1$).

Reusing the unnormalised Gaussian values from the previous topic,

$$G_s = \begin{bmatrix} 0.3679 & 0.6065 & 0.3679 \\ 0.6065 & 1.0000 & 0.6065 \\ 0.3679 & 0.6065 & 0.3679 \end{bmatrix}.$$

Step 4: Range weights ($\sigma_r = 10$, centre value 255).

The range weight is $\exp(-\Delta^2/(2\sigma_r^2)) = \exp(-\Delta^2/200)$ with $\Delta = |f_{\text{neighbour}} - 255|$. For the neighbour valued 52, $\Delta = 203$:

$$\exp\left(-\frac{203^2}{200}\right) = \exp(-206.0) \approx 10^{-90} \approx 0.$$

Every one of the eight neighbours sits roughly 200 levels away, so *all eight range weights vanish*. Only the centre survives, with $\Delta = 0$ and range weight 1.

Step 5: Assemble the bilateral output.

$$g_{\text{bilat}} = \frac{1.0000 \times 1 \times 255 + (\approx 0)}{1.0000 \times 1 + (\approx 0)} = 255.$$

Error: $|255 - 53| = 202$. **The bilateral filter failed completely.**

Step 6: Interpret — this is the key lesson.

The bilateral filter read the impulse as a genuine, very sharp edge and dutifully preserved it. Edge-preserving filters preserve *anything* that looks like an edge, impulse noise included.

Step 7: Tabulate.

Filter	Output	Error	Verdict on impulse noise
Box / Gaussian	75.33	22.33	Smears the speck
Median	53	0	Removes it exactly
Bilateral	255	202	Preserves it as a false edge

Answer.

The median wins outright for salt-and-pepper noise. The correct pipeline when both noise types are present is median *first* to kill impulses, then Gaussian or bilateral to handle the remaining grain.

EXAM PITFALL: Match the filter to the noise

The examiner's favourite question is "which filter would you use and why?". The answer is never "Gaussian" by reflex. Gaussian noise → averaging/Gaussian filter. Salt-and-pepper → median. Noise plus important edges → bilateral, but only *after* impulses have been removed. Remember too that the median filter is not a convolution, so linearity, separability and associativity cannot be invoked for it.

Edge Detection

8.1 Intuitive Theory and Definition

Edges are where the image changes abruptly. They matter because they are where the *information* is: a line drawing of a face is recognisable even though it discards almost every pixel of the photograph. Edges survive changes in lighting far better than raw intensities, which is why nearly every classical vision pipeline begins by converting an image into an edge map.

KEY DEFINITION: What causes an edge

Four physically distinct events all produce an intensity discontinuity, and the image alone cannot tell them apart:

1. **Depth discontinuity** — one surface ends and a farther one begins (occlusion).
2. **Surface-normal discontinuity** — a crease or fold on a single object.
3. **Reflectance discontinuity** — a change of material or paint colour.
4. **Illumination discontinuity** — a cast shadow boundary.

This is the ambiguity of Topic 1 in concrete form: an edge detector reports *that* the intensity changed, never *why*.

PHYSICAL ANALOGY: The hiker and the cliff

Return to the terrain analogy: the image is a landscape of brightness. Walking across a flat plateau, your altimeter barely moves — derivative near zero. Step off a cliff and the altimeter spikes — a large derivative. An edge detector is an altimeter with an alarm: it flags every position where the ground is steep. The gradient direction points straight up the steepest slope, and the edge itself runs *perpendicular* to that direction, along the cliff face.

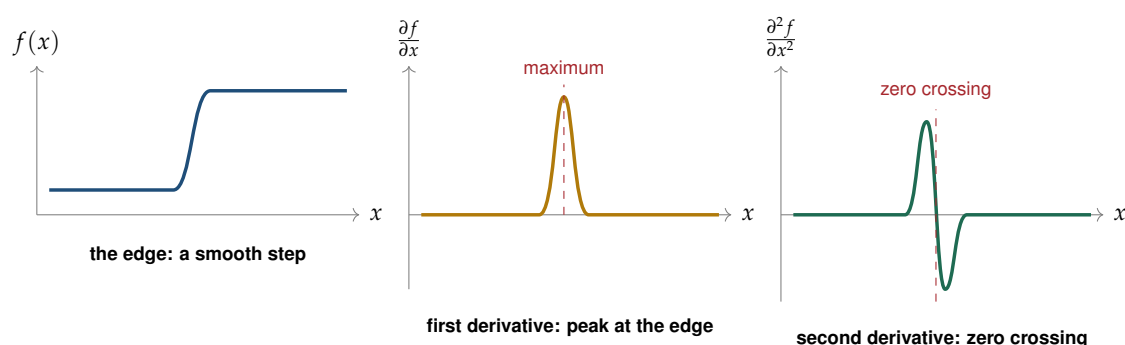


Figure 8.1: The three signatures of an edge. Detectors either look for a *maximum* of the first derivative (Sobel, Canny) or a *zero crossing* of the second derivative (Laplacian of Gaussian, Marr–Hildreth).

Source deck: lec05_edge, slides 5–15

8.2 Mathematical Formulation and Derivation

8.2.1 The image gradient

KEY DEFINITION: Image gradient

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

a vector at every pixel that *points in the direction of the fastest intensity increase*. Its two derived quantities:

$$\text{Magnitude (edge strength): } \|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2} \quad (8.1)$$

$$\text{Orientation (edge normal): } \theta = \text{atan2}\left(\frac{\partial f}{\partial y}, \frac{\partial f}{\partial x}\right) \quad (8.2)$$

The edge *contour* runs at $\theta + 90^\circ$, perpendicular to the gradient.

Step 1: From calculus to pixels.

The definition

$$\frac{\partial f}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

cannot be evaluated on a grid, because ε cannot go below one pixel. Set $\varepsilon = 1$:

$$\frac{\partial f}{\partial x} \approx f[x + 1, y] - f[x, y] \quad (\text{forward difference, kernel } [-1 \ 1]).$$

Step 2: Prefer the centred difference.

The forward difference is biased by half a pixel. The symmetric version,

$$\frac{\partial f}{\partial x} \approx \frac{f[x + 1, y] - f[x - 1, y]}{2}, \quad \text{kernel } \frac{1}{2} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix},$$

is centred exactly on the pixel and is second-order accurate.

Step 3: Add smoothing across the perpendicular direction — the Sobel operator.

A single row of differences is extremely noise-sensitive. Averaging three rows with weights $[1 \ 2 \ 1]$ before differencing gives the separable Sobel kernel:

$$\mathbf{S}_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad \mathbf{S}_y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

Sobel is therefore “smooth, then differentiate” compressed into one 3×3 operation.

8.2.2 Derivative of Gaussian: the crucial trick

Step 4: State the problem.

Differentiation amplifies noise. If $f_{\text{noisy}} = f + n$, then $\partial f_{\text{noisy}} / \partial x = \partial f / \partial x + \partial n / \partial x$, and since noise is high frequency its derivative is enormous. On a noisy signal, the derivative is pure hash with no visible peak at the true edge.

Step 5: The naive fix.

Smooth first, then differentiate:

$$\frac{\partial}{\partial x}(g_\sigma * f).$$

This works, but costs two full convolutions.

Step 6: Apply the differentiation property of convolution.

Because differentiation is linear and shift-invariant, it commutes through the convolution:

$$\frac{\partial}{\partial x}(g_\sigma * f) = \left(\frac{\partial g_\sigma}{\partial x}\right) * f.$$

IMPORTANT FORMULA: Derivative of Gaussian (DoG) filter

Precompute the derivative of the *kernel*, not of the image:

$$\frac{\partial g_\sigma}{\partial x}(u, v) = \frac{\partial}{\partial u} \left[\frac{1}{2\pi\sigma^2} e^{-(u^2+v^2)/2\sigma^2} \right] = -\frac{u}{2\pi\sigma^4} e^{-(u^2+v^2)/2\sigma^2} = -\frac{u}{\sigma^2} g_\sigma(u, v).$$

One convolution replaces two, and the operation is still separable. Large σ detects coarse, blurry edges; small σ detects fine detail and noise.

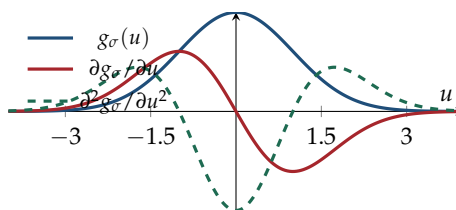


Figure 8.2: The Gaussian and its first two derivatives ($\sigma = 1$). The first derivative is odd-symmetric and peaks on either side of the origin — convolving with it produces a *maximum* at an edge. The second derivative (the “Mexican hat”) is even-symmetric — convolving with it produces a *zero crossing* at an edge.

Source deck: 1ec05_edge, slides 22–30

8.2.3 The Canny edge detector

Canny is the reference algorithm; memorise its four steps in order.

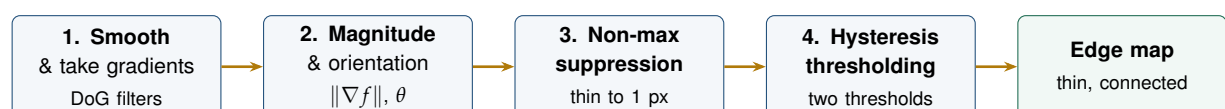


Figure 8.3: The Canny pipeline.

Source deck: 1ec05_edge, slides 32–40

KEY DEFINITION: Non-maximum suppression (NMS)

The gradient magnitude image has *thick* ridges several pixels wide, but a true edge is a curve of zero width. NMS thins the ridge: at each pixel, compare $\|\nabla f\|$ against its two neighbours **along the gradient direction** (i.e. perpendicular to the edge). Keep the pixel only if it is greater than both; otherwise set it to zero. Because the gradient direction is

rarely axis-aligned, the two neighbour values are usually obtained by *interpolation* — the interpolation of Topic 4 reappearing.

KEY DEFINITION: Hysteresis thresholding

A single threshold either fragments strong edges (too high) or admits noise (too low). Use two, $T_{\text{high}} > T_{\text{low}}$:

- $\|\nabla f\| > T_{\text{high}}$: accept unconditionally — a *strong* edge pixel.
- $\|\nabla f\| < T_{\text{low}}$: reject unconditionally.
- In between: accept *only if* the pixel is connected to a strong edge pixel through a chain of other in-between pixels.

A ratio of $T_{\text{high}} : T_{\text{low}} \approx 2:1$ to $3:1$ is typical.

[INSERT IMAGE HERE: Reference lec05_edge, slides 32–40 — “Building an edge detector”, “Non-maximum suppression”, “Hysteresis thresholding”, “Recap: Canny”]

A four-panel photographic sequence on a natural scene: (i) the raw gradient magnitude, showing thick bright ridges along every object boundary plus a haze of texture response; (ii) the same after NMS, with ridges reduced to single-pixel curves; (iii) the result of a single high threshold, showing broken, dashed contours; (iv) the hysteresis result, showing the same contours restored to continuous curves. A companion close-up illustrates the NMS comparison geometry, with an arrow along the gradient direction and the two interpolated neighbour positions marked with open circles.

8.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Full gradient computation with NMS decision

Problem. Consider the patch (a diagonal intensity ramp with a bright corner):

$$f = \begin{bmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 50 & 90 & 90 \\ 10 & 10 & 90 & 90 & 90 \\ 10 & 10 & 90 & 90 & 90 \end{bmatrix}$$

with rows indexed 1–5 top to bottom and columns 1–5 left to right. At the pixel in row 3, column 3 (value 50): (a) compute $\partial f / \partial x$ and $\partial f / \partial y$ using Sobel; (b) compute the gradient magnitude and orientation; (c) decide whether NMS keeps this pixel; (d) classify it under hysteresis with $T_{\text{low}} = 50$ and $T_{\text{high}} = 150$.

Step 1: Extract the 3×3 neighbourhood of (row 3, col 3).

Rows 2–4 and columns 2–4:

$$N = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 90 \\ 10 & 90 & 90 \end{bmatrix}.$$

Step 2 (a): Apply S_x (note: here x increases to the right).

$$\mathbf{S}_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}.$$

Multiply element-wise and sum, row by row:

$$\text{row 1: } (-1)(10) + 0(10) + (1)(10) = -10 + 0 + 10 = 0 \quad (8.3)$$

$$\text{row 2: } (-2)(10) + 0(50) + (2)(90) = -20 + 0 + 180 = 160 \quad (8.4)$$

$$\text{row 3: } (-1)(10) + 0(90) + (1)(90) = -10 + 0 + 90 = 80 \quad (8.5)$$

$$\frac{\partial f}{\partial x} = 0 + 160 + 80 = 240.$$

Step 3: Apply $\mathbf{S}_y$ (with y increasing downward).

$$\mathbf{S}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

$$\text{row 1: } (-1)(10) + (-2)(10) + (-1)(10) = -10 - 20 - 10 = -40 \quad (8.6)$$

$$\text{row 2: } 0(10) + 0(50) + 0(90) = 0 \quad (8.7)$$

$$\text{row 3: } (1)(10) + (2)(90) + (1)(90) = 10 + 180 + 90 = 280 \quad (8.8)$$

$$\frac{\partial f}{\partial y} = -40 + 0 + 280 = 240.$$

Step 4 (b): Gradient magnitude.

$$\|\nabla f\| = \sqrt{240^2 + 240^2} = \sqrt{57600 + 57600} = \sqrt{115200} = 240\sqrt{2} \approx 339.4.$$

Step 5: Gradient orientation.

$$\theta = \text{atan2}(240, 240) = \arctan(1) = 45^\circ.$$

Since y increases *downward*, $\theta = 45^\circ$ means the gradient points down-and-to-the-right — toward the bright region, as it must. The edge contour itself runs at $45^\circ + 90^\circ = 135^\circ$, i.e. along the anti-diagonal.

Step 6 (c): Non-maximum suppression.

The gradient direction 45° quantises to the diagonal neighbour pair: the pixel at (row 2, col 2) and the pixel at (row 4, col 4). We must compare our magnitude 339.4 against the gradient magnitudes *there*.

At (row 2, col 2) the neighbourhood is $\begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 50 \end{bmatrix}$, giving

$$\partial f / \partial x = (-10 + 10) + (-20 + 20) + (-10 + 50) = 40 \quad (8.9)$$

$$\partial f / \partial y = (-10 - 20 - 10) + 0 + (10 + 20 + 50) = -40 + 80 = 40 \quad (8.10)$$

$$\|\nabla f\| = \sqrt{40^2 + 40^2} = 40\sqrt{2} \approx 56.6. \quad (8.11)$$

At (row 4, col 4) the neighbourhood is $\begin{bmatrix} 50 & 90 & 90 \\ 90 & 90 & 90 \\ 90 & 90 & 90 \end{bmatrix}$, giving

$$\partial f / \partial x = (-50 + 90) + (-180 + 180) + (-90 + 90) = 40 \quad (8.12)$$

$$\partial f / \partial y = (-50 - 180 - 90) + 0 + (90 + 180 + 90) = -320 + 360 = 40 \quad (8.13)$$

$$\|\nabla f\| = 40\sqrt{2} \approx 56.6. \quad (8.14)$$

Since $339.4 > 56.6$ and $339.4 > 56.6$, the centre pixel is a local maximum along the gradient direction. **NMS keeps it.**

Step 7 (d): Hysteresis.

$$\|\nabla f\| = 339.4 > T_{\text{high}} = 150.$$

The pixel is classified as a **strong edge pixel** and is accepted unconditionally. It will additionally act as a seed, recruiting any connected neighbours whose magnitude lies between 50 and 150.

Answer.

$\partial f / \partial x = 240, \partial f / \partial y = 240, \|\nabla f\| = 240\sqrt{2} \approx 339.4, \theta = 45^\circ$ (edge runs at 135°); NMS retains the pixel; hysteresis marks it strong.

EXAM PITFALL: Three edge-detection traps

- (1) Reporting the *gradient* direction when asked for the *edge* direction — they differ by 90° .
- (2) Forgetting the sign convention on y : in image coordinates y grows downward, so S_y as written above gives a positive response when the image gets brighter *downward*. State your convention.
- (3) Performing NMS by comparing against the *four or eight* neighbours generally. NMS compares against exactly *two* neighbours, and only along the gradient direction.

Corner Detection: The Harris Detector

9.1 Intuitive Theory and Definition

Edges are useful but insufficient for matching two images. Slide a small window along an edge and every position looks the same — you cannot tell *where* on the edge you are. This is the **aperture problem**, and it is why alignment, stitching and reconstruction are built on *corners*, not edges.

KEY DEFINITION: Keypoint / interest point

A location that can be reliably and repeatably found in multiple images of the same scene, and that is distinctive enough to be matched without ambiguity. The two requirements are **repeatability** (found again under viewpoint and lighting change) and **distinctiveness** (unlike its own neighbours).

PHYSICAL ANALOGY: Moving a window over a jigsaw piece

Cut a small square window out of card and slide it over a photograph. Over a **flat** region — blue sky — the view through the window does not change no matter which way you push it. Over an **edge** — the horizon — the view is unchanged if you slide *along* the horizon, but changes sharply if you slide across it. Over a **corner** — where a roof meets a chimney — the view changes sharply *whichever way* you push. A corner is a location where every possible small movement produces a large change, and that is exactly what makes it locatable.

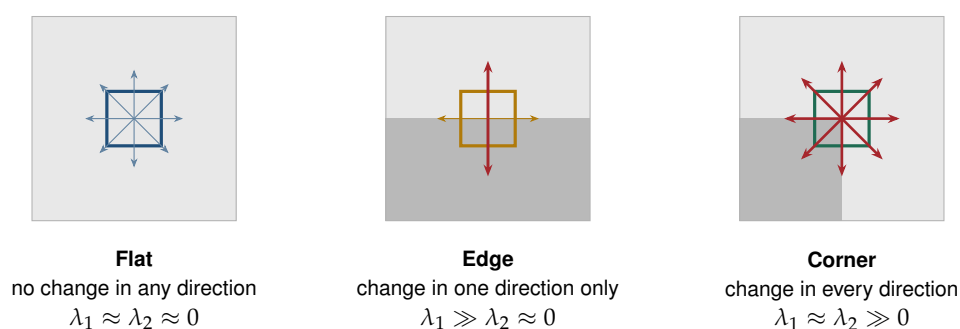


Figure 9.1: The three local structures. Red arrows mark directions of large appearance change. The eigenvalue signature in the third line is the mathematical statement of the picture, derived below.

Source deck: *lec06_corner*, slides 3–11, 18–24

9.2 Mathematical Formulation and Derivation

9.2.1 From “appearance change” to a matrix

Step 1: Write the change of appearance for a shift (u, v) .

Let $w(x, y)$ be a window function (a box, or better a Gaussian). Define

$$E(u, v) = \sum_{x, y} w(x, y) \left[f(x + u, y + v) - f(x, y) \right]^2.$$

This is the sum of squared differences between the patch and a shifted copy of itself. We want $E(u, v)$ to be large for all small (u, v) .

Step 2: Linearise using the first-order Taylor expansion.

For small shifts,

$$f(x + u, y + v) \approx f(x, y) + u \frac{\partial f}{\partial x} + v \frac{\partial f}{\partial y} = f(x, y) + u f_x + v f_y.$$

Step 3: Substitute; the $f(x, y)$ terms cancel.

$$E(u, v) \approx \sum_{x, y} w(x, y) \left[f(x, y) + u f_x + v f_y - f(x, y) \right]^2 = \sum_{x, y} w(x, y) (u f_x + v f_y)^2.$$

Step 4: Expand the square.

$$E(u, v) \approx \sum_{x, y} w \left(u^2 f_x^2 + 2uv f_x f_y + v^2 f_y^2 \right).$$

Step 5: Recognise a quadratic form.

Because u and v are constants with respect to the summation over (x, y) , they pull out:

$$E(u, v) \approx u^2 \underbrace{\sum w f_x^2}_A + 2uv \underbrace{\sum w f_x f_y}_B + v^2 \underbrace{\sum w f_y^2}_C.$$

In matrix form,

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} u & v \end{bmatrix} \mathbf{M} \begin{bmatrix} u \\ v \end{bmatrix}.$$

IMPORTANT FORMULA: The second moment (structure tensor) matrix

$$\mathbf{M} = \sum_{x, y} w(x, y) \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix} = \begin{bmatrix} \sum w f_x^2 & \sum w f_x f_y \\ \sum w f_x f_y & \sum w f_y^2 \end{bmatrix}$$

$\mathbf{M}$ is 2×2 , real and symmetric, so it has two real eigenvalues $\lambda_1 \geq \lambda_2 \geq 0$ and orthogonal eigenvectors. *All the local structure information is compressed into these four numbers.*

9.2.2 Interpreting the eigenvalues

Step 6: Diagonalise.

Since $\mathbf{M}$ is symmetric, write $\mathbf{M} = \mathbf{R}^\top \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \mathbf{R}$ for some rotation $\mathbf{R}$. In the rotated coordinate frame $(u', v') = \mathbf{R}(u, v)^\top$,

$$E \approx \lambda_1 u'^2 + \lambda_2 v'^2.$$

Step 7: Read off the geometry.

The level set $E = \text{const}$ is the ellipse

$$\lambda_1 u'^2 + \lambda_2 v'^2 = c,$$

with semi-axis lengths $\sqrt{c/\lambda_1}$ and $\sqrt{c/\lambda_2}$. So a large eigenvalue corresponds to a short axis: the direction in which appearance changes fastest. This is the “second moment ellipse” of the slides.

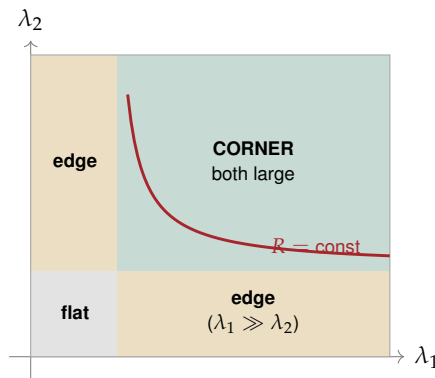


Figure 9.2: The eigenvalue plane. Both eigenvalues small: flat region. One large, one small: edge. Both large: corner. The red curve is a level set of the Harris response R , which carves out the corner region without ever computing eigenvalues explicitly.

Source deck: `lec06_corner`, slides 18–25

9.2.3 The Harris response function

Computing eigenvalues requires a square root at every pixel — expensive in 1988 and still wasteful today. Harris and Stephens observed that the two quantities you actually need are available for free.

Step 8: Recall two identities for a 2×2 matrix.

$$\det \mathbf{M} = \lambda_1 \lambda_2 = AC - B^2, \quad \text{trace } \mathbf{M} = \lambda_1 + \lambda_2 = A + C.$$

Both are computed with multiplications and additions only.

Step 9: Build a function that is large only when both eigenvalues are large.

IMPORTANT FORMULA: Harris corner response

$$R = \det \mathbf{M} - k (\text{trace } \mathbf{M})^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2, \quad k \in [0.04, 0.06].$$

- R large and positive $\Rightarrow$ corner ($\lambda_1 \approx \lambda_2$, both large).
- R large in magnitude and negative $\Rightarrow$ edge (one eigenvalue dominates, so det is small while trace^2 is large).
- $|R|$ small $\Rightarrow$ flat region (both eigenvalues near zero).

Step 10: Why the $k(\text{trace})^2$ term is needed.

Using det alone would fail: $\det = \lambda_1 \lambda_2$ is small for an edge (good) but it is *also* small for a very weak corner, and it grows without bound for a strong corner, making a single threshold hard to set. Subtracting $k(\lambda_1 + \lambda_2)^2$ penalises the strongly anisotropic case, pushing edges firmly negative and creating a clean sign-based decision boundary.

9.2.4 The full algorithm and its invariances

1. Compute image derivatives f_x, f_y (derivative of Gaussian).
2. Compute the products $f_x^2, f_y^2, f_x f_y$ at every pixel.
3. Convolve each of those three images with a Gaussian window w — this performs the $\sum w(\cdot)$ of the definition.
4. Form R at every pixel from det and trace.

5. Threshold R , then apply non-maximum suppression to keep isolated peaks.

Image change	Invariant?	Why
Translation	Yes	Derivatives are shift-invariant; $\mathbf{M}$ travels with the corner.
Rotation	Yes	The ellipse rotates but its <i>eigenvalues</i> are unchanged, and R depends only on those.
Intensity shift $f \rightarrow f + b$	Yes	Derivatives of a constant are zero, so f_x, f_y are untouched.
Intensity scale $f \rightarrow af$	No	$f_x \rightarrow af_x$, so $\mathbf{M} \rightarrow a^2\mathbf{M}$ and $R \rightarrow a^4 \det -ka^4 \text{trace}^2$: R scales by a^4 , so a fixed threshold misfires.
Spatial scaling (zoom)	No	A corner magnified enough becomes a smooth curve — the window no longer sees a corner at all.

Source deck: *lec06_corner*, slides 35–41

EXAM PITFALL: Harris is not scale-invariant — say so

This is the single most-asked follow-up in a viva. Harris is invariant to translation, in-plane rotation and additive intensity change, but *not* to zoom, and only partially to intensity scaling (it can be fixed by normalising the response or thresholding on relative rank). Scale invariance requires the automatic scale-selection machinery of SIFT-style detectors, which is where the course goes next.

[INSERT IMAGE HERE: Reference *lec06_corner*, slides 22–23, 29–33 — “Visualization of second moment ellipses” and “Harris Detector: Example”]

Two figures. First, a photograph overlaid with a grid of small ellipses, one per window: near-circular and tiny in the sky (flat), elongated and lying along the roofline (edge), near-circular and large at chimney corners (corner). Second, a five-panel sequence on a building facade: the input image; the raw R response as a heat map with bright blobs at window corners; the thresholded binary response; the result after non-maximum suppression; and the final detected corners drawn as red crosses on the original image.

9.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Classifying three patches with the Harris response

Problem. Three windows have produced the following accumulated second moment matrices (with $k = 0.05$):

$$\mathbf{M}_1 = \begin{bmatrix} 0.6 & 0.2 \\ 0.2 & 0.4 \end{bmatrix} \times 10^4, \quad \mathbf{M}_2 = \begin{bmatrix} 2.0 & 0.0 \\ 0.0 & 0.01 \end{bmatrix} \times 10^4, \quad \mathbf{M}_3 = \begin{bmatrix} 0.02 & 0.01 \\ 0.01 & 0.02 \end{bmatrix} \times 10^4.$$

Classify each as flat, edge or corner, using both the eigenvalues and the Harris response R , and confirm they agree.

Step 1: Factor out the common scale.

Write $\mathbf{M}_i = 10^4 \mathbf{m}_i$. Since $\det(10^4 \mathbf{m}) = 10^8 \det \mathbf{m}$ and $\text{trace}(10^4 \mathbf{m}) = 10^4 \text{trace} \mathbf{m}$, we have

$$R = 10^8 [\det \mathbf{m} - k(\text{trace} \mathbf{m})^2],$$

so we may work with $\mathbf{m}$ and restore the factor 10^8 at the end.

Step 2: Analyse $\mathbf{m}_1 = \begin{bmatrix} 0.6 & 0.2 \\ 0.2 & 0.4 \end{bmatrix}$.

Eigenvalues solve $\det(\mathbf{m}_1 - \lambda \mathbf{I}) = 0$:

$$(0.6 - \lambda)(0.4 - \lambda) - (0.2)^2 = 0 \quad (9.1)$$

$$0.24 - 0.6\lambda - 0.4\lambda + \lambda^2 - 0.04 = 0 \quad (9.2)$$

$$\lambda^2 - 1.0\lambda + 0.20 = 0 \quad (9.3)$$

$$\lambda = \frac{1.0 \pm \sqrt{1.00 - 0.80}}{2} = \frac{1.0 \pm \sqrt{0.20}}{2} = \frac{1.0 \pm 0.4472}{2} \quad (9.4)$$

$$\lambda_1 = 0.7236, \quad \lambda_2 = 0.2764.$$

The ratio is $\lambda_1/\lambda_2 = 2.62$ — the same order of magnitude, and both comfortably non-zero.

Corner.

Now the Harris response:

$$\det \mathbf{m}_1 = (0.6)(0.4) - (0.2)(0.2) = 0.24 - 0.04 = 0.20, \quad \text{trace } \mathbf{m}_1 = 0.6 + 0.4 = 1.0.$$

$$R_1 = 10^8 [0.20 - 0.05(1.0)^2] = 10^8 (0.20 - 0.05) = 0.15 \times 10^8 = 1.5 \times 10^7.$$

Large and **positive** $\Rightarrow$ corner. $\checkmark$ (Sanity check: $\lambda_1\lambda_2 = 0.7236 \times 0.2764 = 0.2000 \checkmark$ and $\lambda_1 + \lambda_2 = 1.0000 \checkmark$.)

Step 3: Analyse $\mathbf{m}_2 = \begin{bmatrix} 2.0 & 0 \\ 0 & 0.01 \end{bmatrix}$.

The matrix is already diagonal, so $\lambda_1 = 2.0$, $\lambda_2 = 0.01$, a ratio of 200. One direction changes enormously, the perpendicular direction hardly at all. **Edge.**

$$\det \mathbf{m}_2 = (2.0)(0.01) - 0 = 0.02, \quad \text{trace } \mathbf{m}_2 = 2.01.$$

$$R_2 = 10^8 [0.02 - 0.05(2.01)^2] = 10^8 [0.02 - 0.05(4.0401)] \quad (9.5)$$

$$= 10^8 [0.02 - 0.2020] = -0.1820 \times 10^8 = -1.82 \times 10^7. \quad (9.6)$$

Large in magnitude and **negative** $\Rightarrow$ edge. $\checkmark$

Step 4: Analyse $\mathbf{m}_3 = \begin{bmatrix} 0.02 & 0.01 \\ 0.01 & 0.02 \end{bmatrix}$.

For a matrix $\begin{bmatrix} a & b \\ b & a \end{bmatrix}$ the eigenvalues are $a \pm b$, so

$$\lambda_1 = 0.03, \quad \lambda_2 = 0.01.$$

Both are tiny — roughly 1/70 of the corner case. **Flat.**

$$\det \mathbf{m}_3 = (0.02)(0.02) - (0.01)^2 = 0.0004 - 0.0001 = 0.0003, \quad \text{trace } \mathbf{m}_3 = 0.04.$$

$$R_3 = 10^8 [0.0003 - 0.05(0.04)^2] = 10^8 [0.0003 - 0.00008] = 0.00022 \times 10^8 = 2.2 \times 10^4.$$

Positive, but $|R_3|$ is smaller than $|R_1|$ by a factor of $1.5 \times 10^7 / 2.2 \times 10^4 \approx 680$. Against any sensible threshold — say $R > 10^6$ — this is rejected. **Flat.** $\checkmark$

Step 5: Summarise.

Window	λ_1	λ_2	λ_1/λ_2	R	Verdict
M₁	0.72	0.28	2.6	$+1.5 \times 10^7$	Corner
M₂	2.00	0.01	200	-1.8×10^7	Edge
M₃	0.03	0.01	3.0	$+2.2 \times 10^4$	Flat

Note especially the contrast between $\mathbf{M}_1$ and $\mathbf{M}_3$: both have a *similar eigenvalue ratio* (2.6 versus 3.0), so the ratio alone cannot separate them. It is the *magnitude* that distinguishes a corner from flat noise, and R captures magnitude and ratio together in one number. That is precisely the design intent of the Harris response.

Answer.

$\mathbf{M}_1$ corner, $\mathbf{M}_2$ edge, $\mathbf{M}_3$ flat; eigenvalue analysis and the R criterion agree in all three cases.

PART II

Geometric Transformations of Images

Image Manipulation: Range versus Domain

10.1 Intuitive Theory and Definition

Everything in Part I changed pixel *values*. Part II changes pixel *positions*. The distinction was stated in Topic 1 and is now the organising principle of this whole part.

KEY DEFINITION: Filtering versus warping

Image filtering — change the *range*: $g(x, y) = T_r(f(x, y))$ (10.1)

Image warping — change the *domain*: $g(x, y) = f(T_d(x, y))$ (10.2)

Filtering asks “what colour should this pixel be?”. Warping asks “where should this pixel go?”. A filter can turn a photograph white; only a warp can rotate it.

PHYSICAL ANALOGY: Repainting versus rearranging furniture

Filtering is repainting the walls of a room: every object stays where it is, but its colour changes. Warping is picking the furniture up and moving it: nothing changes colour, but everything ends up somewhere new. Notice that a warp raises a question a filter never does — after you move the sofa, what occupies the space it left behind? That question is answered in Topic 15.

KEY DEFINITION: Global (parametric) warping

A warp in which *the same* transformation T applies to every pixel and can be written with a handful of parameters, $\mathbf{x}' = T(\mathbf{x})$. Translation, rotation, scaling, shear, affine and projective warps are all global. A warp that varies from place to place — barrel lens distortion, a face morph — is *local* and needs a per-pixel displacement field instead.

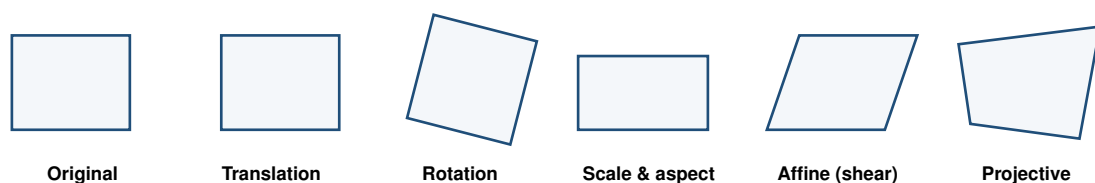


Figure 10.1: Global warps of a rectangle. Reading left to right, each family is strictly more general than the last — and each destroys one more geometric property. Translation preserves everything; projective preserves only straightness.

Source deck: *1__2D_Transformations*, slides 1–3; *lec02_image_processing*, slides 43–47

10.2 Mathematical Formulation

For the whole of Part II the object of study is the map

$$T : \mathbb{R}^2 \rightarrow \mathbb{R}^2, \quad \begin{bmatrix} x \\ y \end{bmatrix} \mapsto \begin{bmatrix} x' \\ y' \end{bmatrix}.$$

The programme is: *which matrices T are useful, how many parameters does each have, and how do we estimate those parameters from matched points?* Topics 11–14 answer the first two; Topic 23 answers the third.

IMPORTANT FORMULA: The master counting argument of Part II

Given a transformation family with n free parameters, and given that each point correspondence $\mathbf{x}_i \leftrightarrow \mathbf{x}'_i$ supplies 2 scalar equations (one for x' , one for y'), the minimum number of correspondences needed is

$$N_{\min} = \left\lceil \frac{n}{2} \right\rceil.$$

Affine: $n = 6 \Rightarrow 3$ points. Homography: $n = 8 \Rightarrow 4$ points. Commit this to memory — it is asked in some form on virtually every paper.

10.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Classifying an operation as filter or warp

Problem. For each operation state whether it changes range or domain, and write it in the appropriate form. (a) Doubling every intensity. (b) Flipping the image left-to-right in an image of width w . (c) Blurring. (d) Rotating by 30° and then inverting the intensities.

Step 1 (a): Doubling intensities.

Pixel positions are untouched, so this is a **range** change:

$$g(x, y) = T_r(f(x, y)) = 2f(x, y), \quad \text{clipped to 255.}$$

Step 2 (b): Horizontal flip.

Pixel values are untouched, so this is a **domain** change. The pixel that should appear at x currently sits at $w - 1 - x$, so

$$g(x, y) = f(T_d(x, y)) = f(w - 1 - x, y).$$

In matrix form this is $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$ followed by a translation of $w - 1$ in x — and, as Topic 12 shows, it is exactly that translation part that forces us into homogeneous coordinates.

Step 3 (c): Blurring.

A blur assigns each pixel a new *value* computed from its neighbours' values, so it is a **range** change — a neighbourhood one, not a pure point process:

$$g(x, y) = \sum_{u, v} h(u, v) f(x - u, y - v).$$

Step 4 (d): Rotate then invert — a composition of both.

$$g(x, y) = 255 - f(\mathbf{R}_{-30^\circ} \mathbf{x}).$$

The inner argument is the domain change; the outer $255 - (\cdot)$ is the range change. Note the sign: to *produce* an output rotated by $+30^\circ$ you must sample the input at the -30° rotated position. That inverse-mapping subtlety is the entire subject of Topic 15.

Answer.

(a) range; (b) domain; (c) range (neighbourhood); (d) both, with the domain part applied to the argument and the range part to the result.

2 × 2 Linear Transformations

11.1 Intuitive Theory and Definition

The simplest useful warps are those expressible as a single 2×2 matrix acting on the coordinate vector.

KEY DEFINITION: Linear transformation of the plane

A map $\mathbf{p}_2 = \mathbf{T} \mathbf{p}_1$, i.e.

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix},$$

with four free parameters. Every such map (i) sends the origin to the origin, (ii) sends straight lines to straight lines, and (iii) sends parallel lines to parallel lines.

PHYSICAL ANALOGY: Stretching a rubber sheet pinned at one point

Imagine the image printed on a rubber sheet with a drawing pin through the origin. You may stretch, squash, rotate, shear or flip the sheet however you like, but the pin holds the origin fixed. Every operation you can perform this way is a 2×2 linear map. The one thing you cannot do is *slide* the sheet — and sliding is precisely what a 2×2 matrix cannot express.

11.2 Mathematical Formulation

11.2.1 The four canonical 2×2 matrices

IMPORTANT FORMULA: Scaling

$$\mathbf{S} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}, \quad x_2 = ax_1, \quad y_2 = by_1.$$

$a = b$: *isotropic* (uniform) scaling, shape preserved. $a \neq b$: *anisotropic*, aspect ratio changed. $a > 1$ stretches, $0 < a < 1$ squishes. Inverse: $\mathbf{S}^{-1} = \begin{bmatrix} 1/a & 0 \\ 0 & 1/b \end{bmatrix}$.

Deriving the rotation matrix.

This derivation is examinable; do not memorise the result blindly.

Step 1: Write the point in polar form.

At distance r and angle ϕ ,

$$x_1 = r \cos \phi, \quad y_1 = r \sin \phi.$$

Step 2: Rotate by θ — the radius is unchanged, the angle increases.

$$x_2 = r \cos(\phi + \theta), \quad y_2 = r \sin(\phi + \theta).$$

Step 3: Expand with the angle-addition identities.

$$x_2 = r(\cos \phi \cos \theta - \sin \phi \sin \theta) \quad (11.1)$$

$$y_2 = r(\sin \phi \cos \theta + \cos \phi \sin \theta) \quad (11.2)$$

Step 4: Substitute $r \cos \phi = x_1$ and $r \sin \phi = y_1$.

$$x_2 = x_1 \cos \theta - y_1 \sin \theta \quad (11.3)$$

$$y_2 = x_1 \sin \theta + y_1 \cos \theta \quad (11.4)$$

Step 5: Read off the matrix.

IMPORTANT FORMULA: Rotation by θ (counter-clockwise)

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Key properties: $\det \mathbf{R} = \cos^2 \theta + \sin^2 \theta = 1$ and $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, so $\mathbf{R}^{-1} = \mathbf{R}^\top = \mathbf{R}(-\theta)$. A matrix with $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$ and $\det \mathbf{R} = +1$ is called **orthonormal**, or a rotation; if $\det = -1$ it includes a reflection.

IMPORTANT FORMULA: Shear (skew)

$$\text{Horizontal: } \begin{bmatrix} 1 & \alpha \\ 0 & 1 \end{bmatrix} \implies x_2 = x_1 + \alpha y_1, \quad y_2 = y_1$$

$$\text{Vertical: } \begin{bmatrix} 1 & 0 \\ \beta & 1 \end{bmatrix} \implies x_2 = x_1, \quad y_2 = \beta x_1 + y_1$$

A horizontal shear displaces each row sideways by an amount proportional to its height — the higher the row, the farther it slides. Note $\det = 1$: **shear preserves area**, which surprises most students, since the shape is visibly distorted.

IMPORTANT FORMULA: Mirror / reflection

$$\text{About the } y\text{-axis: } \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \text{about the } x\text{-axis: } \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Both have $\det = -1$: the sign flip is the algebraic fingerprint of a reflection, which reverses orientation (a clockwise triangle becomes counter-clockwise).

11.2.2 What the determinant tells you

Value of $\det \mathbf{T}$	Geometric meaning	Example
$\det = 1$	Area and orientation preserved	rotation, shear
$\det = -1$	Area preserved, orientation flipped	reflection
$ \det > 1$	Area magnified by $ \det $	scaling with $ab > 1$
$0 < \det < 1$	Area shrunk	scaling with $ab < 1$
$\det = 0$	Plane collapsed to a line; not invertible	projection onto an axis

Why $|\det|$ is the area scale factor.

The unit square has corners $\mathbf{0}$, $\mathbf{e}_1$, $\mathbf{e}_2$ and $\mathbf{e}_1 + \mathbf{e}_2$. Under $\mathbf{T}$ these map to $\mathbf{0}$, the first column of $\mathbf{T}$, the second column of $\mathbf{T}$, and their sum. The image is therefore the parallelogram spanned by the two columns, whose area is $|a_{11}a_{22} - a_{12}a_{21}| = |\det \mathbf{T}|$. ■

11.2.3 The decisive limitation

EXAM PITFALL: A 2×2 matrix cannot translate

Set $\mathbf{p}_1 = \mathbf{0}$. Then $\mathbf{T}\mathbf{0} = \mathbf{0}$ for *any* 2×2 matrix: the origin is always a fixed point. Translation moves the origin, so translation is **not** a linear transformation of $\mathbb{R}^2$ and has no 2×2 matrix. Since translation is the most common image operation of all, this limitation is fatal — and it is the entire motivation for the next topic.

11.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Reading a matrix, and composing two transformations

Problem. (a) Describe geometrically what $\mathbf{T} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$ does, and apply it to $(1, 1)$. (b) Compute the matrix that rotates by 90° and *then* applies $\mathbf{T}$. (c) Compute the matrix for the opposite order. (d) Do they agree? (e) What are the determinants?

Step 1 (a): Read the matrix.

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \implies x_2 = 2x_1, \quad y_2 = 3y_1.$$

This is *anisotropic scaling*: horizontal stretch by 2, vertical stretch by 3. There is no rotation, shear or translation, because the off-diagonal entries are zero. Applying it:

$$\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}.$$

Step 2 (b): Build the 90° rotation.

$$\mathbf{R}(90^\circ) = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ \\ \sin 90^\circ & \cos 90^\circ \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Step 3: Rotate *first*, then scale.

Applying $\mathbf{R}$ first means $\mathbf{p}_2 = \mathbf{T}(\mathbf{R}\mathbf{p}_1) = (\mathbf{TR})\mathbf{p}_1$, so the combined matrix has $\mathbf{T}$ on the *left*:

$$\mathbf{TR} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} (2)(0) + (0)(1) & (2)(-1) + (0)(0) \\ (0)(0) + (3)(1) & (0)(-1) + (3)(0) \end{bmatrix} = \begin{bmatrix} 0 & -2 \\ 3 & 0 \end{bmatrix}.$$

Step 4 (c): Scale *first*, then rotate.

$$\mathbf{RT} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} (0)(2) + (-1)(0) & (0)(0) + (-1)(3) \\ (1)(2) + (0)(0) & (1)(0) + (0)(3) \end{bmatrix} = \begin{bmatrix} 0 & -3 \\ 2 & 0 \end{bmatrix}.$$

Step 5 (d): Compare.

$$\mathbf{TR} = \begin{bmatrix} 0 & -2 \\ 3 & 0 \end{bmatrix} \neq \begin{bmatrix} 0 & -3 \\ 2 & 0 \end{bmatrix} = \mathbf{RT}.$$

Matrix composition does not commute. Verify with the test point $(1, 0)$:

$$\mathbf{TR} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix} \quad \text{rotated to } (0, 1), \text{ then } y\text{-scaled by } 3 \quad (11.5)$$

$$\mathbf{RT} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix} \quad x\text{-scaled to } (2, 0), \text{ then rotated} \quad (11.6)$$

Different answers, confirming the algebra.

Step 6 (e): Determinants.

$$\det \mathbf{T} = (2)(3) - 0 = 6, \quad \det \mathbf{R} = (0)(0) - (-1)(1) = 1,$$

$$\det(\mathbf{TR}) = (0)(0) - (-2)(3) = 6, \quad \det(\mathbf{RT}) = (0)(0) - (-3)(2) = 6.$$

Both compositions have determinant $6 = \det \mathbf{T} \cdot \det \mathbf{R}$, illustrating $\det(\mathbf{AB}) = \det \mathbf{A} \det \mathbf{B}$. So the two orders give *different shapes* but the *same area*: six times the original.

Answer.

$\mathbf{T}$ is an anisotropic stretch with $(1, 1) \mapsto (2, 3)$; $\mathbf{TR} = \begin{bmatrix} 0 & -2 \\ 3 & 0 \end{bmatrix}$, $\mathbf{RT} = \begin{bmatrix} 0 & -3 \\ 2 & 0 \end{bmatrix}$; they differ because composition is non-commutative; both scale area by 6.

Homogeneous Coordinates

12.1 Intuitive Theory and Definition

Translation is not linear in $\mathbb{R}^2$. Homogeneous coordinates repair this with one elegant move: lift the plane into three dimensions, where translation *does* become a matrix operation.

PHYSICAL ANALOGY: The plane at height one

Take the 2D image plane and float it in 3D space at height $z = 1$, so the 2D point (x, y) becomes the 3D point $(x, y, 1)$. Now draw a ray from the 3D origin through that point: every point on that ray — $(2x, 2y, 2)$, $(5x, 5y, 5)$, $(-x, -y, -1)$ — pierces the floating plane at exactly the same spot. So an entire *line through the origin* represents one 2D point. A **shear in 3D**, which is an ordinary linear map, tilts that ray and therefore *slides* the intersection point across the plane. Translation in 2D is shear in 3D. That is the whole trick.

KEY DEFINITION: Homogeneous coordinates

The homogeneous representation of the 2D point $\mathbf{p} = (x, y)$ is the 3D vector $\tilde{\mathbf{p}} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$, and more generally *any* scalar multiple $\begin{bmatrix} wx \\ wy \\ w \end{bmatrix}$ with $w \neq 0$ represents the same point. To return to Cartesian coordinates, **divide by the last element**:

$$\begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{bmatrix} \longrightarrow \left(\frac{\tilde{x}}{\tilde{w}}, \frac{\tilde{y}}{\tilde{w}} \right), \quad \tilde{w} \neq 0.$$

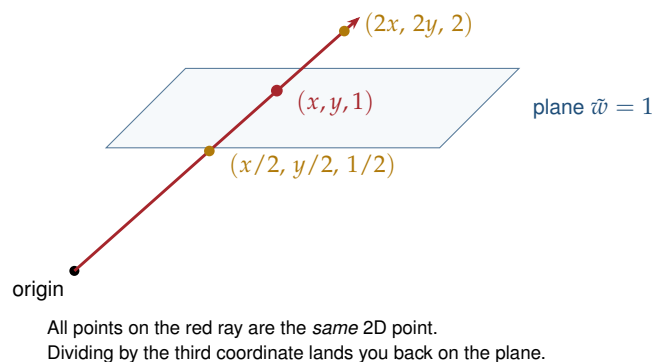


Figure 12.1: The geometry of homogeneous coordinates. A 2D point corresponds to a line through the 3D origin; the canonical representative is where that line pierces $\tilde{w} = 1$.

Source deck: *Homogeneous_Coordinates*, slides 2–11; *2D_3D_Transformations*, slides 3–6

12.2 Mathematical Formulation and Derivation

12.2.1 Translation becomes a matrix

Step 1: Write translation in Cartesian coordinates.

$$x' = x + b_x, \quad y' = y + b_y.$$

As established, no 2×2 matrix does this.

Step 2: Append a 1 to the point and seek a 3×3 matrix $\mathbf{M}$ with

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{M} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

Step 3: Expand a general first row.

Row one gives $x' = m_{11}x + m_{12}y + m_{13}(1)$. Matching against $x' = x + b_x$ term by term forces $m_{11} = 1, m_{12} = 0, m_{13} = b_x$.

Step 4: Repeat for row two.

$y' = m_{21}x + m_{22}y + m_{23}$ matched against $y' = y + b_y$ gives $m_{21} = 0, m_{22} = 1, m_{23} = b_y$.

Step 5: Row three must preserve the homogeneous coordinate,

so it is $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$.

IMPORTANT FORMULA: Translation in homogeneous coordinates

$$\mathbf{T}(b_x, b_y) = \begin{bmatrix} 1 & 0 & b_x \\ 0 & 1 & b_y \\ 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & b_x \\ 0 & 1 & b_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + b_x \\ y + b_y \\ 1 \end{bmatrix}.$$

This 3×3 matrix is a *shear* in 3D — exactly as the physical analogy predicted — and is a linear transformation of $\mathbb{R}^3$ even though the 2D operation it encodes is not linear in $\mathbb{R}^2$.

12.2.2 Embedding the linear transformations

Any 2×2 linear map $\mathbf{A}$ embeds as the top-left block:

$$\begin{bmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & 0 \\ a_{21} & a_{22} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

So homogeneous coordinates *lose nothing* and *gain* translation. Better still, compositions become matrix products:

$$\text{rotate, then translate} = \mathbf{T}\mathbf{R}, \quad \text{translate, then rotate} = \mathbf{R}\mathbf{T},$$

and these are, as always, different.

IMPORTANT FORMULA: Rotation about an arbitrary point (c_x, c_y)

The classic three-step composition, read *right to left*:

$$\mathbf{M} = \underbrace{\mathbf{T}(c_x, c_y)}_{\text{3. move back}} \underbrace{\mathbf{R}(\theta)}_{\text{2. rotate}} \underbrace{\mathbf{T}(-c_x, -c_y)}_{\text{1. bring centre to origin}} = \begin{bmatrix} 1 & 0 & c_x \\ 0 & 1 & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -c_x \\ 0 & 1 & -c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

EXAM PITFALL: Three homogeneous-coordinate traps

(1) **Forgetting to divide by $\tilde{w}$** at the end. For affine transformations $\tilde{w}$ stays 1 so nothing goes wrong, which lulls you into skipping the step — and then a homography (where $\tilde{w} \neq 1$) gives nonsense. (2) **Reading a composition left to right.** $\mathbf{ABCx}$ applies $\mathbf{C}$ first; matrices compose right to left. (3) $\tilde{w} = 0$. A homogeneous point with last coordinate zero is *not* an ordinary point; it is a **point at infinity** representing a pure direction. Far from being an error, this is the machinery that makes vanishing points computable.

12.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Rotating an image about its own centre

Problem. A 100×100 image is to be rotated 90° counter-clockwise about its centre $(50, 50)$. (a) Build the composite 3×3 matrix. (b) Verify it maps the centre to itself. (c) Find the image of the corner $(0, 0)$. (d) Confirm the result is geometrically sensible.

Step 1: Write the three factors.

$$\mathbf{T}_1 = \begin{bmatrix} 1 & 0 & -50 \\ 0 & 1 & -50 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{R} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{T}_2 = \begin{bmatrix} 1 & 0 & 50 \\ 0 & 1 & 50 \\ 0 & 0 & 1 \end{bmatrix}.$$

Step 2: Multiply $\mathbf{R} \mathbf{T}_1$ first (the two rightmost factors).

Row 1 of $\mathbf{R}$ is $[0 \ -1 \ 0]$; against the columns of $\mathbf{T}_1$:

$$(0)(1) + (-1)(0) + (0)(0) = 0; \quad (0)(0) + (-1)(1) + (0)(0) = -1; \quad (12.1)$$

$$(0)(-50) + (-1)(-50) + (0)(1) = 50. \quad (12.2)$$

Row 2 of $\mathbf{R}$ is $[1 \ 0 \ 0]$:

$$1; \ 0; \ (1)(-50) + (0)(-50) + (0)(1) = -50.$$

Row 3 is unchanged, so

$$\mathbf{RT}_1 = \begin{bmatrix} 0 & -1 & 50 \\ 1 & 0 & -50 \\ 0 & 0 & 1 \end{bmatrix}.$$

Step 3: Pre-multiply by $\mathbf{T}_2$.

$$\mathbf{M} = \mathbf{T}_2(\mathbf{RT}_1) = \begin{bmatrix} 1 & 0 & 50 \\ 0 & 1 & 50 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 & 50 \\ 1 & 0 & -50 \\ 0 & 0 & 1 \end{bmatrix}.$$

Row 1: $(1)(0) + (0)(1) + (50)(0) = 0$; $(1)(-1) + (0)(0) + (50)(0) = -1$; $(1)(50) + (0)(-50) + (50)(1) = 100$.

Row 2: $(0)(0) + (1)(1) + (50)(0) = 1$; $(0)(-1) + (1)(0) + (50)(0) = 0$; $(0)(50) + (1)(-50) + (50)(1) = 0$.

$$\mathbf{M} = \begin{bmatrix} 0 & -1 & 100 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 4 (b): Check the centre is fixed.

$$\mathbf{M} \begin{bmatrix} 50 \\ 50 \\ 1 \end{bmatrix} = \begin{bmatrix} (0)(50) + (-1)(50) + (100)(1) \\ (1)(50) + (0)(50) + (0)(1) \\ 1 \end{bmatrix} = \begin{bmatrix} 50 \\ 50 \\ 1 \end{bmatrix} \longrightarrow (50, 50). \checkmark$$

Step 5 (c): Image of the corner $(0, 0)$.

$$\mathbf{M} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 + 0 + 100 \\ 0 + 0 + 0 \\ 1 \end{bmatrix} \longrightarrow (100, 0).$$

Step 6 (d): Sanity check all four corners.

Input corner	Output	Interpretation
$(0, 0)$ top-left	$(100, 0)$	moved to top-right
$(100, 0)$ top-right	$(100, 100)$	moved to bottom-right
$(100, 100)$ bot-right	$(0, 100)$	moved to bottom-left
$(0, 100)$ bot-left	$(0, 0)$	moved to top-left

The four corners cycle round by one position and the square maps onto itself — exactly what a 90° rotation about the centre of a square must do. $\checkmark$ (In screen coordinates, with y pointing down, this cycle reads as clockwise on screen; the mathematics is counter-clockwise in a y -up frame. State your convention.)

Answer.

$\mathbf{M} = \begin{bmatrix} 0 & -1 & 100 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$; the centre is fixed; $(0, 0) \mapsto (100, 0)$; all four corners permute correctly.

The Transformation Hierarchy

13.1 Intuitive Theory and Definition

The transformations of Part II form a strict *nested hierarchy*. Each level adds degrees of freedom and, in exchange, gives up a geometric property that the level below preserved. Knowing the hierarchy tells you which model to fit in Topic 23 and how many correspondences you will need.

PHYSICAL ANALOGY: Progressively loosening the rules of a game

Start with the strictest game: you may only *slide* the shape (translation). Loosen it: you may slide and *spin* (rigid). Loosen again: you may also *resize* uniformly (similarity). Again: you may stretch each axis differently and skew, but opposite sides must stay parallel (affine). Finally, drop even that: sides may converge, as railway tracks do (projective). At every loosening the shape can do more, and you must measure more points to pin down what it did.

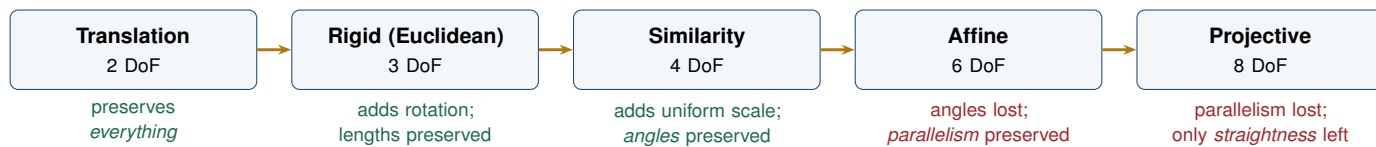


Figure 13.1: The hierarchy. Each arrow adds degrees of freedom and removes an invariant. The red labels mark where a property is destroyed.

Source deck: *Rigid_Transformation, Similarity_Transformation, 2__3D_Transformations slides 8–11; lec10_alignment slides 13–15*

13.2 Mathematical Formulation

Every level is a 3×3 homogeneous matrix acting on $(x, y, 1)^\top$.

13.2.1 Level 1: Translation (2 DoF)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & b_x \\ 0 & 1 & b_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Parameters b_x, b_y . Preserves orientation, lengths, angles, areas, parallelism, straightness — everything.

13.2.2 Level 2: Rigid / Euclidean (3 DoF)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & b_x \\ \sin \theta & \cos \theta & b_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Parameters θ, b_x, b_y . The upper-left block is orthonormal ($\mathbf{R}^\top \mathbf{R} = \mathbf{I}$, $\det = 1$), which is what enforces “rotation only, no stretching”. Preserves lengths and angles: this is the motion of a rigid object in a plane.

13.2.3 Level 3: Similarity (4 DoF)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s \cos \theta & -s \sin \theta & b_x \\ s \sin \theta & s \cos \theta & b_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Parameters s, θ, b_x, b_y . The scale s is *isotropic* — the same in both directions — which is exactly why angles survive. Lengths are no longer preserved, but *ratios* of lengths are.

13.2.4 Level 4: Affine (6 DoF)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \iff \begin{aligned} x' &= ax + by + c \\ y' &= dx + ey + f \end{aligned}$$

Six entirely unconstrained parameters in the top two rows: affine is exactly “any linear map plus any translation”. Angles and lengths are destroyed, but **parallel lines stay parallel** and **ratios of lengths along a line** are preserved. Since the last row is still $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$, $\tilde{w}$ stays at 1 and no division is needed.

13.2.5 Level 5: Projective / homography (8 DoF)

$$\begin{bmatrix} \tilde{x}' \\ \tilde{y}' \\ \tilde{w}' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Nine entries but only **eight** degrees of freedom, because the matrix is defined only up to scale. Detailed in Topic 14.

IMPORTANT FORMULA: The hierarchy reference table — memorise this

Transformation	DoF	Min. pts	Preserves	Form
Translation	2	1	orientation, lengths, angles, parallelism	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$
Rigid / Euclidean	3	2	lengths, angles, areas, parallelism	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$
Similarity	4	2	angles, ratios of lengths, parallelism	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$
Affine	6	3	parallelism, ratios along a line, ratios of areas	$\begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$
Projective	8	4	straightness, cross-ratio only	$\begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{v}^\top & w \end{bmatrix}$

EXAM PITFALL: Counting degrees of freedom

Do not count matrix *entries*; count *free parameters*. A rigid transformation has four non-trivial entries in its upper-left block but only *one* free parameter there (θ), because those four entries must satisfy $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$. Likewise a homography has nine entries but eight

degrees of freedom because of the overall scale ambiguity. Getting this count wrong propagates straight into a wrong minimum-point count and a wrongly sized linear system.

13.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Identifying the transformation family from a matrix

Problem. For each matrix, name the most restrictive family it belongs to, state its degrees of freedom, and extract its parameters.

$$\mathbf{M}_1 = \begin{bmatrix} 0.866 & -0.5 & 10 \\ 0.5 & 0.866 & 20 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{M}_2 = \begin{bmatrix} 1.732 & -1.0 & 10 \\ 1.0 & 1.732 & 20 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{M}_3 = \begin{bmatrix} 2 & 0.5 & 10 \\ 0 & 3 & 20 \\ 0 & 0 & 1 \end{bmatrix}.$$

Step 1: Establish a test procedure.

Let $\mathbf{A}$ be the upper-left 2×2 block.

- If the bottom row is not $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$: projective.
- Else if $\mathbf{A}^\top \mathbf{A} = \mathbf{I}$: rigid.
- Else if $\mathbf{A}^\top \mathbf{A} = s^2 \mathbf{I}$ for some s : similarity.
- Else: affine.

Step 2: Test $\mathbf{M}_1$.

The bottom row is $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$, so it is at most affine. With $\mathbf{A}_1 = \begin{bmatrix} 0.866 & -0.5 \\ 0.5 & 0.866 \end{bmatrix}$:

$$\mathbf{A}_1^\top \mathbf{A}_1 = \begin{bmatrix} 0.866 & 0.5 \\ -0.5 & 0.866 \end{bmatrix} \begin{bmatrix} 0.866 & -0.5 \\ 0.5 & 0.866 \end{bmatrix}.$$

Entry (1,1): $(0.866)^2 + (0.5)^2 = 0.75 + 0.25 = 1.000$. Entry (1,2): $(0.866)(-0.5) + (0.5)(0.866) = -0.433 + 0.433 = 0$. Entry (2,2): $(-0.5)^2 + (0.866)^2 = 1.000$. So $\mathbf{A}_1^\top \mathbf{A}_1 = \mathbf{I}$: $\mathbf{A}_1$ is orthonormal, hence **rigid, 3 DoF**. Extract the angle:

$$\cos \theta = 0.866, \quad \sin \theta = 0.5 \implies \theta = \text{atan2}(0.5, 0.866) = 30^\circ,$$

with translation $(b_x, b_y) = (10, 20)$. Check $\det \mathbf{A}_1 = 0.75 + 0.25 = 1$. ✓

Step 3: Test $\mathbf{M}_2$, with $\mathbf{A}_2 = \begin{bmatrix} 1.732 & -1.0 \\ 1.0 & 1.732 \end{bmatrix}$.

Entry (1,1) of $\mathbf{A}_2^\top \mathbf{A}_2$: $(1.732)^2 + (1.0)^2 = 3.000 + 1.000 = 4.000$. Entry (1,2): $(1.732)(-1.0) + (1.0)(1.732) = 0$. Entry (2,2): $(-1.0)^2 + (1.732)^2 = 4.000$.

$$\mathbf{A}_2^\top \mathbf{A}_2 = 4\mathbf{I} = s^2 \mathbf{I} \implies s = 2.$$

Similarity, 4 DoF. Divide out the scale to recover the rotation:

$$\frac{1}{2} \mathbf{A}_2 = \begin{bmatrix} 0.866 & -0.5 \\ 0.5 & 0.866 \end{bmatrix} \implies \theta = 30^\circ.$$

So $\mathbf{M}_2$ scales by 2, rotates by 30° and translates by $(10, 20)$. Check: $\det \mathbf{A}_2 = (1.732)(1.732) - (-1.0)(1.0) = 3 + 1 = 4 = s^2$ ✓ — for a similarity, area scales by s^2 , here fourfold.

Step 4: Test $\mathbf{M}_3$, with $\mathbf{A}_3 = \begin{bmatrix} 2 & 0.5 \\ 0 & 3 \end{bmatrix}$.

Entry (1,1) of $\mathbf{A}_3^\top \mathbf{A}_3$: $2^2 + 0^2 = 4$. Entry (2,2): $(0.5)^2 + 3^2 = 0.25 + 9 = 9.25$. Since $4 \neq 9.25$, this is not $s^2 \mathbf{I}$ for any s : **affine, 6 DoF**. Reading the rows, $x' = 2x + 0.5y + 10$ and $y' = 3y + 20$ — a horizontal stretch by 2, a vertical stretch by 3, a horizontal shear of 0.5, and a translation of (10,20). Area scales by $\det \mathbf{A}_3 = (2)(3) - (0.5)(0) = 6$.

Step 5: Verify parallelism survives the affine map.

Take two parallel segments $(0,0) \rightarrow (1,0)$ and $(0,1) \rightarrow (1,1)$, both with direction (1,0):

$$(0,0) \mapsto (10,20), \quad (1,0) \mapsto (12,20) \quad \Rightarrow \quad \text{direction } (2,0) \quad (13.1)$$

$$(0,1) \mapsto (10.5,23), \quad (1,1) \mapsto (12.5,23) \quad \Rightarrow \quad \text{direction } (2,0) \quad (13.2)$$

Identical directions, so the segments remain parallel. ✓

Answer.

$\mathbf{M}_1$: rigid, 3 DoF, $\theta = 30^\circ$, $\mathbf{t} = (10,20)$. $\mathbf{M}_2$: similarity, 4 DoF, $s = 2$, $\theta = 30^\circ$, $\mathbf{t} = (10,20)$.

$\mathbf{M}_3$: affine, 6 DoF, stretch (2,3) with shear 0.5, $\mathbf{t} = (10,20)$.

Projective Transformations and the Homography

14.1 Intuitive Theory and Definition

Stand on a straight railway line and photograph it. The rails are parallel in the world, but in the photograph they converge at a point on the horizon. No affine transformation can do that — affine maps preserve parallelism by construction. The transformation that maps the world plane to the image plane is a **projective transformation**, or **homography**.

KEY DEFINITION: Homography

A projective transformation of the plane: an invertible 3×3 matrix $\mathbf{H}$ acting on homogeneous coordinates,

$$\begin{bmatrix} \tilde{x}' \\ \tilde{y}' \\ \tilde{w}' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad x' = \frac{\tilde{x}'}{\tilde{w}'}, \quad y' = \frac{\tilde{y}'}{\tilde{w}'}.$$

Geometrically it is the mapping of one plane onto another *through a single point* — a centre of projection.

PHYSICAL ANALOGY: A slide projector onto a tilted wall

Put a photographic slide in a projector and shine it onto a wall. If the wall is exactly perpendicular to the beam, the projected image is a scaled copy — a similarity. Now tilt the wall. The rectangle on the slide becomes a trapezium: the near edge smaller, the far edge larger, the sides no longer parallel. Everything that was straight is still straight, but nothing else survives. That mapping, from slide to tilted wall, is the homography, and the projector bulb is the centre of projection through which every ray passes.

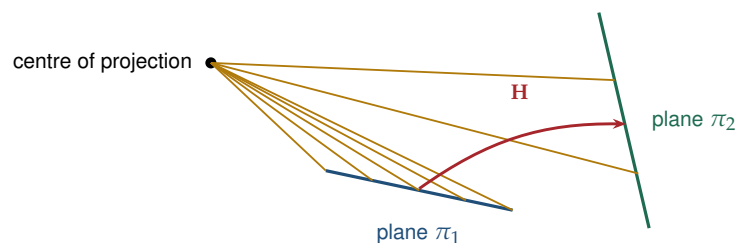


Figure 14.1: A homography is the composition of a projection from plane π_1 to the centre and back out to plane π_2 . Every point on π_1 and its image on π_2 lie on one ray through the centre.

Source deck: 2__3D_Transformations, slides 11–18

14.2 Mathematical Formulation and Derivation

14.2.1 Why eight degrees of freedom, not nine

Step 1: Consider scaling the whole matrix.

For any $\lambda \neq 0$, $(\lambda \mathbf{H})\tilde{\mathbf{x}} = \lambda(\mathbf{H}\tilde{\mathbf{x}})$.

Step 2: Recall the homogeneous equivalence rule.

The vectors $\lambda\tilde{\mathbf{x}}'$ and $\tilde{\mathbf{x}}'$ represent the *same* 2D point, since dividing by the last coordinate cancels λ :

$$\left(\frac{\lambda\tilde{x}'}{\lambda\tilde{w}'} , \frac{\lambda\tilde{y}'}{\lambda\tilde{w}'} \right) = \left(\frac{\tilde{x}'}{\tilde{w}'} , \frac{\tilde{y}'}{\tilde{w}'} \right).$$

Step 3: Conclude.

$\mathbf{H}$ and $\lambda\mathbf{H}$ define *identical* maps, so $\mathbf{H}$ is determined only up to scale, $\mathbf{H} \sim \lambda\mathbf{H}$. Nine entries minus one scale = **8 degrees of freedom**. ■

IMPORTANT FORMULA: Fixing the scale

Because of the scale ambiguity, one extra constraint must be imposed before $\mathbf{H}$ can be solved for numerically. Two standard choices:

$$(i) h_{33} = 1, \quad (ii) \|\mathbf{h}\| = 1,$$

where $\mathbf{h}$ is the 9-vector of entries. Choice (i) is simpler but fails when the true $h_{33} = 0$; choice (ii) is always safe and leads to the SVD solution of Topic 20.

14.2.2 Expanding the division

Step 4: Write out the three rows.

$$\tilde{x}' = h_{11}x + h_{12}y + h_{13} \tag{14.1}$$

$$\tilde{y}' = h_{21}x + h_{22}y + h_{23} \tag{14.2}$$

$$\tilde{w}' = h_{31}x + h_{32}y + h_{33} \tag{14.3}$$

Step 5: Divide by $\tilde{w}'$.

IMPORTANT FORMULA: Homography in Cartesian form

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

The transformation is a *ratio of linear functions* — non-linear in (x, y) . Setting $h_{31} = h_{32} = 0$ and $h_{33} = 1$ makes the denominator 1 and recovers the affine case exactly: **affine is the special case of a homography whose bottom row is $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$.**

14.2.3 Vanishing points from the homogeneous algebra

Step 6: Ask what happens to points at infinity.

A point at infinity in the input, $\tilde{\mathbf{x}} = \begin{bmatrix} d_x \\ d_y \\ 0 \end{bmatrix}$, representing the direction (d_x, d_y) , maps to

$$\mathbf{H} \begin{bmatrix} d_x \\ d_y \\ 0 \end{bmatrix} = \begin{bmatrix} h_{11}d_x + h_{12}d_y \\ h_{21}d_x + h_{22}d_y \\ h_{31}d_x + h_{32}d_y \end{bmatrix},$$

which is a perfectly *finite* point whenever $h_{31}d_x + h_{32}d_y \neq 0$. That finite image is the **vanishing point** of the direction (d_x, d_y) .

KEY DEFINITION: Vanishing point

The image of the point at infinity of a direction. All world lines sharing that direction — the two rails — converge to this single image point. Parallel lines meeting at a vanishing point is not an optical illusion; it is the correct behaviour of projective geometry, in which points at infinity are ordinary points.

14.2.4 Composition of homographies

If $\mathbf{H}_1$ maps $\pi_1 \rightarrow \pi_2$ and $\mathbf{H}_2$ maps $\pi_2 \rightarrow \pi_3$, then

$$\mathbf{H}_{13} = \mathbf{H}_2\mathbf{H}_1,$$

again a 3×3 invertible matrix. Homographies are therefore closed under composition and form a group, with $\mathbf{H}^{-1}$ the inverse mapping. This is why panorama stitching can chain many pairwise homographies into one global mosaic.

EXAM PITFALL: The three homography traps

(1) **Forgetting to divide by $\bar{w}'$.** With affine matrices you get away with it; with a homography you get a badly wrong answer and no warning. (2) **Claiming a homography preserves parallelism.** It does not — that is the whole point. It preserves straightness and the cross-ratio, and nothing else. (3) **Treating $h_{33} = 1$ as a fact rather than a choice.** It is a normalisation you imposed. If a problem states $\|\mathbf{h}\| = 1$ instead, the entries differ by a common factor and both answers are correct.

14.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Applying a homography, and locating its vanishing point

Problem. Given $\mathbf{H} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0.01 & 0 & 1 \end{bmatrix}$, (a) find the images of $(0,0)$, $(50,0)$ and $(100,0)$; (b) show that equally spaced input points do *not* map to equally spaced output points; (c) find the vanishing point of the x -direction; (d) verify that a straight line maps to a straight line.

Step 1 (a): Image of $(0,0)$.

$$\mathbf{H} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \longrightarrow \left(\frac{0}{1}, \frac{0}{1} \right) = (0,0).$$

Step 2: Image of $(50,0)$.

$$\mathbf{H} \begin{bmatrix} 50 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} (2)(50) + 0 + 0 \\ 0 + 0 + 0 \\ (0.01)(50) + 0 + 1 \end{bmatrix} = \begin{bmatrix} 100 \\ 0 \\ 1.5 \end{bmatrix} \longrightarrow \left(\frac{100}{1.5}, \frac{0}{1.5} \right) = (66.67, 0).$$

Step 3: Image of $(100,0)$.

$$\mathbf{H} \begin{bmatrix} 100 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 200 \\ 0 \\ 2.0 \end{bmatrix} \longrightarrow \left(\frac{200}{2}, 0 \right) = (100, 0).$$

Step 4 (b): Compare the spacings.

Input x	Input gap	Output x'	Output gap
0	—	0	—
50	50	66.67	66.67
100	50	100.00	33.33

Equal input gaps of 50 produce unequal output gaps of 66.67 and 33.33: the spacing *compresses* as x grows, exactly the foreshortening you see looking down a railway track. A **homography does not preserve ratios of lengths along a line** — which is precisely the property affine transformations *do* preserve. What survives instead is the *cross-ratio* of four collinear points.

Step 5 (c): The vanishing point of the x -direction.

The direction $(1, 0)$ is the point at infinity $\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$:

$$\mathbf{H} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 0.01 \end{bmatrix} \longrightarrow \left(\frac{2}{0.01}, 0 \right) = (200, 0).$$

Every horizontal line in the input converges, in the output, to $(200, 0)$. Sanity check against Step 4: as input x grows,

$$\lim_{x \rightarrow \infty} \frac{2x}{0.01x + 1} = \frac{2}{0.01} = 200. \checkmark$$

Step 6 (d): A straight line maps to a straight line.

Take the vertical line $x = 50$, parameterised as $(50, t)$:

$$\mathbf{H} \begin{bmatrix} 50 \\ t \\ 1 \end{bmatrix} = \begin{bmatrix} 100 \\ 2t \\ 1.5 \end{bmatrix} \longrightarrow \left(\frac{100}{1.5}, \frac{2t}{1.5} \right) = (66.67, 1.333t).$$

For every t the output has $x' = 66.67$: the image is the vertical line $x' = 66.67$. Straightness is preserved, although the y -spacing has been scaled by 1.333. $\checkmark$

Step 7: Interpret $h_{31} = 0.01$.

Setting $h_{31} = 0$ would give $\tilde{w}' = 1$ everywhere, making the matrix affine (pure scaling by 2) with the vanishing point pushed to infinity. The small value 0.01 is what injects perspective; the vanishing point sits at $h_{11}/h_{31} = 2/0.01 = 200$, so a *smaller* h_{31} pushes the horizon farther away and the distortion becomes milder.

Answer.

$(0, 0) \mapsto (0, 0)$, $(50, 0) \mapsto (66.67, 0)$, $(100, 0) \mapsto (100, 0)$; equal spacing is not preserved; the x -direction vanishing point is $(200, 0)$; straight lines remain straight.

Forward and Backward Image Warping

15.1 Intuitive Theory and Definition

You now have a transformation T . Applying it to *points* is easy. Applying it to an *image* raises a practical problem: an image is a grid of discrete pixels, and T sends grid positions to non-grid positions.

PHYSICAL ANALOGY: Two ways to repaint a mosaic

You have a mosaic and want to produce a rotated copy on a fresh blank grid.

Forward (push): pick up each tile from the original, compute where it should go, and drop it there. Problem: the computed destination usually falls *between* four cells of the new grid, so you must decide which cell gets the tile. Worse, when the copy is bigger than the original, some new cells never receive any tile at all — you get **holes**. When it is smaller, several tiles land on the same cell — you get **collisions**.

Backward (pull): stand on each *empty* cell of the new grid and ask “which point of the original should I be showing?”. Compute that source position with T^{-1} , then read the colour there, interpolating if it falls between original tiles. Every new cell is visited exactly once, so there are no holes and no collisions.

KEY DEFINITION: Forward and inverse warping

Forward warping iterates over *source* pixels and applies T to find their destination. **Inverse (backward) warping** iterates over *destination* pixels and applies T^{-1} to find their source. Inverse warping is the method used in practice, in every graphics and vision library.

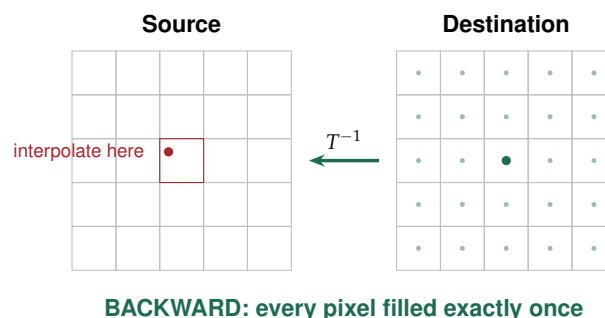
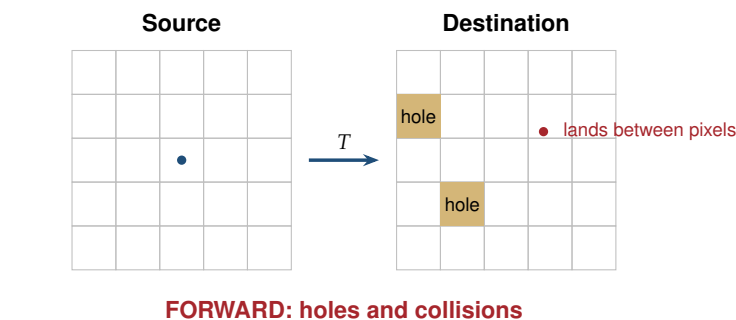


Figure 15.1: Forward warping pushes source pixels forward and leaves gaps. Backward warping pulls from the source for every destination pixel, guaranteeing complete coverage.

Source deck: *Forward_and_Backward_Image_Warping, slides 3–15; lec02_image_processing, slides 67–71*

15.2 Mathematical Formulation

IMPORTANT FORMULA: The two algorithms

Forward warping

```
for each  $(x, y)$  in the source:
     $(x', y') \leftarrow T(x, y)$ 
     $g(\text{round}(x'), \text{round}(y')) \leftarrow f(x, y)$ 
```

Inverse warping

```
for each  $(x', y')$  in the destination:
     $(x, y) \leftarrow T^{-1}(x', y')$ 
     $g(x', y') \leftarrow \text{bilinear\_interpolate}(f, x, y)$ 
```

Why inverse warping needs interpolation and forward warping needs splatting.

In both cases the computed coordinate is fractional; the difference is *which* grid it is fractional on. Forward warping produces a fractional *destination*, so the value must be distributed (“splatted”) over up to four output pixels, with accumulated weights tracked and divided out afterwards. Inverse warping produces a fractional *source*, so the value is simply *read* using the bilinear interpolation of Topic 4 — far simpler, with no bookkeeping.

The requirement: T^{-1} must exist.

For every transformation in the hierarchy it does, provided the matrix is non-singular:

$$\text{Translation: } \mathbf{T}(b_x, b_y)^{-1} = \mathbf{T}(-b_x, -b_y) \quad (15.1)$$

$$\text{Rotation: } \mathbf{R}(\theta)^{-1} = \mathbf{R}(-\theta) = \mathbf{R}(\theta)^\top \quad (15.2)$$

$$\text{Scaling: } \mathbf{S}(a, b)^{-1} = \mathbf{S}(1/a, 1/b) \quad (a, b \neq 0) \quad (15.3)$$

$$\text{Affine: } \begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}^{-1} = \begin{bmatrix} \mathbf{A}^{-1} & -\mathbf{A}^{-1}\mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \quad (\det \mathbf{A} \neq 0) \quad (15.4)$$

$$\text{Homography: } \mathbf{H}^{-1}, \text{ which is again a homography} \quad (15.5)$$

EXAM PITFALL: The direction of the inverse is the exam’s favourite trap

To produce an output rotated by $+\theta$, the inverse warp samples the input at $\mathbf{R}(-\theta)\mathbf{x}'$. Students routinely apply $\mathbf{R}(+\theta)$ in the destination loop and get an image rotated the *wrong way*. Say it out loud each time: “I am standing on the output pixel and asking where in the input it came from, so I apply the inverse.”

15.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Inverse-warping a scaled image, with interpolation

Problem. A 4×4 image is to be scaled by a factor of 2 in both directions, producing an 8×8 output. The source is

$$f = \begin{bmatrix} 0 & 10 & 20 & 30 \\ 40 & 50 & 60 & 70 \\ 80 & 90 & 100 & 110 \\ 120 & 130 & 140 & 150 \end{bmatrix}$$

indexed $f[y][x]$ with x and y running 0..3. (a) Write the forward and inverse transformations. (b) Compute the output value at destination pixel $(x', y') = (3, 5)$. (c) Explain what forward warping would have produced instead.

Step 1 (a): The forward transformation.

$$S = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad x' = 2x, \quad y' = 2y.$$

Step 2: The inverse transformation.

$$S^{-1} = \begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad x = 0.5 x', \quad y = 0.5 y'.$$

Step 3 (b): Map the destination pixel back to the source.

$$x = 0.5(3) = 1.5, \quad y = 0.5(5) = 2.5.$$

The source coordinate $(1.5, 2.5)$ lies *between* pixels, so we interpolate.

Step 4: Identify the four surrounding source pixels.

$$x_1 = \lfloor 1.5 \rfloor = 1, \quad x_2 = 2, \quad y_1 = \lfloor 2.5 \rfloor = 2, \quad y_2 = 3.$$

Reading from f (row = y , column = x):

$$Q_{11} = f[2][1] = 90, \quad Q_{21} = f[2][2] = 100, \quad Q_{12} = f[3][1] = 130, \quad Q_{22} = f[3][2] = 140.$$

Step 5: Compute the fractional offsets.

$$a = 1.5 - 1 = 0.5, \quad b = 2.5 - 2 = 0.5.$$

Step 6: Bilinear weights.

All four are $(1 - 0.5)(1 - 0.5) = 0.25$, since both offsets are exactly one half. They sum to $4 \times 0.25 = 1$. ✓

Step 7: Weighted sum.

$$g[5][3] = 0.25(90) + 0.25(100) + 0.25(130) + 0.25(140) = \frac{460}{4} = 115.$$

Step 8: Two independent checks.

Two-stage route — bottom edge: $0.5(90) + 0.5(100) = 95$; top edge: $0.5(130) + 0.5(140) = 135$; vertical: $0.5(95) + 0.5(135) = 115$. ✓ Structural check — the source is a linear ramp rising 10 per column and 40 per row, so the value at $(1.5, 2.5)$ should be $90 + 0.5(10) + 0.5(40) = 90 + 5 + 20 = 115$. ✓

Step 9 (c): What forward warping would give.

Forward warping visits each of the 16 source pixels and sends it to $(2x, 2y)$, so the destinations are only the *even* coordinates:

$$(0,0), (2,0), (4,0), (6,0), (0,2), (2,2), \dots, (6,6)$$

— just 16 of the 64 output pixels. The other 48, including our target $(3,5)$, are **never written to** and remain holes. Specifically, $(3,5)$ has odd coordinates and so can never be the image of any integer source pixel.

Step 10: Quantify.

Coverage under forward warping is $16/64 = 25\%$; under inverse warping it is 100%. Every hole would need a separate hole-filling pass — which is, in effect, interpolation done later and worse.

Answer.

$g[5][3] = 115$ by inverse warping with bilinear interpolation. Forward warping would have left that pixel — and 75% of the output — empty.

PART III

Image Formation and the Camera Model

Image Formation and Perspective Projection

16.1 Intuitive Theory and Definition

Part II moved points around *within* the image plane. Part III asks the prior question: how did a point in the three-dimensional world get onto that plane at all?

KEY DEFINITION: Image formation

The projection of a 3D scene onto a 2D plane. Every image is the record of light rays converging from the world onto a sensor, and the geometry of that convergence determines where each world point appears.

PHYSICAL ANALOGY: Why a bare sensor sees nothing

Hold a sheet of photographic film up to a window. Every point on the film receives light from *every* point of the scene simultaneously, so the film records a uniform grey fog with no image at all. Now put a wall with a single tiny hole between the scene and the film. Now each point of the film can only receive light along *one* ray — the one that happens to pass through the hole. Suddenly there is a sharp image. The hole did not add information; it *removed* rays, and by removing them it made the remaining ones interpretable. That hole is the pinhole, and an ideal camera is the geometry of those surviving rays.

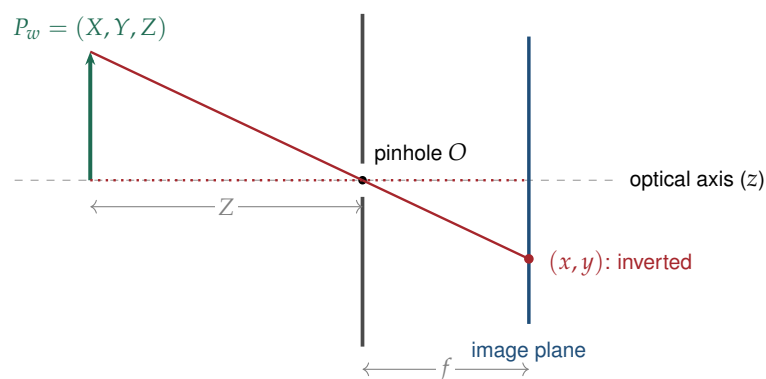


Figure 16.1: The pinhole camera. A world point at height Y and depth Z projects through the pinhole onto the image plane at distance f behind it. The image is *inverted*, which is why the projection formula carries a minus sign — usually absorbed by placing a “virtual” image plane in front of the pinhole instead.

Source deck: *s3_1_Perspective_Projection*, slides 1–6

16.2 Mathematical Formulation and Derivation

16.2.1 The perspective projection equations

Step 1: Set up the coordinate frame.

Put the pinhole at the origin with the optical axis along z . A world point is $P = (X, Y, Z)$ with $Z > 0$ (in front of the camera). The image plane sits at $z = f$, where f is the **focal length**.

Step 2: Draw the two similar triangles.

The ray from P to the origin and its continuation to the image plane form two triangles sharing the vertex at O . The large triangle has horizontal extent Z and vertical extent Y ; the small one has horizontal extent f and vertical extent y .

Step 3: Write the similarity relation.

$$\frac{y}{f} = \frac{Y}{Z} \quad (\text{A})$$

Step 4: Repeat in the horizontal plane.

$$\frac{x}{f} = \frac{X}{Z} \quad (\text{B})$$

Step 5: Solve (A) and (B).

IMPORTANT FORMULA: Perspective projection

$$x = f \frac{X}{Z}, \quad y = f \frac{Y}{Z}$$

The defining feature is the **division by Z** . It makes projection non-linear in Cartesian coordinates, and it is the source of every perspective effect: distant objects appear smaller, parallel lines converge, and depth cannot be recovered from a single image (because (X, Y, Z) and $(2X, 2Y, 2Z)$ produce exactly the same (x, y)).

16.2.2 Magnification

Step 6: Take a small object of height ΔY at depth Z .

Its image height is

$$\Delta y = f \frac{\Delta Y}{Z}.$$

Step 7: Define magnification as the ratio of image size to object size.

IMPORTANT FORMULA: Image magnification

$$m = \frac{\Delta y}{\Delta Y} = \frac{f}{Z}$$

Magnification is proportional to focal length and inversely proportional to depth. Doubling f doubles the image size (a telephoto lens); doubling Z halves it. This single formula answers most “how big will it appear?” questions.

16.2.3 Vanishing points, derived properly

Step 8: Parameterise a 3D line.

A line through the point $\mathbf{A} = (A_x, A_y, A_z)$ with direction $\mathbf{d} = (d_x, d_y, d_z)$ is

$$P(t) = \mathbf{A} + t \mathbf{d} = (A_x + td_x, A_y + td_y, A_z + td_z), \quad t \in \mathbb{R}.$$

Step 9: Project it.

$$x(t) = f \frac{A_x + td_x}{A_z + td_z}, \quad y(t) = f \frac{A_y + td_y}{A_z + td_z}.$$

Step 10: Take the limit as the point recedes to infinity.

Divide numerator and denominator by t and let $t \rightarrow \infty$:

$$\lim_{t \rightarrow \infty} x(t) = \lim_{t \rightarrow \infty} f \frac{A_x/t + d_x}{A_z/t + d_z} = f \frac{d_x}{d_z}, \quad \lim_{t \rightarrow \infty} y(t) = f \frac{d_y}{d_z}.$$

IMPORTANT FORMULA: Vanishing point of a 3D direction

$$\mathbf{v} = \left(f \frac{d_x}{d_z}, f \frac{d_y}{d_z} \right)$$

The limit depends only on the direction $\mathbf{d}$, not on the starting point $\mathbf{A}$. That is the theorem: *all* 3D lines sharing a direction project to image lines that converge on *one* common point. Two special cases worth knowing:

- $d_z = 0$ — a direction parallel to the image plane. The vanishing point is at infinity, so those lines stay parallel in the image. This is why a building photographed square-on has vertical edges that do *not* converge.
- $\mathbf{d} = (0, 0, 1)$ — the viewing direction itself. Its vanishing point is $(0, 0)$, the principal point: a road running straight away from you vanishes at the centre of the image.

KEY DEFINITION: Horizon line

The set of vanishing points of all directions lying in a given 3D plane. For the ground plane, that set is the horizon. Since a vanishing point depends only on direction, the horizon depends only on the *orientation* of the camera, never on its position — which is why the horizon never moves as you walk toward it.

[INSERT IMAGE HERE: Reference s3_1_Perspective_Projection, slides 7–12 — “Vanishing Point”, “Finding the Vanishing Point”, “Use of Vanishing Point in Art”, “False Perspective”]

A photograph of a straight railway or road receding to the horizon, with the two rails drawn in and extended by dashed lines until they intersect at a single marked point on the horizon line. Two Renaissance paintings follow, overlaid with construction lines showing that the architectural orthogonals all meet at one vanishing point placed deliberately behind the subject’s head. The final panel is Hogarth’s “False Perspective” engraving, in which the perspective cues are deliberately inconsistent — a fisherman’s line passes behind a distant figure, a sign hangs from two buildings at incompatible depths — illustrating that the visual system reads depth from exactly these projective cues.

16.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Photographing a building: size, depth and vanishing point

Problem. A camera has focal length $f = 50$ mm with a sensor of 36×24 mm. (a) A person 1.8 m tall stands 20 m away. How tall is their image, in mm and in pixels if the sensor is 6000 pixels wide? (b) How far away must they stand for their image to be exactly half the sensor height? (c) The camera looks down a straight street whose direction in camera coordinates is $\mathbf{d} = (0.1, 0, 1)$. Where is the vanishing point? (d) A lamp post of the same height as the person stands 40 m away — what is the ratio of the two image heights?

Step 1: Fix consistent units.

Work in millimetres throughout:

$$f = 50 \text{ mm}, \quad Y = 1.8 \text{ m} = 1800 \text{ mm}, \quad Z = 20 \text{ m} = 20\,000 \text{ mm}.$$

Step 2 (a): Apply the projection formula.

$$y = f \frac{Y}{Z} = 50 \times \frac{1800}{20\,000} = 50 \times 0.09 = 4.5 \text{ mm}.$$

Step 3: Convert to pixels.

The sensor is 36 mm wide and 6000 pixels wide, so the pixel pitch is

$$\frac{36 \text{ mm}}{6000 \text{ px}} = 0.006 \text{ mm per pixel},$$

and therefore

$$y_{\text{px}} = \frac{4.5}{0.006} = 750 \text{ pixels}.$$

Sanity check: the sensor is 24 mm = 4000 pixels tall, and $750/4000 = 18.75\%$ of the frame. A person filling under a fifth of the frame at 20 m with a standard lens is exactly what photographic experience predicts. ✓

Step 4 (b): Invert the formula for the required depth.

Half the sensor height is $y = 12 \text{ mm}$. Rearranging $y = fY/Z$:

$$Z = \frac{fY}{y} = \frac{50 \times 1800}{12} = \frac{90\,000}{12} = 7500 \text{ mm} = 7.5 \text{ m}.$$

Step 5: Cross-check with magnification.

$$m = \frac{f}{Z} = \frac{50}{7500} = 0.006\bar{6}, \quad \Delta y = m \Delta Y = 0.006\bar{6} \times 1800 = 12 \text{ mm}. \quad \checkmark$$

Step 6 (c): Vanishing point of the street direction.

With $\mathbf{d} = (0.1, 0, 1)$:

$$v_x = f \frac{d_x}{d_z} = 50 \times \frac{0.1}{1} = 5 \text{ mm}, \quad v_y = f \frac{d_y}{d_z} = 50 \times \frac{0}{1} = 0 \text{ mm}.$$

So the vanishing point sits 5 mm to the right of the image centre, on the horizontal centre line. In pixels:

$$v_x = \frac{5}{0.006} = 833 \text{ pixels right of centre}.$$

Since the image is 6000 pixels wide, the centre is at column 3000, so the street vanishes at approximately column 3833 — comfortably inside the frame.

Step 7 (d): Ratio of image heights at two depths.

$$\frac{y_{20}}{y_{40}} = \frac{fY/20\,000}{fY/40\,000} = \frac{40\,000}{20\,000} = 2.$$

The object twice as far away appears exactly half as tall. Explicitly,

$$y_{40} = 50 \times \frac{1800}{40\,000} = 2.25 \text{ mm} = 375 \text{ pixels},$$

half the 750 pixels of Step 3. ✓

Step 8: Note the deeper point.

Steps 2 and 7 together demonstrate the depth ambiguity of a single image: a 1.8 m person at 20 m and a 0.9 m child at 10 m produce *identical* 4.5 mm images. Nothing in one photograph distinguishes them. This is the ambiguity of Topic 1, and resolving it requires either a second viewpoint (stereo) or a known calibration object — which is where Topic 24 goes.

Answer.

(a) 4.5 mm = 750 px; (b) 7.5 m; (c) 5 mm right of centre ($\approx$ column 3833); (d) the ratio is exactly 2.

EXAM PITFALL: Unit mismatches sink perspective problems

Focal length comes in millimetres, scene distances in metres, and answers are wanted in pixels. Convert everything to one unit before touching the formula, and convert to pixels only at the very end using the pixel pitch. A second trap: f in *millimetres* and f in *pixels* are different numbers — the next topic calls the latter f_x and f_y , and the conversion is $f_{\text{px}} = f_{\text{mm}} / \text{pixel pitch}$.

The Linear Camera Model

17.1 Intuitive Theory and Definition

Perspective projection maps a point in *camera* coordinates to a position on the image plane in millimetres. A real system needs two more stages: the world does not use the camera's coordinate frame, and the sensor does not use millimetres. The linear camera model chains all three stages into one matrix.

PHYSICAL ANALOGY: Three changes of address

A world point's journey to a pixel is three changes of address. *First*, it is re-described from the world's frame into the camera's frame — the same point, new axes. This is a rigid motion: rotate and translate, nothing more. *Second*, it is projected through the pinhole onto the image plane. This is the only step that loses information, because depth is divided away. *Third*, the millimetre position on the plane is converted into a row and column on the sensor, which requires knowing the pixel size and where the sensor's corner is relative to the optical axis. Only the last two are properties of the *camera*; the first is a property of where you *put* the camera. Hence the terminology.

KEY DEFINITION: Intrinsic versus extrinsic parameters

Intrinsic parameters describe the camera itself — focal length, pixel size, the principal point, skew — and do not change when you move the camera. **Extrinsic** parameters describe the camera's pose in the world — its rotation $\mathbf{R}$ and translation $\mathbf{t}$ — and change every time you move it.

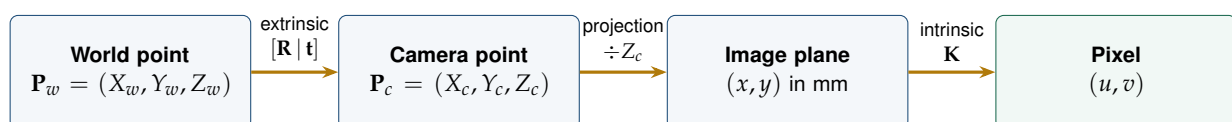


Figure 17.1: The forward imaging model. Extrinsics place the camera in the world; projection flattens 3D to 2D; intrinsics convert millimetres to pixels.

Source deck: *s3_2_Linear_Camera_Model*, slides 2–8, 21–28

17.2 Mathematical Formulation and Derivation

17.2.1 Stage 3 first: image plane to sensor

Step 1: Convert millimetres to pixels.

Let the sensor have m_x pixels per millimetre horizontally and m_y vertically. Then

$$u = m_x x, \quad v = m_y y.$$

Step 2: Account for the origin offset.

Pixel indices are measured from a *corner* of the sensor, while x and y are measured from the optical axis. Let the optical axis pierce the sensor at pixel (o_x, o_y) — the **principal point**. Then

$$u = m_x x + o_x, \quad v = m_y y + o_y.$$

Step 3: Substitute the projection equations $x = fX_c/Z_c, y = fY_c/Z_c$.

$$u = m_x f \frac{X_c}{Z_c} + o_x, \quad v = m_y f \frac{Y_c}{Z_c} + o_y.$$

Step 4: Absorb the products into two focal lengths measured in pixels.

Define $f_x = m_x f$ and $f_y = m_y f$:

$$u = f_x \frac{X_c}{Z_c} + o_x, \quad v = f_y \frac{Y_c}{Z_c} + o_y.$$

Note $f_x \neq f_y$ in general — not because the lens differs by direction, but because the photosites may not be square.

17.2.2 Removing the division with homogeneous coordinates

Step 5: Multiply through by Z_c .

$$Z_c u = f_x X_c + o_x Z_c, \quad Z_c v = f_y Y_c + o_y Z_c.$$

These equations are now *linear* in (X_c, Y_c, Z_c) — the non-linearity has been moved into the scale factor Z_c on the left.

Step 6: Write them as a matrix product with an unknown scale.

Writing $s = Z_c$,

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}.$$

Verify row 3: it gives $s(1) = Z_c$, confirming $s = Z_c$, exactly as assumed. Rows 1 and 2 then reproduce Step 5. ✓

IMPORTANT FORMULA: The intrinsic matrix $\mathbf{K}$

$$\mathbf{K} = \begin{bmatrix} f_x & \gamma & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

Five intrinsic parameters: f_x, f_y (focal lengths in pixels), o_x, o_y (principal point in pixels), and γ (**skew**, non-zero only if the sensor axes are not perpendicular — essentially always 0 for real cameras). $\mathbf{K}$ is upper-triangular, a fact that Topic 24 exploits to recover $\mathbf{K}$ from a calibrated projection matrix.

17.2.3 Stage 1: world to camera (extrinsics)

Step 7: A change of coordinate frame is a rigid transformation.

$$\mathbf{P}_c = \mathbf{R} \mathbf{P}_w + \mathbf{t}$$

where $\mathbf{R}$ is a 3×3 rotation matrix and $\mathbf{t}$ a 3-vector translation.

Step 8: Count the parameters.

$\mathbf{R}$ has nine entries but satisfies $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$ — six independent constraints (three unit-length conditions and three orthogonality conditions) — leaving $9 - 6 = 3$ degrees of freedom, the three rotation angles. Adding $\mathbf{t}$'s three gives **six extrinsic parameters**.

KEY DEFINITION: Orthonormal matrix

$\mathbf{R}$ is orthonormal when its columns (equivalently its rows) are mutually perpendicular unit vectors:

$$\mathbf{r}_i \cdot \mathbf{r}_j = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \iff \mathbf{R}^\top \mathbf{R} = \mathbf{I} \iff \mathbf{R}^{-1} = \mathbf{R}^\top.$$

With $\det \mathbf{R} = +1$ it is a pure rotation. This property is what makes inverting a camera pose trivial: $\mathbf{P}_w = \mathbf{R}^\top (\mathbf{P}_c - \mathbf{t})$.

Step 9: Write the extrinsic step in homogeneous form.

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}.$$

Note the shape: 3×4 acting on a 4-vector, producing a 3-vector.

17.2.4 The complete projection matrix

Step 10: Compose the two stages.

IMPORTANT FORMULA: The linear camera model

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\mathbf{K}}_{3 \times 3} \underbrace{\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}}_{3 \times 4} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = \underbrace{\mathbf{P}}_{3 \times 4} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}, \quad \mathbf{P} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}.$$

Parameter count: 5 intrinsic + 6 extrinsic = **11** degrees of freedom. $\mathbf{P}$ has 12 entries but, like a homography, is defined only up to scale, so $12 - 1 = 11$. ✓ The two counts agree — a reassuring consistency check, and one worth quoting in an exam.

Step 11: Recovering the pixel coordinates.

After computing $\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \mathbf{P} \mathbf{P}_w$, divide by the third component:

$$u = \frac{\tilde{u}}{\tilde{w}}, \quad v = \frac{\tilde{v}}{\tilde{w}},$$

and note that $\tilde{w} = s = Z_c$ is the point's *depth in camera coordinates* — a useful by-product that tells you whether the point is in front of the camera ($Z_c > 0$) or behind it.

EXAM PITFALL: $\mathbf{t}$ is not the camera's position

The most common conceptual error in this topic. In $\mathbf{P}_c = \mathbf{R}\mathbf{P}_w + \mathbf{t}$, the vector $\mathbf{t}$ is the position of the *world* origin expressed in *camera* coordinates. The camera centre in world coordinates is obtained by setting $\mathbf{P}_c = \mathbf{0}$:

$$\mathbf{0} = \mathbf{R}\mathbf{C} + \mathbf{t} \implies \mathbf{C} = -\mathbf{R}^\top \mathbf{t}.$$

Reporting $\mathbf{t}$ when asked for the camera position loses the mark; the minus sign and the transpose are both essential.

17.3 Solved Numerical Example**SOLVED NUMERICAL EXAMPLE: Building $\mathbf{P}$ and projecting a world point**

Problem. A camera has $f_x = f_y = 800$ pixels, principal point $(o_x, o_y) = (320, 240)$, zero skew. It is rotated 90° about the world y -axis and its centre sits at world position $\mathbf{C} = (10, 0, 0)$ metres. (a) Write $\mathbf{K}$. (b) Write $\mathbf{R}$ and compute $\mathbf{t}$. (c) Form $\mathbf{P}$. (d) Project the world point $(10, 1, 5)$ and (e) verify the answer independently.

Step 1 (a): The intrinsic matrix.

$$\mathbf{K} = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix}.$$

Step 2 (b): The rotation matrix for 90° about y .

The standard form is

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} \xrightarrow{\theta=90^\circ} \mathbf{R} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}.$$

Check orthonormality: column 1 is $(0, 0, -1)$ with length 1; column 3 is $(1, 0, 0)$ with length 1; their dot product is $(0)(1) + (0)(0) + (-1)(0) = 0$. ✓ And $\det \mathbf{R} = 0[(1)(0) - (0)(0)] - 0 + 1[(0)(0) - (1)(-1)] = 1$. ✓

Step 3: Compute $\mathbf{t}$ from the camera centre.

From $\mathbf{C} = -\mathbf{R}^\top \mathbf{t}$ we get $\mathbf{t} = -\mathbf{R}\mathbf{C}$:

$$\mathbf{t} = - \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 10 \\ 0 \\ 0 \end{bmatrix} = - \begin{bmatrix} (0)(10) + (0)(0) + (1)(0) \\ (0)(10) + (1)(0) + (0)(0) \\ (-1)(10) + (0)(0) + (0)(0) \end{bmatrix} = - \begin{bmatrix} 0 \\ 0 \\ -10 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 10 \end{bmatrix}.$$

Step 4: Assemble the extrinsic block.

$$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 10 \end{bmatrix}.$$

Step 5 (c): Multiply to form $\mathbf{P} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}]$.

$$\mathbf{P} = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 10 \end{bmatrix}.$$

Row 1 of $\mathbf{K}$ is $[800 \ 0 \ 320]$, applied to each column of the extrinsic block:

$$(800)(0) + (0)(0) + (320)(-1) = -320; \quad 0; \quad (800)(1) = 800; \quad (320)(10) = 3200.$$

Row 2, $[0 \ 800 \ 240]$:

$$(240)(-1) = -240; \quad 800; \quad 0; \quad (240)(10) = 2400.$$

Row 3, $[0 \ 0 \ 1]$:

$$-1; \quad 0; \quad 0; \quad 10.$$

$$\mathbf{P} = \begin{bmatrix} -320 & 0 & 800 & 3200 \\ -240 & 800 & 0 & 2400 \\ -1 & 0 & 0 & 10 \end{bmatrix}$$

Step 6 (d): Project the world point $(10, 1, 5)$.

$$\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \mathbf{P} \begin{bmatrix} 10 \\ 1 \\ 5 \\ 1 \end{bmatrix}.$$

$$\tilde{u} = (-320)(10) + (0)(1) + (800)(5) + (3200)(1) = -3200 + 4000 + 3200 = 4000 \quad (17.1)$$

$$\tilde{v} = (-240)(10) + (800)(1) + (0)(5) + (2400)(1) = -2400 + 800 + 2400 = 800 \quad (17.2)$$

$$\tilde{w} = (-1)(10) + 0 + 0 + (10)(1) = -10 + 10 = 0 \quad (17.3)$$

Step 7: Interpret $\tilde{w} = 0$ — do not just divide.

A zero third coordinate means $Z_c = 0$: the point lies *in the plane of the pinhole*, neither in front of nor behind the camera. Its image is a point at infinity; the projection is undefined. This is a genuine geometric fact, not an arithmetic slip, and it happens because the camera sits at world $x = 10$ looking along the world x -direction, while the query point also sits at $x = 10$ — directly beside the camera.

Step 8: Choose a point that is actually visible and repeat.

Take $\mathbf{P}_w = (16, 1, 5)$, which is 6 m along the viewing direction:

$$\tilde{u} = (-320)(16) + 0 + (800)(5) + 3200 = -5120 + 4000 + 3200 = 2080 \quad (17.4)$$

$$\tilde{v} = (-240)(16) + (800)(1) + 0 + 2400 = -3840 + 800 + 2400 = -640 \quad (17.5)$$

$$\tilde{w} = (-1)(16) + 10 = -6 \quad (17.6)$$

$$u = \frac{2080}{-6} = -346.7, \quad v = \frac{-640}{-6} = 106.7.$$

Here $\tilde{w} = Z_c = -6$, i.e. the point is *behind* the camera under this rotation convention: $\mathbf{R}_y(+90^\circ)$ turned the camera to look along world $-x$, so a point at larger x is behind it. Taking instead $\mathbf{P}_w = (4, 1, 5)$:

$$\tilde{u} = (-320)(4) + 0 + 4000 + 3200 = -1280 + 7200 = 5920 \quad (17.7)$$

$$\tilde{v} = (-240)(4) + 800 + 0 + 2400 = -960 + 3200 = 2240 \quad (17.8)$$

$$\tilde{w} = -4 + 10 = 6 \quad (17.9)$$

$$u = \frac{5920}{6} = 986.7, \quad v = \frac{2240}{6} = 373.3.$$

Step 9 (e): Verify independently through the three stages.

First, world to camera:

$$\mathbf{P}_c = \mathbf{R}\mathbf{P}_w + \mathbf{t} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \\ 5 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 10 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \\ -4 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 10 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \\ 6 \end{bmatrix}.$$

So $Z_c = 6$, matching $\tilde{w} = 6$. ✓ Second, project and convert to pixels:

$$u = f_x \frac{X_c}{Z_c} + o_x = 800 \left(\frac{5}{6} \right) + 320 = 666.7 + 320 = 986.7 \quad \checkmark$$

$$v = f_y \frac{Y_c}{Z_c} + o_y = 800 \left(\frac{1}{6} \right) + 240 = 133.3 + 240 = 373.3 \quad \checkmark$$

Both routes agree exactly.

Answer.

$\mathbf{K}, \mathbf{R}, \mathbf{t} = (0, 0, 10)^\top$ and $\mathbf{P}$ as boxed above. The point $(10, 1, 5)$ lies in the pinhole plane and has no image ($\tilde{w} = 0$). The point $(4, 1, 5)$ projects to $(986.7, 373.3)$ with depth $Z_c = 6$ m, confirmed by an independent stage-by-stage calculation.

PART IV

The Optimisation Toolkit

Constrained Optimisation and Lagrange Multipliers

18.1 Intuitive Theory and Definition

This topic looks like a detour into calculus. It is not. It is the mathematical engine of everything that follows: total least squares, homography fitting and camera calibration all end with the same problem — *minimise a quadratic form subject to a unit-norm constraint* — and that problem is solved with Lagrange multipliers.

KEY DEFINITION: Constrained optimisation problem

Find the extremum of an **objective function** $f(x, y)$ while a **constraint** $g(x, y) = c$ must hold exactly:

$$\max_{x,y} f(x, y) \quad \text{subject to} \quad g(x, y) = c.$$

The unconstrained condition $\nabla f = \mathbf{0}$ does *not* apply, because the true unconstrained extremum generally does not satisfy the constraint.

PHYSICAL ANALOGY: Walking a fenced path across a hillside

The function f is the altitude of a hillside. Unconstrained, you find the summit by walking uphill until the ground is flat. Now you are confined to a fenced circular path, $g(x, y) = c$, and you must find the highest point *on that path*. Walking the path, the ground rises, then levels, then falls. At the highest point on the path, the ground is *not* flat — it still slopes steeply, straight off to the side. What *is* true is that the slope has no component *along* the path: any further step along the fence would gain nothing. So at the solution, the uphill direction ∇f points exactly perpendicular to the path — and the perpendicular to the path is ∇g . Therefore ∇f and ∇g must be parallel. That is the entire method.

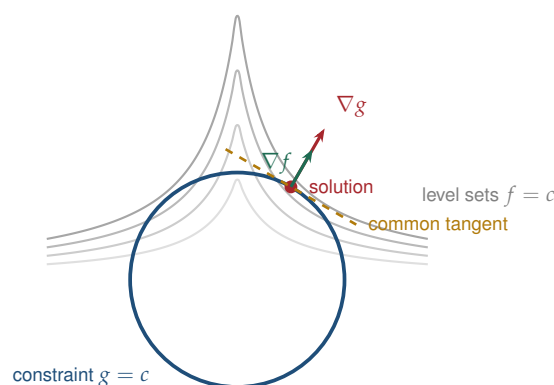


Figure 18.1: At the constrained optimum the level curve of f is *tangent* to the constraint curve, so their normals — the two gradients — are parallel. The multiplier λ is precisely the ratio of their lengths.

Source deck: *Constrained_Optimization_Problem*, slides 12–22

18.2 Mathematical Formulation and Derivation

18.2.1 Deriving the condition

Step 1: Parameterise the constraint curve.

Suppose the curve $g(x, y) = c$ is traced by $(x(t), y(t))$. On the curve the objective becomes a function of one variable,

$$F(t) = f(x(t), y(t)).$$

Step 2: Apply the stationarity condition to F .

At a constrained extremum,

$$\frac{dF}{dt} = 0.$$

Step 3: Expand by the chain rule.

$$\frac{dF}{dt} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt} = \nabla f \cdot \mathbf{T} = 0,$$

where $\mathbf{T} = (dx/dt, dy/dt)$ is the tangent vector to the constraint curve. So $\nabla f \perp \mathbf{T}$.

Step 4: Do the same to the constraint itself.

Since $g(x(t), y(t)) = c$ is constant along the curve, differentiating gives

$$\nabla g \cdot \mathbf{T} = 0,$$

so $\nabla g \perp \mathbf{T}$ as well.

Step 5: Conclude.

In the plane, two vectors perpendicular to the same non-zero vector must be parallel. Hence there exists a scalar λ with

$$\nabla f = \lambda \nabla g. \quad \blacksquare$$

IMPORTANT FORMULA: The method of Lagrange multipliers

To optimise f subject to $g = c$, solve the system

$$\nabla f = \lambda \nabla g, \quad g(x, y) = c,$$

i.e. in components

$$\frac{\partial f}{\partial x} = \lambda \frac{\partial g}{\partial x}, \quad \frac{\partial f}{\partial y} = \lambda \frac{\partial g}{\partial y}, \quad g(x, y) = c.$$

Equivalently, form the **Lagrangian**

$$\mathcal{L}(x, y, \lambda) = f(x, y) - \lambda(g(x, y) - c)$$

and set all three of its partial derivatives to zero. Differentiating with respect to λ reproduces the constraint automatically, which is why the Lagrangian formulation is the one used in practice.

KEY DEFINITION: What λ means

The multiplier is not merely bookkeeping. It is the **sensitivity** of the optimum to the

constraint level:

$$\lambda = \frac{df^*}{dc},$$

the rate at which the best achievable value improves as the constraint is relaxed. In the eigenvalue problems of Topics 20 and 24, λ turns out to be the eigenvalue — and the minimum residual — which is why those methods say “take the eigenvector of the smallest eigenvalue”.

18.2.2 The vector form used throughout the rest of the course

Step 6: State the problem that will recur.

Minimise

$$E(\mathbf{a}) = \|\mathbf{X}\mathbf{a}\|^2 \quad \text{subject to} \quad \|\mathbf{a}\|^2 = 1.$$

The constraint is needed because $\mathbf{a} = \mathbf{0}$ would otherwise be the trivial and useless minimiser.

Step 7: Form the Lagrangian.

$$\mathcal{L}(\mathbf{a}, \lambda) = \|\mathbf{X}\mathbf{a}\|^2 - \lambda (\|\mathbf{a}\|^2 - 1) = \mathbf{a}^\top \mathbf{X}^\top \mathbf{X} \mathbf{a} - \lambda (\mathbf{a}^\top \mathbf{a} - 1).$$

Step 8: Differentiate with respect to $\mathbf{a}$.

Using the standard identity $\partial(\mathbf{a}^\top \mathbf{M} \mathbf{a}) / \partial \mathbf{a} = 2\mathbf{M}\mathbf{a}$ for symmetric $\mathbf{M}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}} = 2\mathbf{X}^\top \mathbf{X} \mathbf{a} - 2\lambda \mathbf{a} = \mathbf{0}.$$

Step 9: Recognise the eigenvalue problem.

$$\mathbf{X}^\top \mathbf{X} \mathbf{a} = \lambda \mathbf{a}.$$

So the solution $\mathbf{a}$ must be an **eigenvector** of $\mathbf{X}^\top \mathbf{X}$.

Step 10: Determine which eigenvector.

Substitute back into the objective:

$$\|\mathbf{X}\mathbf{a}\|^2 = \mathbf{a}^\top \mathbf{X}^\top \mathbf{X} \mathbf{a} = \mathbf{a}^\top (\lambda \mathbf{a}) = \lambda \mathbf{a}^\top \mathbf{a} = \lambda \cdot 1 = \lambda.$$

IMPORTANT FORMULA: The master result of the second half of this course

$$\arg \min_{\|\mathbf{a}\|=1} \|\mathbf{X}\mathbf{a}\|^2 = \text{the eigenvector of } \mathbf{X}^\top \mathbf{X} \text{ with the **smallest** eigenvalue,}$$

and the value of the minimum is that smallest eigenvalue itself. Equivalently, it is the **last column of $\mathbf{V}$** in the singular value decomposition $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. Total least squares (Topic 20), homography fitting (Topic 23) and camera calibration (Topic 24) are all this one result wearing different clothes.

18.3 Solved Numerical Examples

Three worked problems follow. The first two are the lecturer’s own posted examples; the third connects the method to the vector form just derived.

SOLVED NUMERICAL EXAMPLE: Maximise $f(x, y) = x^2y$ on the unit circle

Problem. Maximise $f(x, y) = x^2y$ subject to $x^2 + y^2 = 1$.

Step 1: Identify the pieces.

$$f(x, y) = x^2y, \quad g(x, y) = x^2 + y^2, \quad c = 1.$$

Step 2: Compute both gradients.

$$\nabla f = \begin{bmatrix} \partial f / \partial x \\ \partial f / \partial y \end{bmatrix} = \begin{bmatrix} 2xy \\ x^2 \end{bmatrix}, \quad \nabla g = \begin{bmatrix} 2x \\ 2y \end{bmatrix}.$$

Step 3: Set $\nabla f = \lambda \nabla g$, giving two scalar equations.

$$2xy = \lambda (2x) \tag{I}$$

$$x^2 = \lambda (2y) \tag{II}$$

together with the constraint

$$x^2 + y^2 = 1. \tag{III}$$

Step 4: Solve (I). Two branches — handle both.

$$2xy - 2\lambda x = 0 \implies 2x(y - \lambda) = 0 \implies x = 0 \quad \text{or} \quad y = \lambda.$$

Step 5: Dispose of the branch $x = 0$.

If $x = 0$ then $f = x^2y = 0$, which is the smallest possible magnitude, not a maximum. (Constraint (III) would give $y = \pm 1$, and indeed $f(0, \pm 1) = 0$.) *Discard* — but note that discarding it requires the check just performed, not an assumption.

Step 6: Take the branch $y = \lambda$ and substitute into (II).

$$x^2 = 2y \cdot y = 2y^2.$$

Step 7: Substitute this into the constraint (III).

$$2y^2 + y^2 = 1 \implies 3y^2 = 1 \implies y^2 = \frac{1}{3} \implies y = \pm \frac{1}{\sqrt{3}}.$$

Step 8: Recover x .

$$x^2 = 2y^2 = \frac{2}{3} \implies x = \pm \sqrt{\frac{2}{3}}.$$

Step 9: List all four candidate points.

$$\left(\sqrt{\frac{2}{3}}, \frac{1}{\sqrt{3}} \right), \left(-\sqrt{\frac{2}{3}}, \frac{1}{\sqrt{3}} \right), \left(\sqrt{\frac{2}{3}}, -\frac{1}{\sqrt{3}} \right), \left(-\sqrt{\frac{2}{3}}, -\frac{1}{\sqrt{3}} \right).$$

Step 10: Evaluate f at each.

Since $f = x^2y$ and $x^2 = 2/3$ in every case, the sign of f follows the sign of y :

$$y = +\frac{1}{\sqrt{3}}: \quad f = \frac{2}{3} \cdot \frac{1}{\sqrt{3}} = \frac{2}{3\sqrt{3}} = \frac{2}{3} \sqrt{\frac{1}{3}} \approx +0.3849 \tag{18.1}$$

$$y = -\frac{1}{\sqrt{3}}: \quad f = -\frac{2}{3\sqrt{3}} \approx -0.3849 \tag{18.2}$$

Step 11: Select the maximum.

The two points with $y = +1/\sqrt{3}$ both achieve the maximum

$$f_{\max} = \frac{2}{3\sqrt{3}} = \frac{2\sqrt{3}}{9} \approx 0.3849,$$

attained at $(\pm\sqrt{2/3}, 1/\sqrt{3})$. The symmetry in x is expected: f depends on x only through x^2 .

Step 12: Sanity check against the constraint.

$$x^2 + y^2 = \frac{2}{3} + \frac{1}{3} = 1. \checkmark$$

And check the multiplier: $\lambda = y = 1/\sqrt{3} \approx 0.577$, so from (II), $x^2 = 2\lambda y = 2(0.577)(0.577) = 0.667 = 2/3$. $\checkmark$

Answer.

$$f_{\max} = \frac{2\sqrt{3}}{9} \approx 0.385 \text{ at } (\pm\sqrt{2/3}, 1/\sqrt{3}), \text{ with } \lambda = 1/\sqrt{3}.$$

SOLVED NUMERICAL EXAMPLE: Minimum fencing for a fixed area

Problem. A farmer wants a rectangular field of area 100 m^2 . The field has length x and width y . Find the dimensions that minimise the perimeter, i.e. the amount of fencing required.

Step 1: Write the objective — the perimeter.

$$P(x, y) = 2x + 2y.$$

Step 2: Write the constraint — the fixed area.

$$g(x, y) = xy = 100.$$

Step 3: Form the Lagrangian.

$$\mathcal{L}(x, y, \lambda) = 2x + 2y - \lambda (xy - 100).$$

Step 4: Take the three partial derivatives and set each to zero.

$$\frac{\partial \mathcal{L}}{\partial x} = 2 - \lambda y = 0 \tag{I}$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2 - \lambda x = 0 \tag{II}$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = -(xy - 100) = 0 \tag{III}$$

Note that (III) simply restores the constraint — the advantage of the Lagrangian form.

Step 5: Subtract (II) from (I) to eliminate the constant.

$$(2 - \lambda y) - (2 - \lambda x) = 0 \implies \lambda(x - y) = 0.$$

Step 6: Rule out $\lambda = 0$.

If $\lambda = 0$ then (I) reads $2 = 0$, a contradiction. Therefore $\lambda \neq 0$ and

$$x = y.$$

The optimal rectangle is a **square** — a result worth remembering in its own right.

Step 7: Apply the constraint.

$$x \cdot x = 100 \implies x^2 = 100 \implies x = 10 \text{ m}$$

(taking the positive root, since a length cannot be negative), and hence $y = 10$ m.

Step 8: Compute the minimum perimeter.

$$P_{\min} = 2(10) + 2(10) = 40 \text{ m.}$$

Step 9: Recover and interpret λ .

From (I), $\lambda = 2/y = 2/10 = 0.2$. Its meaning: relaxing the area constraint by 1 m^2 changes the required fencing by about 0.2 m. Check this directly — for area 101 , the optimal square has side $\sqrt{101} = 10.0499$ and perimeter $4(10.0499) = 40.1995$, an increase of $0.1995 \approx 0.2$. ✓

Step 10: Verify it is a minimum, not a maximum.

Test a non-square rectangle of the same area, say $x = 20$, $y = 5$:

$$P = 2(20) + 2(5) = 50 > 40. \checkmark$$

And $x = 50$, $y = 2$ gives $P = 104$. The perimeter grows without bound as the rectangle degenerates, confirming 40 m is the minimum.

Answer.

A $10 \text{ m} \times 10 \text{ m}$ square, requiring 40 m of fencing, with $\lambda = 0.2$ m per square metre.

SOLVED NUMERICAL EXAMPLE: The constrained eigenvalue problem, done by hand

Problem. Minimise $E(\mathbf{a}) = \|\mathbf{X}\mathbf{a}\|^2$ subject to $\|\mathbf{a}\| = 1$, where

$$\mathbf{X} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \\ 0 & 0 \end{bmatrix}.$$

Construct this as a Lagrange problem, solve it, and confirm the general result of Step 10 of the derivation.

Step 1: Form the scatter matrix $\mathbf{X}^\top \mathbf{X}$.

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 0 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 1 & 3 \\ 0 & 0 \end{bmatrix}.$$

Entry (1,1): $(3)(3) + (1)(1) + (0)(0) = 10$. Entry (1,2): $(3)(1) + (1)(3) + 0 = 6$. Entry (2,1): 6 by symmetry. Entry (2,2): $(1)(1) + (3)(3) = 10$.

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix}.$$

Step 2: Write the Lagrangian explicitly.

With $\mathbf{a} = (a_1, a_2)^\top$,

$$\mathcal{L} = 10a_1^2 + 12a_1a_2 + 10a_2^2 - \lambda(a_1^2 + a_2^2 - 1).$$

Step 3: Set the partial derivatives to zero.

$$\frac{\partial \mathcal{L}}{\partial a_1} = 20a_1 + 12a_2 - 2\lambda a_1 = 0 \quad (18.3)$$

$$\frac{\partial \mathcal{L}}{\partial a_2} = 12a_1 + 20a_2 - 2\lambda a_2 = 0 \quad (18.4)$$

Dividing both by 2 and collecting terms gives exactly

$$\begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \lambda \begin{bmatrix} a_1 \\ a_2 \end{bmatrix},$$

confirming the general eigenvalue result. ✓

Step 4: Solve the characteristic equation.

$$\det \begin{bmatrix} 10 - \lambda & 6 \\ 6 & 10 - \lambda \end{bmatrix} = (10 - \lambda)^2 - 36 = 0.$$

$$(10 - \lambda)^2 = 36 \implies 10 - \lambda = \pm 6 \implies \lambda = 4 \text{ or } \lambda = 16.$$

Step 5: Find the eigenvector for the *smaller* eigenvalue $\lambda = 4$.

$$\begin{bmatrix} 10 - 4 & 6 \\ 6 & 10 - 4 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 6 & 6 \\ 6 & 6 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \mathbf{0} \implies 6a_1 + 6a_2 = 0 \implies a_2 = -a_1.$$

So $\mathbf{a} \propto (1, -1)^\top$, normalised to

$$\mathbf{a} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.7071 \\ -0.7071 \end{bmatrix}.$$

Step 6: Verify the minimum equals $\lambda_{\min}$.

$$\mathbf{X}\mathbf{a} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0.7071 \\ -0.7071 \end{bmatrix} = \begin{bmatrix} (3)(0.7071) + (1)(-0.7071) \\ (1)(0.7071) + (3)(-0.7071) \\ 0 \end{bmatrix} = \begin{bmatrix} 1.4142 \\ -1.4142 \\ 0 \end{bmatrix}.$$

$$\|\mathbf{X}\mathbf{a}\|^2 = (1.4142)^2 + (-1.4142)^2 = 2 + 2 = 4 = \lambda_{\min}. \quad \checkmark$$

Step 7: Confirm the other eigenvector is worse.

For $\lambda = 16$: $(10 - 16)a_1 + 6a_2 = 0 \Rightarrow a_2 = a_1$, so $\mathbf{a} = \frac{1}{\sqrt{2}}(1, 1)^\top$. Then

$$\mathbf{X}\mathbf{a} = \begin{bmatrix} 4(0.7071) \\ 4(0.7071) \\ 0 \end{bmatrix}, \quad \|\mathbf{X}\mathbf{a}\|^2 = 8 + 8 = 16 = \lambda_{\max},$$

four times larger. The largest eigenvalue gives the *maximum*, the smallest gives the *minimum*. ✓

Answer.

$\mathbf{a} = \frac{1}{\sqrt{2}}(1, -1)^\top$ with minimum value 4, exactly the smallest eigenvalue of $\mathbf{X}^\top \mathbf{X}$ — the result that Topics 20, 23 and 24 will invoke without re-deriving.

EXAM PITFALL: Four Lagrange-multiplier traps

(1) **Dropping a branch.** When a factorisation gives $2x(y - \lambda) = 0$, both $x = 0$ and $y = \lambda$ are candidates. You must evaluate f on both and compare, not assume the interesting one. (2) **Forgetting the constraint equation.** You have three unknowns (x, y, λ) and therefore need three equations; the constraint is the third. (3) **Not checking maximum versus minimum.** The stationarity condition finds both. Evaluate f at every candidate, or — in the eigenvalue setting — remember that smallest eigenvalue means minimum. (4) **Sign of λ .** Writing $\mathcal{L} = f + \lambda(g - c)$ instead of $f - \lambda(g - c)$ flips the sign of λ but not the solution points. Both conventions appear in textbooks; be internally consistent.

PART V

Fitting: Lines, Curves and Robustness

Least-Squares Fitting of Lines and Curves

19.1 Intuitive Theory and Definition

Topic 8 gave you a set of edge pixels. They are not a line — they are a scattered cloud of points that *suggests* a line. Fitting is the act of choosing the single line (or curve, or transformation) that best explains a noisy set of measurements.

KEY DEFINITION: Fitting

Given a parametric model and a set of noisy data points, find the parameter values that minimise a chosen measure of disagreement between model and data. Three sub-problems must always be settled: which *model*, which *error measure*, and which *optimisation method*.

PHYSICAL ANALOGY: A taut string through a scatter of nails

Hammer nails into a board at your data points, then stretch a string across. You cannot touch every nail, so you position the string to be “as close as possible” to all of them. But close in what sense? If you measure the *vertical* gap from each nail to the string, you get ordinary least squares. If you measure the *shortest* (perpendicular) gap, you get total least squares. The two give different answers, and which is correct depends on where you believe the error lives.

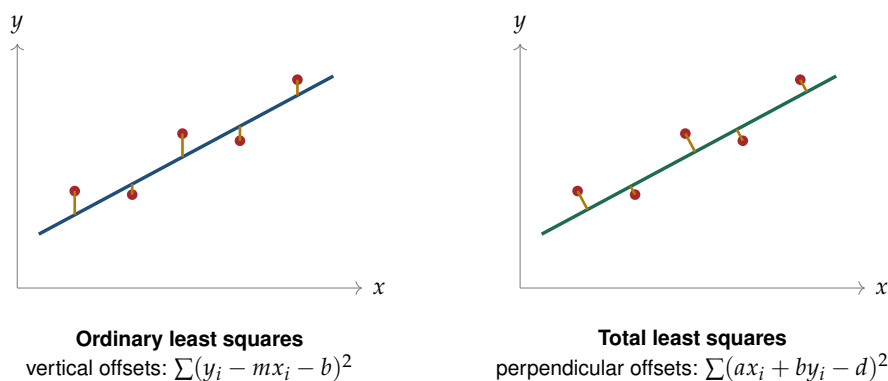


Figure 19.1: The same five points, the same line, two different error measures. The gold segments are the residuals being squared and summed.

Source deck: *Regression_least-squares_line_and_parabola_fitting*, slides 11–20; *lec09_fitting*, slides 6–12

19.2 Mathematical Formulation and Derivation

19.2.1 Line fitting by ordinary least squares

Step 1: Choose the model.

The line $y = mx + b$, with two unknowns m and b .

Step 2: Define the residual at each data point.

For the i -th measurement (x_i, y_i) , the model predicts $mx_i + b$, so the residual is

$$r_i = y_i - (mx_i + b).$$

Step 3: Define the objective as the sum of squared residuals.

$$E(m, b) = \sum_{i=1}^n [y_i - (mx_i + b)]^2.$$

Why squared? Because it is differentiable everywhere (unlike $|r_i|$), it penalises large errors more than small ones, and it has a closed-form solution.

Step 4: Differentiate with respect to b and set to zero.

$$\frac{\partial E}{\partial b} = \sum_{i=1}^n 2[y_i - mx_i - b](-1) = 0 \implies \sum_i y_i - m \sum_i x_i - nb = 0.$$

Step 5: Solve for b , introducing the means $\bar{x} = \frac{1}{n} \sum x_i$ and $\bar{y} = \frac{1}{n} \sum y_i$.

$$nb = \sum_i y_i - m \sum_i x_i \implies b = \bar{y} - m\bar{x}.$$

The fitted line always passes through the centroid $(\bar{x}, \bar{y})$ — a fact worth remembering, as it provides a free check on any numerical answer.

Step 6: Differentiate with respect to m and set to zero.

$$\frac{\partial E}{\partial m} = \sum_i 2[y_i - mx_i - b](-x_i) = 0 \implies \sum_i x_i y_i - m \sum_i x_i^2 - b \sum_i x_i = 0.$$

Step 7: Substitute $b = \bar{y} - m\bar{x}$ from Step 5.

$$\sum_i x_i y_i - m \sum_i x_i^2 - (\bar{y} - m\bar{x}) \sum_i x_i = 0.$$

Using $\sum_i x_i = n\bar{x}$:

$$\sum_i x_i y_i - m \sum_i x_i^2 - n\bar{x}\bar{y} + mn\bar{x}^2 = 0.$$

Step 8: Collect the m terms and solve.

$$m \left(\sum_i x_i^2 - n\bar{x}^2 \right) = \sum_i x_i y_i - n\bar{x}\bar{y}$$

IMPORTANT FORMULA: Least-squares line fit

$$m = \frac{\sum_i x_i y_i - n\bar{x}\bar{y}}{\sum_i x_i^2 - n\bar{x}^2} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad b = \bar{y} - m\bar{x}.$$

The second form for m is the more numerically stable one and reads as “covariance of x and y , divided by variance of x ”.

19.2.2 The matrix formulation — the version that generalises

Step 9: Stack the n equations $y_i = mx_i + b$.

$$\underbrace{\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} m \\ b \end{bmatrix}}_{\mathbf{p}} = \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}}_{\mathbf{y}}.$$

With $n > 2$ this system is **over-determined**: more equations than unknowns, so no exact solution exists unless the points happen to be exactly collinear.

Step 10: Minimise $\|\mathbf{Ap} - \mathbf{y}\|^2$.

Expand:

$$E = (\mathbf{Ap} - \mathbf{y})^\top (\mathbf{Ap} - \mathbf{y}) = \mathbf{p}^\top \mathbf{A}^\top \mathbf{Ap} - 2\mathbf{p}^\top \mathbf{A}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}.$$

Step 11: Differentiate with respect to $\mathbf{p}$ and set to zero.

$$\frac{\partial E}{\partial \mathbf{p}} = 2\mathbf{A}^\top \mathbf{Ap} - 2\mathbf{A}^\top \mathbf{y} = \mathbf{0}.$$

IMPORTANT FORMULA: Normal equations and the pseudo-inverse

$$\mathbf{A}^\top \mathbf{Ap} = \mathbf{A}^\top \mathbf{y} \quad \implies \quad \mathbf{p} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{y} = \mathbf{A}^+ \mathbf{y}$$

where $\mathbf{A}^+$ is the Moore–Penrose pseudo-inverse. This single formula solves line fitting, parabola fitting, affine fitting and the calibration problem — only the contents of $\mathbf{A}$ change.

19.2.3 Parabola and polynomial fitting

Step 12: The model is $y = ax^2 + bx + c$.

It is quadratic in x but **linear in the unknowns** a, b, c — which is all that least squares requires.

$$\mathbf{A} = \begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n^2 & x_n & 1 \end{bmatrix}, \quad \mathbf{p} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix},$$

and the normal equations are solved exactly as before. A degree- k polynomial needs $k + 1$ columns and at least $k + 1$ points.

EXAM PITFALL: Vertical least squares cannot fit a vertical line

The model $y = mx + b$ requires a finite slope. For a perfectly vertical set of points, $m \rightarrow \infty$ and the method breaks down; for a nearly vertical set it is *numerically unstable*, producing wildly varying answers from small perturbations. This is not a coding bug; it is a defect of the *model*, and it is the motivation for total least squares in the next topic. A second, related flaw: the vertical-offset error measure is not rotation-invariant — rotate your data and the “best” line changes.

19.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Fitting a line to five edge points

Problem. An edge detector has returned the points

$$(1,2), (2,3), (3,5), (4,6), (5,8).$$

Fit the line $y = mx + b$ by ordinary least squares, verify via the matrix route, and report the residuals.

Step 1: Tabulate the sums.

i	x_i	y_i	$x_i y_i$	x_i^2
1	1	2	2	1
2	2	3	6	4
3	3	5	15	9
4	4	6	24	16
5	5	8	40	25
Σ	15	24	87	55

Step 2: Compute the means.

$$\bar{x} = \frac{15}{5} = 3, \quad \bar{y} = \frac{24}{5} = 4.8.$$

Step 3: Compute the numerator of m .

$$\sum_i x_i y_i - n \bar{x} \bar{y} = 87 - (5)(3)(4.8) = 87 - 72 = 15.$$

Step 4: Compute the denominator.

$$\sum_i x_i^2 - n \bar{x}^2 = 55 - (5)(9) = 55 - 45 = 10.$$

Step 5: Solve for the slope.

$$m = \frac{15}{10} = 1.5.$$

Step 6: Solve for the intercept.

$$b = \bar{y} - m \bar{x} = 4.8 - (1.5)(3) = 4.8 - 4.5 = 0.3.$$

$$\boxed{y = 1.5x + 0.3}$$

Step 7: Verify via the normal equations.

$$\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & n \end{bmatrix} = \begin{bmatrix} 55 & 15 \\ 15 & 5 \end{bmatrix}, \quad \mathbf{A}^\top \mathbf{y} = \begin{bmatrix} \sum x_i y_i \\ \sum y_i \end{bmatrix} = \begin{bmatrix} 87 \\ 24 \end{bmatrix}.$$

Invert the 2×2 matrix. Its determinant is

$$\det = (55)(5) - (15)(15) = 275 - 225 = 50,$$

so

$$(\mathbf{A}^\top \mathbf{A})^{-1} = \frac{1}{50} \begin{bmatrix} 5 & -15 \\ -15 & 55 \end{bmatrix}.$$

Then

$$\mathbf{p} = \frac{1}{50} \begin{bmatrix} 5 & -15 \\ -15 & 55 \end{bmatrix} \begin{bmatrix} 87 \\ 24 \end{bmatrix} = \frac{1}{50} \begin{bmatrix} (5)(87) + (-15)(24) \\ (-15)(87) + (55)(24) \end{bmatrix} = \frac{1}{50} \begin{bmatrix} 435 - 360 \\ -1305 + 1320 \end{bmatrix} = \frac{1}{50} \begin{bmatrix} 75 \\ 15 \end{bmatrix}.$$

$$m = \frac{75}{50} = 1.5, \quad b = \frac{15}{50} = 0.3. \checkmark$$

Step 8: Check the centroid property.

$$y(\bar{x}) = 1.5(3) + 0.3 = 4.8 = \bar{y}. \checkmark$$

Step 9: Compute the residuals.

x_i	y_i	predicted $1.5x_i + 0.3$	r_i	r_i^2
1	2	1.8	+0.2	0.04
2	3	3.3	-0.3	0.09
3	5	4.8	+0.2	0.04
4	6	6.3	-0.3	0.09
5	8	7.8	+0.2	0.04
			$\sum r_i = 0.0$	$E = 0.30$

Note $\sum r_i = 0$ exactly — a consequence of the $\partial E / \partial b = 0$ condition, and another free check on your arithmetic. The total squared error is $E = 0.30$.

Answer.

$y = 1.5x + 0.3$, with $E = 0.30$ and zero residual sum.

Total Least Squares

20.1 Intuitive Theory and Definition

Ordinary least squares assumes x is known exactly and all the error lives in y . For a calibration curve that may be true. For *edge points extracted from an image* it is plainly false: an edge pixel's position is uncertain in both coordinates equally. Total least squares (TLS) fixes both the error measure and the vertical-line failure in one move.

PHYSICAL ANALOGY: Which gap is the honest one?

Your data are dots on a page and you are holding a ruler. Ordinary least squares measures how far each dot is from the ruler *straight up or down*. But if the ruler is nearly vertical, “straight up” is almost *along* the ruler, so those distances explode even for dots sitting right on it. Total least squares instead measures the *shortest* distance — perpendicular to the ruler. That number does not care which way the ruler is pointing, which is exactly the property you want when both coordinates are equally uncertain.

KEY DEFINITION: Total least squares

Fit a line by minimising the sum of squared *perpendicular* distances from the data to the line. Because perpendicular distance is invariant to rotation of the coordinate frame, TLS handles lines of any orientation, vertical included.

20.2 Mathematical Formulation and Derivation

20.2.1 A line parameterisation that admits vertical lines

Step 1: Abandon $y = mx + b$.

Use the implicit form

$$ax + by = d, \quad \text{with the normalisation } a^2 + b^2 = 1.$$

Here $\mathbf{n} = (a, b)^\top$ is the **unit normal** to the line and d is the (signed) distance from the origin. A vertical line is simply $a = 1, b = 0$ — no infinity anywhere.

Step 2: Derive the point-to-line distance.

For a point (x_i, y_i) , the perpendicular distance to the line is

$$|ax_i + by_i - d|,$$

because with $a^2 + b^2 = 1$ the expression $ax + by$ is precisely the projection of (x, y) onto the unit normal direction. This is why the constraint $a^2 + b^2 = 1$ is not cosmetic: it is what makes the residual an actual distance.

Step 3: Write the objective.

$$E(a, b, d) = \sum_{i=1}^n (ax_i + by_i - d)^2 \quad \text{subject to} \quad a^2 + b^2 = 1.$$

This is a *constrained* optimisation problem — which is why Topic 18 came first.

20.2.2 Eliminating d

Step 4: Differentiate with respect to d and set to zero.

$$\frac{\partial E}{\partial d} = \sum_i 2(ax_i + by_i - d)(-1) = 0 \implies \sum_i (ax_i + by_i) = nd.$$

Step 5: Solve for d .

$$d = \frac{a \sum_i x_i + b \sum_i y_i}{n} = a\bar{x} + b\bar{y}.$$

Once again **the fitted line passes through the centroid.**

Step 6: Substitute d back into E .

$$E(a, b) = \sum_i [ax_i + by_i - a\bar{x} - b\bar{y}]^2 = \sum_i [a(x_i - \bar{x}) + b(y_i - \bar{y})]^2.$$

Centring the data has removed one unknown entirely.

20.2.3 The eigenvalue solution

Step 7: Write E in matrix form.

Define the centred design matrix

$$\mathbf{X} = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ x_2 - \bar{x} & y_2 - \bar{y} \\ \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} \end{bmatrix}, \quad \mathbf{n} = \begin{bmatrix} a \\ b \end{bmatrix}.$$

Then

$$E = \|\mathbf{X}\mathbf{n}\|^2 = \mathbf{n}^\top \mathbf{X}^\top \mathbf{X} \mathbf{n} \quad \text{subject to} \quad \|\mathbf{n}\|^2 = 1.$$

Step 8: Recognise the problem.

This is *exactly* the master result of Topic 18. Forming the Lagrangian

$$\mathcal{L} = \mathbf{n}^\top \mathbf{X}^\top \mathbf{X} \mathbf{n} - \lambda(\mathbf{n}^\top \mathbf{n} - 1)$$

and setting $\partial \mathcal{L} / \partial \mathbf{n} = \mathbf{0}$ gives

$$\mathbf{X}^\top \mathbf{X} \mathbf{n} = \lambda \mathbf{n},$$

with minimum value $E = \lambda$.

IMPORTANT FORMULA: Total least squares solution

$$\mathbf{n} = \begin{bmatrix} a \\ b \end{bmatrix} = \text{eigenvector of } \mathbf{X}^\top \mathbf{X} = \begin{bmatrix} \sum (x_i - \bar{x})^2 & \sum (x_i - \bar{x})(y_i - \bar{y}) \\ \sum (x_i - \bar{x})(y_i - \bar{y}) & \sum (y_i - \bar{y})^2 \end{bmatrix}$$

associated with its **smallest eigenvalue**, and then $d = a\bar{x} + b\bar{y}$. The matrix $\mathbf{X}^\top \mathbf{X}$ is the **scatter matrix** (the unnormalised covariance) of the data. Note the structural identity with the Harris second-moment matrix of Topic 9 — both are 2×2 scatter matrices whose eigenvectors describe the principal directions of a point set.

KEY DEFINITION: Why the *smallest* eigenvalue

The eigenvector of the *largest* eigenvalue is the direction of greatest spread — it lies *along* the line. The eigenvector of the *smallest* eigenvalue is the direction of least spread — it is *perpendicular* to the line, which is exactly the normal $\mathbf{n}$ we are solving for. Choosing the wrong eigenvector gives a line rotated by 90° : the single most common error in this topic.

20.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: TLS on the same five points, and a vertical-line stress test

Problem. (a) Fit the points $(1, 2), (2, 3), (3, 5), (4, 6), (5, 8)$ by total least squares. (b) Compare with the ordinary least-squares answer. (c) Show that TLS succeeds on the near-vertical set $(0.0, 1), (0.1, 2), (0.0, 3), (0.1, 4)$ where OLS fails.

Step 1: Centre the data.

From the previous topic, $\bar{x} = 3, \bar{y} = 4.8$.

x_i	y_i	$u_i = x_i - 3$	$v_i = y_i - 4.8$	u_i^2	$u_i v_i$	v_i^2
1	2	-2	-2.8	4	5.6	7.84
2	3	-1	-1.8	1	1.8	3.24
3	5	0	0.2	0	0.0	0.04
4	6	1	1.2	1	1.2	1.44
5	8	2	3.2	4	6.4	10.24
Σ				10	15.0	22.80

Step 2: Form the scatter matrix.

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 10 & 15 \\ 15 & 22.8 \end{bmatrix}.$$

Step 3: Solve the characteristic equation.

$$\det \begin{bmatrix} 10 - \lambda & 15 \\ 15 & 22.8 - \lambda \end{bmatrix} = 0$$

$$(10 - \lambda)(22.8 - \lambda) - 225 = 0 \quad (20.1)$$

$$228 - 10\lambda - 22.8\lambda + \lambda^2 - 225 = 0 \quad (20.2)$$

$$\lambda^2 - 32.8\lambda + 3 = 0 \quad (20.3)$$

Step 4: Apply the quadratic formula.

$$\lambda = \frac{32.8 \pm \sqrt{32.8^2 - 4(3)}}{2} = \frac{32.8 \pm \sqrt{1075.84 - 12}}{2} = \frac{32.8 \pm \sqrt{1063.84}}{2} = \frac{32.8 \pm 32.6166}{2}.$$

$$\lambda_{\max} = \frac{65.4166}{2} = 32.7083, \quad \lambda_{\min} = \frac{0.1834}{2} = 0.0917.$$

Step 5: Find the eigenvector for $\lambda_{\min} = 0.0917$.

$$(10 - 0.0917)a + 15b = 0 \implies 9.9083a = -15b \implies \frac{a}{b} = \frac{-15}{9.9083} = -1.5139.$$

Take $b = 1$, $a = -1.5139$, then normalise. The length is

$$\sqrt{(-1.5139)^2 + 1^2} = \sqrt{2.2919 + 1} = \sqrt{3.2919} = 1.8144,$$

so

$$a = \frac{-1.5139}{1.8144} = -0.8344, \quad b = \frac{1}{1.8144} = 0.5512.$$

Check: $a^2 + b^2 = 0.6962 + 0.3038 = 1.0000$. ✓

Step 6: Compute d .

$$d = a\bar{x} + b\bar{y} = (-0.8344)(3) + (0.5512)(4.8) = -2.5032 + 2.6458 = 0.1426.$$

$$\boxed{-0.8344x + 0.5512y = 0.1426}$$

Step 7 (b): Convert to slope–intercept form for comparison.

Solving for y :

$$y = \frac{0.8344}{0.5512}x + \frac{0.1426}{0.5512} = 1.5138x + 0.2587.$$

Against the OLS result $y = 1.5x + 0.3$. The two are close — as they must be, since these points lie near a moderately sloped line — but *not identical*. The TLS residual is

$$E_{\text{TLS}} = \lambda_{\min} = 0.0917,$$

which is the sum of squared *perpendicular* distances, and is necessarily smaller than the OLS figure of 0.30 because perpendicular distance is never longer than vertical distance. (The two numbers measure different things, so this is not evidence that one method is “better” — only that the errors are being measured differently.)

Step 8 (c): The near-vertical stress test. First, what OLS does.

Points: $(0.0, 1), (0.1, 2), (0.0, 3), (0.1, 4)$. Then $\bar{x} = 0.05$, $\bar{y} = 2.5$,

$$\sum (x_i - \bar{x})^2 = 4(0.05)^2 = 0.01,$$

$$\begin{aligned} \sum (x_i - \bar{x})(y_i - \bar{y}) &= (-0.05)(-1.5) + (0.05)(-0.5) + (-0.05)(0.5) + (0.05)(1.5) \\ &= 0.075 - 0.025 - 0.025 + 0.075 = 0.10. \end{aligned}$$

$$m_{\text{OLS}} = \frac{0.10}{0.01} = 10, \quad b = 2.5 - 10(0.05) = 2.0.$$

A slope of 10 from data that is visually vertical — and if one x value were perturbed by 0.01, the denominator would change by tens of percent and the slope would swing wildly. The answer is meaningless in any practical sense.

Step 9: Now TLS on the same points.

$$\sum (y_i - \bar{y})^2 = (-1.5)^2 + (-0.5)^2 + (0.5)^2 + (1.5)^2 = 2.25 + 0.25 + 0.25 + 2.25 = 5.0.$$

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 0.01 & 0.10 \\ 0.10 & 5.00 \end{bmatrix}.$$

Characteristic equation:

$$(0.01 - \lambda)(5.00 - \lambda) - 0.01 = 0 \implies \lambda^2 - 5.01\lambda + (0.05 - 0.01) = 0 \implies \lambda^2 - 5.01\lambda + 0.04 = 0.$$

$$\lambda = \frac{5.01 \pm \sqrt{25.1001 - 0.16}}{2} = \frac{5.01 \pm \sqrt{24.9401}}{2} = \frac{5.01 \pm 4.9940}{2}.$$

$$\lambda_{\min} = \frac{0.0160}{2} = 0.0080, \quad \lambda_{\max} = 5.0020.$$

Eigenvector for $\lambda_{\min}$:

$$(0.01 - 0.008)a + 0.10b = 0 \implies 0.002a = -0.10b \implies \frac{b}{a} = -0.02.$$

Take $a = 1, b = -0.02$; the length is $\sqrt{1 + 0.0004} = 1.0002$, so

$$a = 0.9998, \quad b = -0.0200.$$

$$d = (0.9998)(0.05) + (-0.02)(2.5) = 0.04999 - 0.05 = -0.00001 \approx 0.$$

The fitted line is $0.9998x - 0.02y \approx 0$, i.e. essentially $x = 0.05$ — the vertical line through the middle of the data, exactly as the eye reports. TLS handled it without difficulty, with a residual of only $\lambda_{\min} = 0.008$.

Answer.

(a) $-0.8344x + 0.5512y = 0.1426$, i.e. $y \approx 1.514x + 0.259$, with perpendicular residual 0.0917. (b) Close to but distinct from the OLS fit. (c) On near-vertical data OLS returns an unstable slope of 10; TLS returns the correct vertical line $x \approx 0.05$.

20.3.1 Fitting curves to edge points: the general over-determined system

The same machinery fits any model that is linear in its parameters. The lecturer's *Fitting Lines and Curves* deck states the general recipe:

IMPORTANT FORMULA: Over-determined homogeneous system

Given m unknowns $\{a_j\}$ and $n > m$ edge points, each point contributes one equation. Stacking gives $\mathbf{A}\mathbf{a} = \mathbf{0}$ with $\mathbf{A}$ of size $n \times m$. Since exact equality is impossible with noisy data, solve

$$\min_{\|\mathbf{a}\|=1} \|\mathbf{A}\mathbf{a}\|^2 \implies \mathbf{a} = \text{eigenvector of } \mathbf{A}^\top \mathbf{A} \text{ for the smallest eigenvalue.}$$

Inhomogeneous systems $\mathbf{A}\mathbf{p} = \mathbf{b}$ (where $\mathbf{b} \neq \mathbf{0}$) instead use the pseudo-inverse $\mathbf{p} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$. Knowing which of the two you are facing is the first decision in every fitting problem in this course.

Problem	System type	Solution method
Line fit $y = mx + b$	Inhomogeneous $\mathbf{A}\mathbf{p} = \mathbf{y}$	Pseudo-inverse
Parabola fit	Inhomogeneous	Pseudo-inverse
Total least squares line	Homogeneous, $\ \mathbf{n}\ = 1$	Smallest eigenvector
Affine transformation (Topic 23)	Inhomogeneous	Pseudo-inverse
Homography (Topic 23)	Homogeneous, $\ \mathbf{h}\ = 1$	Smallest eigenvector / SVD
Camera matrix (Topic 24)	Homogeneous, $\ \mathbf{p}\ = 1$	Smallest eigenvector / SVD

Robust Fitting and RANSAC

21.1 Intuitive Theory and Definition

Least squares has a fatal weakness: it squares the residuals, so a point that is wildly wrong contributes an enormous penalty and drags the fit toward itself. One bad correspondence can ruin an otherwise perfect estimate.

KEY DEFINITION: Inlier and outlier

An **inlier** is a data point generated by the model plus small noise. An **outlier** is a point generated by something else entirely — a mismatched feature, an edge from a different object, a sensor glitch. Outliers do not have larger noise; they have *no relationship* to the model at all.

PHYSICAL ANALOGY: The polling analogy

You are estimating the average height of a crowd, and one of your “measurements” is accidentally the height of the building behind them. Averaging all measurements gives an absurd answer, and no amount of extra crowd measurements fixes it, because the building contributes 40 metres to every sum. The answer is not a better average — it is to stop treating that measurement as data. RANSAC does exactly this: it guesses a model from a tiny random sample, asks how many measurements *agree*, and keeps the guess with the largest agreement.

KEY DEFINITION: RANSAC — RANdom SAmple Consensus

An iterative procedure that fits a model to data containing outliers, by repeatedly (i) drawing the *minimum* number of points needed to instantiate the model, (ii) instantiating it, (iii) counting how many of the remaining points lie within a distance threshold (the **consensus set** or inlier set), and (iv) keeping the hypothesis with the largest consensus set.

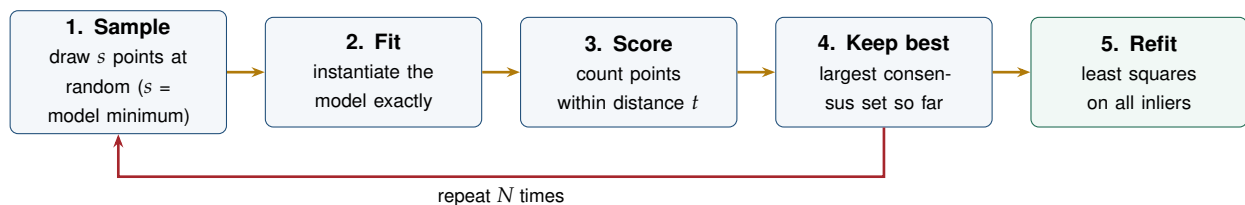


Figure 21.1: The RANSAC loop. Step 5 is often forgotten but matters: the final model should be refitted to *all* inliers, not left as the two-point hypothesis.

Source deck: RANSAC, slides 4–10; lec09_fitting, slides 23–37

21.2 Mathematical Formulation

21.2.1 The three parameters

IMPORTANT FORMULA: RANSAC parameters

s — **sample size**. The *minimum* number of points that determines the model. Line: $s = 2$. Circle: $s = 3$. Affine: $s = 3$. Homography: $s = 4$. Keep s minimal: a larger sample is more likely to contain an outlier and gains nothing.

t — **distance threshold**. A point is an inlier if its distance to the model is below t . Chosen so that an inlier is accepted with probability (say) 0.95; for zero-mean Gaussian noise of standard deviation σ in each coordinate, $t \approx 2\sigma$ to 3σ .

N — **number of iterations**. Derived below.

21.2.2 Deriving the number of iterations

Step 1: Let w be the inlier fraction.

$w = (\text{number of inliers}) / (\text{total points})$.

Step 2: Probability that one random sample of s points is all inliers.

Sampling with replacement (a good approximation for large data sets), the draws are independent:

$$P(\text{good sample}) = w^s.$$

Step 3: Probability that a sample is contaminated.

$$P(\text{bad sample}) = 1 - w^s.$$

Step 4: Probability that all N independent samples are contaminated.

$$P(\text{all } N \text{ bad}) = (1 - w^s)^N.$$

Step 5: Demand that this failure probability be at most $1 - p$,

where p is the desired success probability (typically 0.99):

$$(1 - w^s)^N \leq 1 - p.$$

Step 6: Take logarithms of both sides.

Since $0 < 1 - w^s < 1$, the logarithm is negative and the inequality reverses on division:

$$N \ln(1 - w^s) \leq \ln(1 - p) \implies N \geq \frac{\ln(1 - p)}{\ln(1 - w^s)}.$$

IMPORTANT FORMULA: Required RANSAC iterations

$$N = \left\lceil \frac{\log(1 - p)}{\log(1 - w^s)} \right\rceil$$

Three properties to quote in an exam:

- N is *independent of the number of data points*. Ten thousand points need no more iterations than one hundred, for the same inlier fraction.
- N grows *exponentially* in the sample size s . This is why the sample must be minimal, and why homography RANSAC ($s = 4$) needs far more iterations than line RANSAC ($s = 2$).

- w is usually unknown in advance. In practice one starts with a pessimistic estimate and *updates N adaptively* as better consensus sets are found.

Model (s)	Iterations N for $p = 0.99$ at inlier fraction w				
	$w = 0.9$	$w = 0.7$	$w = 0.5$	$w = 0.3$	$w = 0.2$
Line ($s = 2$)	3	7	17	49	113
Affine ($s = 3$)	4	11	35	169	574
Homography ($s = 4$)	5	17	72	567	2875

Table 21.1: The cost of a larger minimal sample. At $w = 0.2$, fitting a homography requires 25 times more iterations than fitting a line.

21.2.3 Robust loss functions — the alternative

An alternative to resampling: change the penalty.

Squared error grows without bound; a **robust** loss saturates, so an outlier can only do limited damage.

IMPORTANT FORMULA: Huber loss

$$\rho(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta \\ \delta \left(|r| - \frac{1}{2}\delta \right), & |r| > \delta \end{cases}$$

Quadratic for small residuals (so it behaves like least squares on the inliers) and *linear* for large ones (so an outlier's influence grows only linearly, not quadratically). The scale parameter δ plays the same role as RANSAC's t , and choosing it badly is the method's weakness: too large and outliers still dominate; too small and genuine inliers are down-weighted, flattening the fit.

EXAM PITFALL: Choosing the scale is the hard part

The lec09_fitting slides devote three slides to exactly this — “just right”, “too large”, “the error value is almost the same for every hypothesis”. With too large a scale, robustness is lost. With too small a scale, the objective becomes almost flat, every candidate model scores about the same, and the optimisation has nothing to descend. RANSAC's threshold t suffers the identical sensitivity.

21.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: RANSAC line fitting with iteration count

Problem. Seven points are given:

$$P_1(1,1), P_2(2,2), P_3(3,3), P_4(4,4), P_5(5,5), P_6(2,9), P_7(5,1).$$

Use RANSAC with threshold $t = 0.5$. (a) Evaluate the hypothesis from the sample $\{P_1, P_5\}$. (b) Evaluate the hypothesis from $\{P_6, P_7\}$. (c) Choose the winner and refit. (d)

Compute how many iterations are needed for $p = 0.99$. (e) Compare with plain least squares on all seven points.

Step 1 (a): Fit a line through $P_1(1,1)$ and $P_5(5,5)$.

$$m = \frac{5-1}{5-1} = 1, \quad b = 1 - (1)(1) = 0 \implies y = x.$$

In normalised implicit form, $x - y = 0$ divided by $\sqrt{1^2 + (-1)^2} = \sqrt{2}$:

$$\frac{x-y}{\sqrt{2}} = 0, \quad \text{distance} = \frac{|x_i - y_i|}{\sqrt{2}}.$$

Step 2: Score every point against this hypothesis.

Point	$ x_i - y_i $	distance	inlier? ($t = 0.5$)
$P_1(1,1)$	0	0.000	yes
$P_2(2,2)$	0	0.000	yes
$P_3(3,3)$	0	0.000	yes
$P_4(4,4)$	0	0.000	yes
$P_5(5,5)$	0	0.000	yes
$P_6(2,9)$	7	4.950	no
$P_7(5,1)$	4	2.828	no

$$|\text{consensus set}| = 5.$$

Step 3 (b): Fit a line through $P_6(2,9)$ and $P_7(5,1)$.

$$m = \frac{1-9}{5-2} = \frac{-8}{3} = -2.6\bar{6}, \quad b = 9 - (-2.6\bar{6})(2) = 9 + 5.3\bar{3} = 14.3\bar{3}.$$

So $y = -2.667x + 14.333$, or in implicit form $2.667x + y - 14.333 = 0$ with normaliser $\sqrt{2.667^2 + 1} = \sqrt{7.111 + 1} = \sqrt{8.111} = 2.848$.

Step 4: Score every point against this second hypothesis.

$$\text{distance} = \frac{|2.667x_i + y_i - 14.333|}{2.848}$$

Point	$2.667x_i + y_i - 14.333$	distance	inlier?
$P_1(1,1)$	$2.667 + 1 - 14.333 = -10.666$	3.745	no
$P_2(2,2)$	$5.334 + 2 - 14.333 = -6.999$	2.458	no
$P_3(3,3)$	$8.001 + 3 - 14.333 = -3.332$	1.170	no
$P_4(4,4)$	$10.668 + 4 - 14.333 = 0.335$	0.118	yes
$P_5(5,5)$	$13.335 + 5 - 14.333 = 4.002$	1.405	no
$P_6(2,9)$	$5.334 + 9 - 14.333 = 0.001$	0.000	yes
$P_7(5,1)$	$13.335 + 1 - 14.333 = 0.002$	0.001	yes

$$|\text{consensus set}| = 3.$$

Step 5 (c): Select the winner.

$5 > 3$, so the hypothesis $y = x$ wins, with inlier set $\{P_1, \dots, P_5\}$ and P_6, P_7 correctly rejected as outliers.

Step 6: Refit by least squares on the inliers only.

For $\{(1, 1), (2, 2), (3, 3), (4, 4), (5, 5)\}$: $\bar{x} = 3, \bar{y} = 3$,

$$\sum (x_i - \bar{x})(y_i - \bar{y}) = 4 + 1 + 0 + 1 + 4 = 10, \quad \sum (x_i - \bar{x})^2 = 10,$$

$$m = \frac{10}{10} = 1, \quad b = 3 - (1)(3) = 0.$$

The refit confirms $y = x$ exactly. (In general the refit shifts the answer slightly; here the inliers are perfectly collinear.)

Step 7 (d): Required iterations.

The inlier fraction is $w = 5/7 = 0.714$, the sample size is $s = 2$, and $p = 0.99$:

$$w^s = (0.714)^2 = 0.5102.$$

$$1 - w^s = 0.4898.$$

$$N = \frac{\log(1 - 0.99)}{\log(0.4898)} = \frac{\log(0.01)}{\log(0.4898)} = \frac{-2.0000}{-0.3100} = 6.45 \rightarrow \lceil 6.45 \rceil = 7.$$

Only **seven** random samples are needed for a 99% chance of hitting an all-inlier pair.

Step 8: Contrast with a harder model.

Had this been a homography ($s = 4$) at the same w :

$$w^4 = 0.714^4 = 0.2603, \quad N = \frac{-2}{\log(0.7397)} = \frac{-2}{-0.1309} = 15.3 \rightarrow 16.$$

More than double, from the same data quality — the exponential-in- s behaviour in action.

Step 9 (e): What plain least squares would have given.

Using all seven points: $\bar{x} = (1 + 2 + 3 + 4 + 5 + 2 + 5)/7 = 22/7 = 3.143$, $\bar{y} = (1 + 2 + 3 + 4 + 5 + 9 + 1)/7 = 25/7 = 3.571$.

$$\sum x_i y_i = 1 + 4 + 9 + 16 + 25 + 18 + 5 = 78, \quad \sum x_i^2 = 1 + 4 + 9 + 16 + 25 + 4 + 25 = 84.$$

$$m = \frac{78 - 7(3.143)(3.571)}{84 - 7(3.143)^2} = \frac{78 - 78.57}{84 - 69.14} = \frac{-0.57}{14.86} = -0.038.$$

$$b = 3.571 - (-0.038)(3.143) = 3.571 + 0.120 = 3.691.$$

Least squares returns $y \approx -0.04x + 3.69$: an essentially *horizontal* line through the middle of everything. Two outliers out of seven have destroyed the estimate completely, turning a perfect slope of 1 into a slope of -0.04 .

Answer.

RANSAC recovers $y = x$ with five inliers, needing only 7 iterations at $p = 0.99$; plain least squares on the same data returns the useless fit $y \approx -0.04x + 3.69$.

EXAM PITFALL: RANSAC pros and cons — know both sides

Pros: simple and general, applies to any model with a minimal-sample fit, tolerates a very large outlier fraction, and the iteration count does not grow with the data size. **Cons:**

requires a threshold t you must choose; N grows exponentially with s , so it becomes impractical for high-parameter models; it is randomised, so two runs can give different answers; and when the inlier fraction is very low it can fail silently, returning a confident but wrong model supported by a small chance consensus.

The Hough Transform

22.1 Intuitive Theory and Definition

RANSAC guesses models and tests them. The Hough transform inverts the logic entirely: it lets each data point *vote* for every model that could have produced it, then looks for the model that collected the most votes.

PHYSICAL ANALOGY: Every witness names all the suspects they saw

A crime was committed by an unknown line. Each edge point is a witness, and each witness cannot name the culprit — but can say “the line that did it was one of these”. Since infinitely many lines pass through any one point, each witness names a whole family. You collect all the testimony in a tally book, one entry per candidate line, and at the end you look for the candidate that was named by the most witnesses. That tally book is the **accumulator array**, and the peak in it is the detected line.

KEY DEFINITION: Parameter space and the accumulator

Every point in *image space* corresponds to a curve in *parameter space*, and every point in parameter space corresponds to a line in image space. The accumulator is a discretised grid over parameter space; each image point increments every cell on its corresponding curve. Peaks in the accumulator are the detected models.

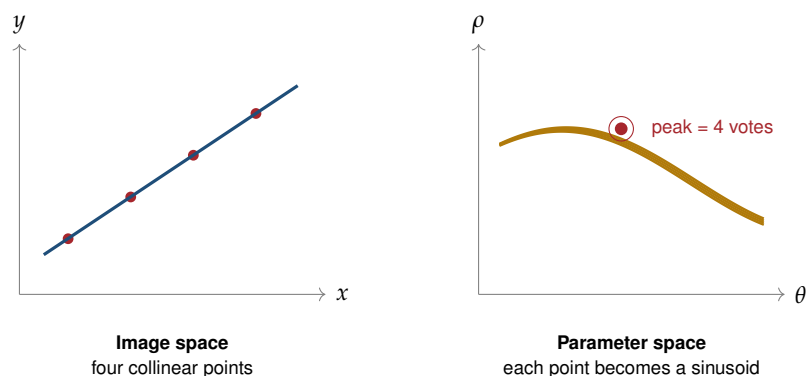


Figure 22.1: Four collinear image points produce four sinusoids in (ρ, θ) space, and all four cross at the single cell describing the line they lie on.

Source deck: *lec09_fitting*, slides 39–54

22.2 Mathematical Formulation

22.2.1 Why not (m, b) space

Step 1: The naive parameterisation.

A point (x_i, y_i) satisfies $y_i = mx_i + b$, which rearranges to

$$b = -x_i m + y_i,$$

a *straight line* in (m, b) space. Simple — but fatally flawed.

Step 2: The flaw.

A vertical image line has $m = \infty$. The parameter space is therefore unbounded, and no finite accumulator array can represent it.

22.2.2 The polar (normal) parameterisation

Step 3: Use the implicit form of Topic 20.

Write the line as

$$x \cos \theta + y \sin \theta = \rho,$$

where $\rho \geq 0$ is the perpendicular distance from the origin and $\theta \in [0, \pi)$ is the angle of the normal.

Step 4: Check the ranges are bounded.

For an image of diagonal D ,

$$0 \leq \rho \leq D, \quad 0 \leq \theta < \pi.$$

Both finite — so a finite accumulator suffices. This is exactly the same reparameterisation that rescued total least squares, for exactly the same reason.

Step 5: Invert the roles.

Holding (x_i, y_i) fixed and varying θ ,

$$\rho(\theta) = x_i \cos \theta + y_i \sin \theta$$

traces a **sinusoid** in (θ, ρ) space. Using the identity $A \cos \theta + B \sin \theta = R \cos(\theta - \phi)$ with $R = \sqrt{A^2 + B^2}$:

$$\rho(\theta) = \sqrt{x_i^2 + y_i^2} \cos(\theta - \arctan(y_i/x_i)),$$

a sinusoid of amplitude equal to the point's distance from the origin.

IMPORTANT FORMULA: The Hough transform algorithm

1. Initialise an accumulator $H[\rho, \theta]$ to zero over the discretised ranges.
2. For each edge point (x_i, y_i) : for each quantised θ , compute $\rho = x_i \cos \theta + y_i \sin \theta$, quantise it, and increment $H[\rho, \theta]$.
3. Find local maxima of H above a vote threshold.
4. Each such maximum (ρ^*, θ^*) is a detected line.

Cost: $O(n \cdot n_\theta)$ for n edge points and n_θ angle bins.

KEY DEFINITION: Choosing the bin size — the central trade-off

Bins too coarse: genuinely different lines fall into one cell, so distinct lines are merged and localisation is poor. **Bins too fine:** votes from the same line scatter across neighbouring cells because of noise, so no cell accumulates a clear peak and the line is missed. The standard remedy is to smooth the accumulator before peak-finding, or to let each point vote for a small neighbourhood of cells.

22.2.3 The generalised Hough transform

Extending to arbitrary shapes.

For a shape with no analytic equation, replace the parametric vote with a **lookup table** (an R-table): during training, record for each boundary point its displacement vector to a chosen reference point, indexed by the local edge orientation. At test time each edge point looks up its orientation and votes for the implied reference-point locations. The peak in $(x_{\text{ref}}, y_{\text{ref}})$ space localises the object.

This is the direct ancestor of the **implicit shape model**, in which the “boundary points” become visual-codebook entries learned from training images, and every matched codebook entry votes for the object centre — the method the fitting lecture builds toward.

	Hough transform	RANSAC
Strategy	Every point votes for all compatible models	Sample models, count agreement
Multiple instances	Natural — one peak per instance	Awkward — must remove inliers and repeat
Outlier tolerance	Excellent (outliers vote incoherently)	Excellent (outliers never reach consensus)
Parameter count	Poor — accumulator size grows exponentially	Good — only N grows
Determinism	Deterministic	Randomised
Key tuning knob	Bin size	Threshold t

Table 22.1: Hough versus RANSAC. Use Hough for low-dimensional models with possibly several instances (lines, circles); use RANSAC for higher-dimensional models with one dominant instance (homography, fundamental matrix).

EXAM PITFALL: Noise, and the “more complicated image” problem

On a clean synthetic image the accumulator has crisp isolated peaks. On a real photograph the edge map contains texture and clutter, and the accumulator becomes a broad noisy surface in which spurious peaks compete with real ones. The `lec09_fitting` slides show exactly this progression — square, circle, several lines, then “a more complicated image” where the peaks blur together. The fix is a combination of coarser bins, accumulator smoothing, and restricting each point’s vote using its known gradient *orientation* so it votes for one θ instead of all of them.

22.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: Hough voting for three collinear points

Problem. Three edge points are detected at $(0,0)$, $(1,1)$ and $(2,2)$. Using bins of $\theta \in \{0^\circ, 45^\circ, 90^\circ, 135^\circ\}$, (a) compute the ρ vote of each point at each θ ; (b) build the accumulator; (c) identify the peak; (d) convert the peak back to a line and verify.

Step 1: Write the voting equation.

$$\rho = x \cos \theta + y \sin \theta.$$

The trigonometric values needed:

$$\begin{aligned} \cos 0^\circ &= 1, \sin 0^\circ = 0; & \cos 45^\circ &= \sin 45^\circ = 0.7071; \\ \cos 90^\circ &= 0, \sin 90^\circ = 1; & \cos 135^\circ &= -0.7071, \sin 135^\circ = 0.7071. \end{aligned}$$

Step 2 (a): Votes from (0, 0).

$$\theta = 0^\circ : \rho = 0(1) + 0(0) = 0; \quad \theta = 45^\circ : \rho = 0(0.7071) + 0(0.7071) = 0;$$

$$\theta = 90^\circ : \rho = 0; \quad \theta = 135^\circ : \rho = 0.$$

The origin votes $\rho = 0$ for *every* angle — correctly, since every line through the origin has zero perpendicular distance from it.

Step 3: Votes from (1, 1).

$$\theta = 0^\circ : \rho = 1(1) + 1(0) = 1.000 \quad (22.1)$$

$$\theta = 45^\circ : \rho = 1(0.7071) + 1(0.7071) = 1.414 \quad (22.2)$$

$$\theta = 90^\circ : \rho = 1(0) + 1(1) = 1.000 \quad (22.3)$$

$$\theta = 135^\circ : \rho = 1(-0.7071) + 1(0.7071) = 0.000 \quad (22.4)$$

Step 4: Votes from (2, 2).

$$\theta = 0^\circ : \rho = 2(1) + 2(0) = 2.000 \quad (22.5)$$

$$\theta = 45^\circ : \rho = 2(0.7071) + 2(0.7071) = 2.828 \quad (22.6)$$

$$\theta = 90^\circ : \rho = 2(0) + 2(1) = 2.000 \quad (22.7)$$

$$\theta = 135^\circ : \rho = 2(-0.7071) + 2(0.7071) = 0.000 \quad (22.8)$$

Step 5 (b): Tabulate every vote.

Point	$\theta = 0^\circ$	$\theta = 45^\circ$	$\theta = 90^\circ$	$\theta = 135^\circ$
(0,0)	$\rho = 0$	$\rho = 0$	$\rho = 0$	$\rho = 0$
(1,1)	$\rho = 1.000$	$\rho = 1.414$	$\rho = 1.000$	$\rho = 0$
(2,2)	$\rho = 2.000$	$\rho = 2.828$	$\rho = 2.000$	$\rho = 0$

Step 6: Build the accumulator by counting identical (ρ, θ) cells.

ρ	$\theta = 0^\circ$	$\theta = 45^\circ$	$\theta = 90^\circ$	$\theta = 135^\circ$
0.000	1	1	1	3
1.000	1	0	1	0
1.414	0	1	0	0
2.000	1	0	1	0
2.828	0	1	0	0

Step 7 (c): Identify the peak.

The only cell with more than one vote is

$$(\rho^*, \theta^*) = (0, 135^\circ), \quad \text{votes} = 3.$$

All three points agreed on it, and on nothing else.

Step 8 (d): Convert the peak back to a line.

$$x \cos 135^\circ + y \sin 135^\circ = 0 \implies -0.7071 x + 0.7071 y = 0.$$

Dividing through by 0.7071:

$$-x + y = 0 \implies y = x.$$

Step 9: Verify against the data.

All three points $(0,0)$, $(1,1)$, $(2,2)$ satisfy $y = x$ exactly. ✓ Geometric check: the line $y = x$ passes through the origin, so its perpendicular distance from the origin is $\rho = 0$ ✓; and its normal direction is perpendicular to the 45° line direction, i.e. at 135° ✓.

Step 10: Note the near-miss, and what it teaches.

The cells $(\rho = 0, \theta = 0^\circ)$, $(0, 45^\circ)$ and $(0, 90^\circ)$ each collected exactly one vote, from the origin alone. If the bins were coarser — say, a single θ bin spanning 90° to 180° — those stray votes could merge with the true peak and inflate it, or, worse, two genuinely different lines could merge into one. This is the bin-size trade-off in miniature.

Answer.

The accumulator peaks at $(\rho, \theta) = (0, 135^\circ)$ with three votes, giving the line $y = x$, which all three points satisfy exactly.

PART VI

Image Alignment

Image Alignment

23.1 Intuitive Theory and Definition

Everything in Parts II and V now converges. Part II gave you the *family* of transformations. Part V gave you the *machinery* for fitting a model to noisy data with outliers. Alignment is the problem of using the second to determine the first.

KEY DEFINITION: Image alignment

Given two images of the same scene, find the transformation T that maps one onto the other. Formally: given a set of putative correspondences $\{(\mathbf{x}_i, \mathbf{x}'_i)\}_{i=1}^n$, find the parameters $\mathbf{p}$ of a chosen transformation family minimising

$$\sum_{i=1}^n \|T(\mathbf{x}_i; \mathbf{p}) - \mathbf{x}'_i\|^2.$$

Alignment is simply fitting, where the model is a transformation.

PHYSICAL ANALOGY: Overlaying two tracings

You have two tracing-paper copies of the same map, drawn by different people from different angles. To superimpose them you find a few landmarks that appear on both — a church, a bridge, a crossroads — and slide, rotate and stretch one sheet until those landmarks line up. Three decisions define the problem. *Which landmarks?* (feature detection and matching). *How much are you allowed to deform the sheet?* (the transformation family). *What if one “landmark” was matched wrongly?* (robustness, i.e. RANSAC).

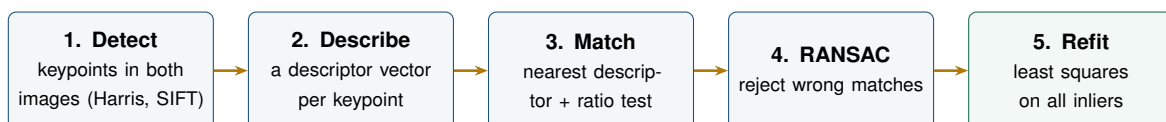


Figure 23.1: The robust feature-based alignment pipeline. Topic 9 supplied stage 1; Topic 21 supplied stage 4; this topic supplies stages 2, 3 and 5.

Source deck: *lec10_alignment*, slides 8–11, 29–47

23.2 Mathematical Formulation and Derivation

23.2.1 Fitting an affine transformation

Step 1: Write the model.

From Topic 13, the affine transformation has six unknowns:

$$x' = ax + by + c, \quad y' = dx + ey + f.$$

Step 2: Note that the equations are *linear in the unknowns*.

Although a multiplies x , the unknowns $a, \dots, f$ appear only to the first power. Least squares applies directly — no iteration needed.

Step 3: Write one correspondence as two equations.

For the pair $(x_i, y_i) \leftrightarrow (x'_i, y'_i)$:

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_i & y_i & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix} = \begin{bmatrix} x'_i \\ y'_i \end{bmatrix}.$$

Check by multiplying out: row 1 gives $ax_i + by_i + c = x'_i$ ✓ and row 2 gives $dx_i + ey_i + f = y'_i$ ✓. Note the block structure — the x' equation involves only a, b, c and the y' equation only d, e, f , so the system decouples into two independent three-unknown problems if you prefer.

Step 4: Stack n correspondences.

$$\underbrace{\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_2 & y_2 & 1 \\ \vdots & & & & & \vdots \\ x_n & y_n & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_n & y_n & 1 \end{bmatrix}}_{\mathbf{A} \ (2n \times 6)} \underbrace{\begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix}}_{\mathbf{p}} = \underbrace{\begin{bmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ \vdots \\ x'_n \\ y'_n \end{bmatrix}}_{\mathbf{b}}.$$

IMPORTANT FORMULA: Affine alignment

$$\mathbf{A}\mathbf{p} = \mathbf{b} \quad \implies \quad \mathbf{p} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$$

An **inhomogeneous** system: the right-hand side is non-zero, so the pseudo-inverse of Topic 19 applies directly — no eigenvector needed. **Minimum correspondences:** 6 unknowns $\div$ 2 equations per point = 3 points.

23.2.2 Fitting a homography — and why it is harder

Step 5: Write the model with its division.

From Topic 14,

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}.$$

These are *not* linear in the unknowns: the h 's appear in the denominator.

Step 6: Clear the denominators — the Direct Linear Transform trick.

Multiply both sides by the denominator:

$$x'(h_{31}x + h_{32}y + h_{33}) = h_{11}x + h_{12}y + h_{13} \quad (23.1)$$

$$y'(h_{31}x + h_{32}y + h_{33}) = h_{21}x + h_{22}y + h_{23} \quad (23.2)$$

Step 7: Move everything to one side.

$$h_{11}x + h_{12}y + h_{13} - h_{31}xx' - h_{32}yx' - h_{33}x' = 0 \quad (23.3)$$

$$h_{21}x + h_{22}y + h_{23} - h_{31}xy' - h_{32}yy' - h_{33}y' = 0 \quad (23.4)$$

These *are* linear in the nine unknowns — that is the whole point of the manoeuvre. The price is that we are now minimising an *algebraic* error rather than the geometric reprojection error, which is why practical implementations follow the DLT with a non-linear refinement.

Step 8: Write the two rows in matrix form.

With $\mathbf{h} = (h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}, h_{33})^\top$:

$$\begin{bmatrix} x & y & 1 & 0 & 0 & 0 & -xx' & -yx' & -x' \\ 0 & 0 & 0 & x & y & 1 & -xy' & -yy' & -y' \end{bmatrix} \mathbf{h} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Step 9: Stack and solve.

With n correspondences, $\mathbf{A}$ is $2n \times 9$ and

$$\mathbf{A}\mathbf{h} = \mathbf{0},$$

a **homogeneous** system. By Topic 18's master result, the solution subject to $\|\mathbf{h}\| = 1$ is the eigenvector of $\mathbf{A}^\top \mathbf{A}$ with the smallest eigenvalue — equivalently, the last column of $\mathbf{V}$ in the SVD $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$.

IMPORTANT FORMULA: Homography alignment (DLT)

$$\mathbf{A}\mathbf{h} = \mathbf{0}, \quad \|\mathbf{h}\| = 1 \quad \implies \quad \mathbf{h} = \text{last column of } \mathbf{V}, \quad \mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top.$$

Minimum correspondences: 8 degrees of freedom $\div$ 2 equations per point = 4 points. The four points must be in *general position* — no three collinear — or the system is rank-deficient and $\mathbf{H}$ is not determined.

EXAM PITFALL: Affine versus homography: two different solution methods

This is the distinction most often muddled under exam pressure. **Affine** $\rightarrow$ *inhomogeneous* $\mathbf{A}\mathbf{p} = \mathbf{b} \rightarrow$ *pseudo-inverse*, 3 points. **Homography** $\rightarrow$ *homogeneous* $\mathbf{A}\mathbf{h} = \mathbf{0} \rightarrow$ *smallest eigenvector / SVD*, 4 points. The reason for the difference is the scale ambiguity: a homography is defined only up to scale, so the right-hand side can be made zero, whereas an affine matrix has a fixed bottom row $\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$ that anchors the scale.

23.2.3 Getting the correspondences: descriptors and matching

KEY DEFINITION: Feature descriptor

A vector summarising the local image appearance around a keypoint, designed so that the *same* physical point yields a similar vector in both images despite changes in viewpoint and lighting. The standard construction (SIFT) computes gradient orientations in a grid of sub-regions around the keypoint, histograms them, concatenates the histograms into a 128-dimensional vector, and normalises it to unit length so that a global intensity scaling has no effect.

KEY DEFINITION: The ratio test

Matching by “nearest descriptor” alone produces many false matches, because repeated structure (windows on a building, leaves on a tree) makes many descriptors nearly identical. The remedy, due to Lowe: for each descriptor, find the nearest neighbour at distance d_1 and the *second*-nearest at d_2 , and accept the match only if

$$\frac{d_1}{d_2} < \tau, \quad \tau \approx 0.8.$$

A *distinctive* match has $d_1 \ll d_2$ and passes; an *ambiguous* match has $d_1 \approx d_2$ and is rejected. Crucially, the test uses the *ratio* rather than the absolute distance, so it adapts automatically to how distinctive the local region is.

[INSERT IMAGE HERE: Reference lec10_alignment, slides 37–53 — “Generating putative correspondences”, “Rejection of ambiguous matches”, “RANSAC example: Translation”, “Lowe paper results”]

A four-panel photographic sequence on two overlapping views of a building: (i) both images side by side with several hundred detected keypoints drawn as circles whose radius encodes scale and whose radial line encodes orientation; (ii) the same pair with every putative match drawn as a connecting line — a dense tangle including many obviously wrong long diagonal lines; (iii) the same after the ratio test, with the tangle substantially thinned; (iv) the RANSAC inliers only, in which every remaining line is parallel to its neighbours, revealing the consistent transformation. A companion histogram shows the distribution of d_1/d_2 for correct versus incorrect matches, with correct matches clustered below 0.6 and incorrect ones spread near 1.0, which is what justifies the threshold at 0.8.

23.3 Solved Numerical Examples

Both examples below are the lecturer’s own posted problems, worked in full.

SOLVED NUMERICAL EXAMPLE: Fitting an affine transformation from three correspondences

Problem. Three points in the first image,

$$(8, 8), \quad (1, 8), \quad (8, 1),$$

are known to map to

$$(1, 3), \quad (4, 5), \quad (2, 4)$$

respectively in the second image. Find the affine transformation $x' = ax + by + t_x$, $y' = cx + dy + t_y$.

Step 1: Count the unknowns and check feasibility.

There are six unknowns a, b, c, d, t_x, t_y . Each correspondence gives two equations, so three correspondences give exactly six — a square system with a unique solution, provided the three source points are not collinear. Check: the vectors $(1, 8) - (8, 8) = (-7, 0)$ and $(8, 1) - (8, 8) = (0, -7)$ are clearly independent, so the points form a genuine triangle. ✓

Step 2: Write the equations from the first correspondence $(8, 8) \rightarrow (1, 3)$.

$$a(8) + b(8) + t_x = 1 \tag{1}$$

$$c(8) + d(8) + t_y = 3 \tag{2}$$

Step 3: Second correspondence $(1, 8) \rightarrow (4, 5)$.

$$a(1) + b(8) + t_x = 4 \quad (3)$$

$$c(1) + d(8) + t_y = 5 \quad (4)$$

Step 4: Third correspondence $(8, 1) \rightarrow (2, 4)$.

$$a(8) + b(1) + t_x = 2 \quad (5)$$

$$c(8) + d(1) + t_y = 4 \quad (6)$$

Step 5: Solve the x -group. Subtract (3) from (1) to eliminate b and t_x .

$$8a + 8b + t_x - (a + 8b + t_x) = 1 - 4$$

$$7a = -3 \implies a = -\frac{3}{7} \approx -0.4286.$$

Step 6: Subtract (5) from (1) to eliminate a and t_x .

$$8a + 8b + t_x - (8a + b + t_x) = 1 - 2$$

$$7b = -1 \implies b = -\frac{1}{7} \approx -0.1429.$$

Step 7: Back-substitute into (1) for t_x .

$$8\left(-\frac{3}{7}\right) + 8\left(-\frac{1}{7}\right) + t_x = 1$$

$$-\frac{24}{7} - \frac{8}{7} + t_x = 1 \implies -\frac{32}{7} + t_x = 1$$

$$t_x = 1 + \frac{32}{7} = \frac{7 + 32}{7} = \frac{39}{7} \approx 5.5714.$$

Step 8: Solve the y -group. Subtract (4) from (2).

$$8c + 8d + t_y - (c + 8d + t_y) = 3 - 5$$

$$7c = -2 \implies c = -\frac{2}{7} \approx -0.2857.$$

Step 9: Subtract (6) from (2).

$$8c + 8d + t_y - (8c + d + t_y) = 3 - 4$$

$$7d = -1 \implies d = -\frac{1}{7} \approx -0.1429.$$

Step 10: Back-substitute into (2) for t_y .

$$8\left(-\frac{2}{7}\right) + 8\left(-\frac{1}{7}\right) + t_y = 3$$

$$-\frac{16}{7} - \frac{8}{7} + t_y = 3 \implies -\frac{24}{7} + t_y = 3$$

$$t_y = 3 + \frac{24}{7} = \frac{21 + 24}{7} = \frac{45}{7} \approx 6.4286.$$

Step 11: Assemble the transformation matrix.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \frac{1}{7} \begin{bmatrix} -3 & -1 & 39 \\ -2 & -1 & 45 \\ 0 & 0 & 7 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Step 12: Verify all three correspondences.

For (8,8):

$$x' = \frac{-3(8) - 1(8) + 39}{7} = \frac{-24 - 8 + 39}{7} = \frac{7}{7} = 1 \checkmark$$

$$y' = \frac{-2(8) - 1(8) + 45}{7} = \frac{-16 - 8 + 45}{7} = \frac{21}{7} = 3 \checkmark$$

For (1,8):

$$x' = \frac{-3 - 8 + 39}{7} = \frac{28}{7} = 4 \checkmark, \quad y' = \frac{-2 - 8 + 45}{7} = \frac{35}{7} = 5 \checkmark$$

For (8,1):

$$x' = \frac{-24 - 1 + 39}{7} = \frac{14}{7} = 2 \checkmark, \quad y' = \frac{-16 - 1 + 45}{7} = \frac{28}{7} = 4 \checkmark$$

All six equations are satisfied exactly, as they must be for a square non-singular system.

Step 13: Interpret the result.

The linear part is $\mathbf{A} = \frac{1}{7} \begin{bmatrix} -3 & -1 \\ -2 & -1 \end{bmatrix}$ with

$$\det \mathbf{A} = \frac{1}{49} [(-3)(-1) - (-1)(-2)] = \frac{3 - 2}{49} = \frac{1}{49}.$$

The positive determinant means orientation is preserved, and $|\det| = 1/49$ means areas shrink by a factor of 49 — a sevenfold reduction in each linear dimension, consistent with source points spread over a 7×7 region mapping into a region of side about 1.

Step 14: Note what would change with four or more correspondences.

With $n > 3$ points the system becomes over-determined ($2n > 6$ equations), no exact solution exists, and you would solve it by the pseudo-inverse $\mathbf{p} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$ of Topic 19. The verification of Step 12 would then show small non-zero residuals rather than exact agreement.

Answer.

$$a = -3/7, b = -1/7, t_x = 39/7, c = -2/7, d = -1/7, t_y = 45/7.$$

SOLVED NUMERICAL EXAMPLE: Fitting a homography: the point-count argument

Problem. (a) Explain why a homography needs *four* point correspondences, not three. (b) Set up the DLT system for the correspondences

$$(0,0) \rightarrow (0,0), \quad (1,0) \rightarrow (1,0), \quad (0,1) \rightarrow (0,1), \quad (1,1) \rightarrow (1,1)$$

and show it recovers the identity. (c) Repeat with the last correspondence changed to $(1, 1) \rightarrow (2, 2)$ and interpret.

Step 1 (a): Count the unknowns.

$$\mathbf{H} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \quad \text{has 9 entries.}$$

Step 2: Remove the scale ambiguity.

Since $\mathbf{H}$ and $\lambda\mathbf{H}$ produce identical mappings (Topic 14), one degree of freedom is redundant:

$$9 - 1 = 8 \text{ effective unknowns.}$$

We may therefore impose $h_{33} = 1$ (or $\|\mathbf{h}\| = 1$) without loss of generality.

Step 3: Count the equations per correspondence.

Each correspondence supplies one equation for x' and one for y' :

$$2 \text{ equations per point.}$$

Step 4: Divide.

$$N_{\min} = \frac{8}{2} = 4 \text{ points.}$$

Three points give only six equations for eight unknowns — under-determined by two, leaving a two-parameter family of homographies all consistent with the data. **Contrast with affine:** six unknowns and two equations per point gives three points, which is exactly why the previous example worked with three.

Step 5 (b): Build one DLT row pair.

For the correspondence $(x, y) \rightarrow (x', y')$ the two rows are

$$\begin{bmatrix} x & y & 1 & 0 & 0 & 0 & -xx' & -yx' & -x' \\ 0 & 0 & 0 & x & y & 1 & -xy' & -yy' & -y' \end{bmatrix}.$$

Step 6: Instantiate for each of the four identity correspondences.

For $(0, 0) \rightarrow (0, 0)$:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

For $(1, 0) \rightarrow (1, 0)$:

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & -1 & 0 & -1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

For $(0, 1) \rightarrow (0, 1)$:

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & -1 & -1 \end{bmatrix}$$

For $(1, 1) \rightarrow (1, 1)$:

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & -1 & -1 & -1 \\ 0 & 0 & 0 & 1 & 1 & 1 & -1 & -1 & -1 \end{bmatrix}$$

Step 7: Read off the constraints directly.

Row 1 of the first pair gives $h_{13} = 0$; row 2 gives $h_{23} = 0$. With those substituted, row 1 of the third pair gives $h_{12} + h_{13} = 0 \Rightarrow h_{12} = 0$, and row 2 of the second pair gives $h_{21} + h_{23} = 0 \Rightarrow h_{21} = 0$. Row 1 of the second pair then gives

$$h_{11} + h_{13} - h_{31} - h_{33} = 0 \Rightarrow h_{11} = h_{31} + h_{33},$$

and row 2 of the third pair gives $h_{22} = h_{32} + h_{33}$. Row 1 of the fourth pair gives

$$h_{11} + h_{12} + h_{13} - h_{31} - h_{32} - h_{33} = 0 \Rightarrow h_{11} = h_{31} + h_{32} + h_{33}.$$

Comparing with $h_{11} = h_{31} + h_{33}$ forces $h_{32} = 0$, and symmetrically $h_{31} = 0$.

Step 8: Assemble.

With $h_{31} = h_{32} = 0$, we get $h_{11} = h_{33}$ and $h_{22} = h_{33}$, so setting $h_{33} = 1$:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \mathbf{I}. \checkmark$$

The identity, exactly as expected — the four correspondences said nothing moved.

Step 9 (c): Change the fourth correspondence to $(1, 1) \rightarrow (2, 2)$.

The first three correspondences still force $h_{12} = h_{13} = h_{21} = h_{23} = 0$, $h_{11} = h_{31} + h_{33}$ and $h_{22} = h_{32} + h_{33}$. The new fourth pair, with $x = y = 1$ and $x' = y' = 2$, gives

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & -2 & -2 & -2 \\ 0 & 0 & 0 & 1 & 1 & 1 & -2 & -2 & -2 \end{bmatrix}.$$

Row 1: $h_{11} + h_{12} + h_{13} - 2h_{31} - 2h_{32} - 2h_{33} = 0$, i.e.

$$h_{11} = 2h_{31} + 2h_{32} + 2h_{33}.$$

Row 2 similarly gives $h_{22} = 2h_{31} + 2h_{32} + 2h_{33}$, so $h_{11} = h_{22}$.

Step 10: Combine with the earlier constraints.

From the second correspondence, $h_{11} = h_{31} + h_{33}$. Setting the two expressions equal:

$$h_{31} + h_{33} = 2h_{31} + 2h_{32} + 2h_{33} \Rightarrow 0 = h_{31} + 2h_{32} + h_{33}. \quad (*)$$

From the third correspondence, $h_{22} = h_{32} + h_{33}$, and $h_{22} = h_{11}$, so symmetrically

$$0 = 2h_{31} + h_{32} + h_{33}. \quad (**)$$

Subtracting $(*)$ from $(**)$: $h_{31} - h_{32} = 0$, so $h_{31} = h_{32}$. Substituting back into $(*)$: $3h_{31} + h_{33} = 0$, i.e. $h_{31} = -h_{33}/3$.

Step 11: Set $h_{33} = 1$ and evaluate.

$$h_{31} = h_{32} = -\frac{1}{3}, \quad h_{11} = h_{31} + h_{33} = -\frac{1}{3} + 1 = \frac{2}{3}, \quad h_{22} = \frac{2}{3}.$$

$$\mathbf{H} = \begin{bmatrix} 2/3 & 0 & 0 \\ 0 & 2/3 & 0 \\ -1/3 & -1/3 & 1 \end{bmatrix}.$$

Step 12: Verify each correspondence.

$$(0,0): \mathbf{H}(0,0,1)^\top = (0,0,1)^\top \rightarrow (0,0). \checkmark$$

$$(1,0): \mathbf{H}(1,0,1)^\top = (2/3, 0, -1/3 + 1)^\top = (2/3, 0, 2/3)^\top \rightarrow (1,0). \checkmark$$

$$(0,1): \mathbf{H}(0,1,1)^\top = (0, 2/3, 2/3)^\top \rightarrow (0,1). \checkmark$$

$$(1,1): \mathbf{H}(1,1,1)^\top = (2/3, 2/3, -1/3 - 1/3 + 1)^\top = (2/3, 2/3, 1/3)^\top \rightarrow (2,2). \checkmark$$

Step 13: Interpret.

The non-zero bottom row $h_{31} = h_{32} = -1/3$ is genuine perspective: the unit square has been mapped to a non-square quadrilateral in which the far corner is pushed outward. No affine transformation could have done this, since an affine map sending $(1,0) \rightarrow (1,0)$ and $(0,1) \rightarrow (0,1)$ and $(0,0) \rightarrow (0,0)$ would be forced to be the identity and would send $(1,1) \rightarrow (1,1)$, not $(2,2)$. The single changed correspondence is precisely what distinguishes the two families — which is the practical content of “affine needs three points, homography needs four”.

Answer.

(a) 8 degrees of freedom $\div$ 2 equations per point = 4 points. (b) The identity homography.

(c) $\mathbf{H} = \begin{bmatrix} 2/3 & 0 & 0 \\ 0 & 2/3 & 0 \\ -1/3 & -1/3 & 1 \end{bmatrix}$, a genuine perspective map that no affine transformation can reproduce.

EXAM PITFALL: Normalise your coordinates before running the DLT

In a real problem, pixel coordinates run into the thousands while the homogeneous coordinate is 1, so the columns of $\mathbf{A}$ differ in magnitude by three or four orders of magnitude and $\mathbf{A}^\top \mathbf{A}$ becomes badly conditioned. The standard remedy (Hartley normalisation) is to translate each point set so its centroid is at the origin and scale it so the mean distance from the origin is $\sqrt{2}$, solve for the normalised $\tilde{\mathbf{H}}$, then un-normalise: $\mathbf{H} = \mathbf{T}'^{-1} \tilde{\mathbf{H}} \mathbf{T}$. Mention this if a question asks why a DLT implementation gives poor results on real data.

PART VII

Camera Calibration

Camera Calibration

24.1 Intuitive Theory and Definition

Topic 17 built the projection matrix $\mathbf{P} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}]$ from *known* parameters. Calibration runs that backwards: given a set of 3D points whose world coordinates you know, and the pixels where they appear, recover $\mathbf{P}$ — and then extract $\mathbf{K}$, $\mathbf{R}$ and $\mathbf{t}$ from it.

KEY DEFINITION: Camera calibration

Also called *camera resectioning*. The estimation of a camera's intrinsic parameters ($\mathbf{K}$: focal lengths, principal point, skew) and extrinsic parameters ($\mathbf{R}$, $\mathbf{t}$: pose) from correspondences between known 3D world points and their observed 2D image points.

PHYSICAL ANALOGY: Surveying a room from one photograph

You are handed a photograph of a room and told the exact 3D position of a dozen landmarks — corners of a checkerboard, say, whose squares you measured with a ruler. Each landmark gives you one clue: “a ray from the camera centre through *this* pixel must pass through *that* 3D point.” One clue is nowhere near enough. But stack up eleven or more clues and the constraints intersect at a single answer: where the camera was, which way it pointed, and what its lens was doing. That is calibration — solving for the camera by using a scene you already know.

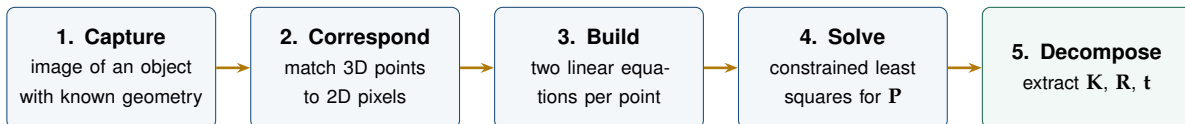


Figure 24.1: The calibration procedure. Steps 3 and 4 are the DLT of Topic 23 applied to 3D-to-2D rather than 2D-to-2D correspondences.

Source deck: *s5a_Camera_Calibration*, slides 1–12; *s5_Camera_Calibration*, slides 31–46

24.2 Mathematical Formulation and Derivation

24.2.1 Setting up the equations

Step 1: Write the projection with unknown $\mathbf{P}$.

For the i -th correspondence $(X_i, Y_i, Z_i) \leftrightarrow (u_i, v_i)$:

$$s_i \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}.$$

Step 2: Expand the three rows.

$$s_i u_i = p_{11}X_i + p_{12}Y_i + p_{13}Z_i + p_{14} \quad (24.1)$$

$$s_i v_i = p_{21}X_i + p_{22}Y_i + p_{23}Z_i + p_{24} \quad (24.2)$$

$$s_i = p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34} \quad (24.3)$$

Step 3: Eliminate the unknown scale s_i by division.

$$u_i = \frac{p_{11}X_i + p_{12}Y_i + p_{13}Z_i + p_{14}}{p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}}, \quad v_i = \frac{p_{21}X_i + p_{22}Y_i + p_{23}Z_i + p_{24}}{p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}}.$$

Note the structure — identical to the homography of Topic 14, except that the numerator and denominator are linear in *three* world coordinates rather than two.

Step 4: Cross-multiply to linearise.

$$u_i(p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}) = p_{11}X_i + p_{12}Y_i + p_{13}Z_i + p_{14} \quad (24.4)$$

$$v_i(p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}) = p_{21}X_i + p_{22}Y_i + p_{23}Z_i + p_{24} \quad (24.5)$$

Step 5: Rearrange to homogeneous form.

$$p_{11}X_i + p_{12}Y_i + p_{13}Z_i + p_{14} - u_i p_{31}X_i - u_i p_{32}Y_i - u_i p_{33}Z_i - u_i p_{34} = 0 \quad (24.6)$$

$$p_{21}X_i + p_{22}Y_i + p_{23}Z_i + p_{24} - v_i p_{31}X_i - v_i p_{32}Y_i - v_i p_{33}Z_i - v_i p_{34} = 0 \quad (24.7)$$

Step 6: Write as two rows of a matrix.

With $\mathbf{p} = (p_{11}, \dots, p_{14}, p_{21}, \dots, p_{24}, p_{31}, \dots, p_{34})^\top \in \mathbb{R}^{12}$:

$$\begin{bmatrix} X_i & Y_i & Z_i & 1 & 0 & 0 & 0 & 0 & -u_i X_i & -u_i Y_i & -u_i Z_i & -u_i \\ 0 & 0 & 0 & 0 & X_i & Y_i & Z_i & 1 & -v_i X_i & -v_i Y_i & -v_i Z_i & -v_i \end{bmatrix} \mathbf{p} = \mathbf{0}.$$

24.2.2 How many points?

IMPORTANT FORMULA: The calibration point count

$\mathbf{P}$ has 12 entries. Like a homography, it is **homogeneous** — multiplying $\mathbf{P}$ by λ scales s_i by λ and leaves (u_i, v_i) unchanged — so it has

$$12 - 1 = 11 \text{ degrees of freedom.}$$

Each correspondence gives 2 equations, so

$$N_{\min} = \left\lceil \frac{11}{2} \right\rceil = 6 \text{ points.}$$

Six points give 12 equations for 11 unknowns — just over-determined. In practice one uses far more (a checkerboard supplies dozens) and solves in the least-squares sense.

Consistency check: 11 = 5 intrinsic + 6 extrinsic, exactly as Topic 17 counted.

EXAM PITFALL: The calibration object must not be planar

If all the 3D points lie in a single plane, the system is rank-deficient and $\mathbf{P}$ cannot be recovered: any transformation of that plane onto the image is a homography, and infinitely many camera poses produce the same homography. Calibration objects are therefore either 3D (two or three mutually perpendicular checkerboard faces) or — as in Zhang’s widely used method — a single planar board photographed from *several different orientations*, which supplies the missing depth variation across images.

24.2.3 Fixing the scale: two options

Option 1: Set $p_{34} = 1$.

Move the p_{34} column to the right-hand side, turning $\mathbf{A}\mathbf{p} = \mathbf{0}$ into an *inhomogeneous* system

$$\mathbf{A}'\mathbf{p}' = \mathbf{b},$$

with 11 unknowns, solved by the pseudo-inverse of Topic 19. *Weakness:* if the true p_{34} is zero or near zero — which happens when the world origin lies at or near the camera centre — this normalisation is singular or ill-conditioned.

Option 2: Set $\|\mathbf{p}\| = 1$.

Keep the homogeneous form and solve the constrained problem

$$\min_{\|\mathbf{p}\|=1} \|\mathbf{A}\mathbf{p}\|^2.$$

By the master result of Topic 18, $\mathbf{p}$ is the eigenvector of $\mathbf{A}^\top \mathbf{A}$ with the smallest eigenvalue, equivalently the last column of $\mathbf{V}$ in $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$. *This is the method to quote:* it never degenerates, and it is what “constrained least squares” means in the calibration slides.

IMPORTANT FORMULA: Calibration by constrained least squares

$\mathbf{A}\mathbf{p} = \mathbf{0}, \quad \|\mathbf{p}\| = 1 \quad \implies \quad \mathbf{p} = \text{last column of } \mathbf{V}, \quad \mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top,$
then reshape $\mathbf{p} \in \mathbb{R}^{12}$ into the 3×4 matrix $\mathbf{P}$ row by row.

24.2.4 Decomposing $\mathbf{P}$ into $\mathbf{K}$, $\mathbf{R}$ and $\mathbf{t}$

Step 7: Split $\mathbf{P}$ into its blocks.

$$\mathbf{P} = \begin{bmatrix} \mathbf{M} & \mathbf{p}_4 \end{bmatrix}, \quad \mathbf{M} = \mathbf{K}\mathbf{R} \quad (3 \times 3), \quad \mathbf{p}_4 = \mathbf{K}\mathbf{t} \quad (3 \times 1).$$

Step 8: Exploit the structural forms.

$\mathbf{K}$ is **upper-triangular** with positive diagonal, and $\mathbf{R}$ is **orthonormal**. The decomposition of a matrix into (upper-triangular) $\times$ (orthonormal) is exactly an **RQ decomposition**, which is unique up to signs. So:

$$\mathbf{M} = \mathbf{K}\mathbf{R} \xrightarrow{\text{RQ}} \mathbf{K} \text{ and } \mathbf{R}.$$

Step 9: Fix the sign ambiguity.

RQ returns $\mathbf{K}$ and $\mathbf{R}$ only up to a diagonal sign matrix $\mathbf{D}$ with entries ± 1 , since $\mathbf{K}\mathbf{R} = (\mathbf{K}\mathbf{D})(\mathbf{D}\mathbf{R})$ when $\mathbf{D}^2 = \mathbf{I}$. Choose $\mathbf{D}$ so that the diagonal of $\mathbf{K}$ is positive (focal lengths must be positive), then ensure $\det \mathbf{R} = +1$.

Step 10: Recover $\mathbf{t}$.

From $\mathbf{p}_4 = \mathbf{K}\mathbf{t}$,

$$\mathbf{t} = \mathbf{K}^{-1}\mathbf{p}_4,$$

and $\mathbf{K}^{-1}$ is cheap because $\mathbf{K}$ is triangular.

Step 11: Recover the camera centre.

The camera centre $\mathbf{C}$ is the world point that projects to nothing, i.e. satisfies

$$\mathbf{P} \begin{bmatrix} \mathbf{C} \\ 1 \end{bmatrix} = \mathbf{0} \implies \mathbf{M}\mathbf{C} + \mathbf{p}_4 = \mathbf{0} \implies \mathbf{C} = -\mathbf{M}^{-1}\mathbf{p}_4.$$

Equivalently $\mathbf{C} = -\mathbf{R}^\top \mathbf{t}$, as derived in Topic 17. In SVD terms, $\mathbf{C}$ in homogeneous form is the null vector of $\mathbf{P}$.

IMPORTANT FORMULA: The full decomposition recipe

1. Split: $\mathbf{P} = [\mathbf{M} \mid \mathbf{p}_4]$.
2. RQ-decompose $\mathbf{M} = \mathbf{K}\mathbf{R}$.
3. Normalise so $\mathbf{K}_{33} = 1$ and the diagonal of $\mathbf{K}$ is positive; flip signs consistently so $\det \mathbf{R} = +1$.
4. Read off $f_x = \mathbf{K}_{11}$, $f_y = \mathbf{K}_{22}$, $o_x = \mathbf{K}_{13}$, $o_y = \mathbf{K}_{23}$, $\gamma = \mathbf{K}_{12}$.
5. $\mathbf{t} = \mathbf{K}^{-1}\mathbf{p}_4$; camera centre $\mathbf{C} = -\mathbf{M}^{-1}\mathbf{p}_4$.

24.2.5 What the linear method leaves out

KEY DEFINITION: Radial (lens) distortion

The pinhole model assumes rays travel in perfect straight lines through a point. Real lenses bend them, so straight world lines appear curved — bowing outward (*barrel* distortion, typical of wide-angle lenses) or inward (*pincushion*, typical of telephotos). The standard correction applies a radial polynomial to the normalised image coordinates:

$$x_{\text{corrected}} = x(1 + k_1r^2 + k_2r^4 + k_3r^6), \quad y_{\text{corrected}} = y(1 + k_1r^2 + k_2r^4 + k_3r^6),$$

where $r^2 = x^2 + y^2$ is the squared distance from the principal point. The distortion coefficients k_1, k_2, k_3 are *non-linear* in the projection equations and so cannot be recovered by the DLT; they are estimated by a non-linear refinement (typically Levenberg–Marquardt) initialised from the linear solution.

	Linear method (DLT)	Non-linear refinement
Solves for	$\mathbf{P}$, hence $\mathbf{K}$, $\mathbf{R}$, $\mathbf{t}$	All of the above plus k_1, k_2, k_3
Error minimised	Algebraic error $\ \mathbf{A}\mathbf{p}\ $	Geometric reprojection error
Needs an initial guess	No	Yes — supplied by the DLT
Enforces $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$	No	Yes
Speed	Closed form, fast	Iterative

Table 24.1: The linear method is not the final answer; it is the initialisation that makes the final answer reachable. Naming both stages is what a complete exam answer looks like.

EXAM PITFALL: The DLT does not enforce orthonormality

Solving $\mathbf{Ap} = \mathbf{0}$ treats all twelve entries of $\mathbf{P}$ as free. Nothing in the system forces the recovered $\mathbf{M} = \mathbf{KR}$ to contain a genuine rotation, so with noisy data the RQ decomposition returns an $\mathbf{R}$ that is only approximately orthonormal. Two standard fixes: project $\mathbf{R}$ onto the nearest true rotation using the SVD ($\mathbf{R} \leftarrow \mathbf{UV}^\top$ from $\mathbf{R} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$), or let the non-linear refinement re-parameterise the rotation by three angles so orthonormality holds by construction.

24.3 Solved Numerical Example

SOLVED NUMERICAL EXAMPLE: The DLT point count, and decomposing a given $\mathbf{P}$

Problem. (a) Show that the projection matrix has 11 independent unknowns and therefore needs 6 point correspondences. (b) Given

$$\mathbf{P} = \begin{bmatrix} -320 & 0 & 800 & 3200 \\ -240 & 800 & 0 & 2400 \\ -1 & 0 & 0 & 10 \end{bmatrix}$$

(the matrix constructed in Topic 17), recover $\mathbf{K}$, $\mathbf{R}$, $\mathbf{t}$ and the camera centre, and verify against the known ground truth.

Step 1 (a): Count entries.

$\mathbf{P}$ is 3×4 , so it has 12 entries.

Step 2: Subtract the scale ambiguity.

For any $\lambda \neq 0$, $\lambda\mathbf{P}$ maps a world point to $\lambda(\tilde{u}, \tilde{v}, \tilde{w})^\top$, and dividing by the third component gives

$$\frac{\lambda\tilde{u}}{\lambda\tilde{w}} = \frac{\tilde{u}}{\tilde{w}'}$$

the same pixel. So $\mathbf{P}$ and $\lambda\mathbf{P}$ are indistinguishable and one degree of freedom is redundant: $12 - 1 = 11$.

Step 3: Cross-check against the physical parameters.

$$\underbrace{f_x, f_y, o_x, o_y, \gamma}_{5 \text{ intrinsic}} + \underbrace{3 \text{ rotation angles} + 3 \text{ translation components}}_{6 \text{ extrinsic}} = 11. \checkmark$$

Two entirely independent arguments give the same number, which is the strongest evidence that the count is right.

Step 4: Divide by equations per point.

$$N_{\min} = \left\lceil \frac{11}{2} \right\rceil = 6.$$

Five points give only 10 equations — one short. Six give 12, one more than needed, and that extra equation is what makes the system solvable in the least-squares sense rather than exactly.

Step 5 (b): Split $\mathbf{P}$ into blocks.

$$\mathbf{M} = \begin{bmatrix} -320 & 0 & 800 \\ -240 & 800 & 0 \\ -1 & 0 & 0 \end{bmatrix}, \quad \mathbf{p}_4 = \begin{bmatrix} 3200 \\ 2400 \\ 10 \end{bmatrix}.$$

Step 6: Recover $\mathbf{R}$ and $\mathbf{K}$ by inspection of the structure.

We seek $\mathbf{M} = \mathbf{K}\mathbf{R}$ with $\mathbf{K}$ upper-triangular and $\mathbf{R}$ orthonormal. Write $\mathbf{K} = \begin{bmatrix} f_x & \gamma & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$ and let $\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3$ be the rows of $\mathbf{R}$. Then

$$\text{row 3 of } \mathbf{M} = \mathbf{r}_3 \quad (24.8)$$

$$\text{row 2 of } \mathbf{M} = f_y \mathbf{r}_2 + o_y \mathbf{r}_3 \quad (24.9)$$

$$\text{row 1 of } \mathbf{M} = f_x \mathbf{r}_1 + \gamma \mathbf{r}_2 + o_x \mathbf{r}_3 \quad (24.10)$$

Step 7: Extract $\mathbf{r}_3$ from the bottom row.

Row 3 of $\mathbf{M}$ is $(-1, 0, 0)$, which already has unit length:

$$\mathbf{r}_3 = (-1, 0, 0).$$

Step 8: Extract o_y and f_y from row 2.

Row 2 is $(-240, 800, 0)$. Since $\mathbf{r}_2 \perp \mathbf{r}_3$, taking the dot product of row 2 with $\mathbf{r}_3$ isolates o_y :

$$o_y = (\text{row 2}) \cdot \mathbf{r}_3 = (-240)(-1) + (800)(0) + (0)(0) = 240.$$

Subtract that component off:

$$f_y \mathbf{r}_2 = (-240, 800, 0) - 240(-1, 0, 0) = (-240 + 240, 800, 0) = (0, 800, 0).$$

Its length is f_y :

$$f_y = \|(0, 800, 0)\| = 800, \quad \mathbf{r}_2 = (0, 1, 0).$$

Step 9: Extract o_x, γ and f_x from row 1.

Row 1 is $(-320, 0, 800)$.

$$o_x = (\text{row 1}) \cdot \mathbf{r}_3 = (-320)(-1) + 0 + 0 = 320.$$

$$\gamma = (\text{row 1}) \cdot \mathbf{r}_2 = (-320)(0) + (0)(1) + (800)(0) = 0.$$

Subtract both components:

$$f_x \mathbf{r}_1 = (-320, 0, 800) - 320(-1, 0, 0) - 0 \cdot (0, 1, 0) = (0, 0, 800),$$

$$f_x = 800, \quad \mathbf{r}_1 = (0, 0, 1).$$

Step 10: Assemble $\mathbf{K}$ and $\mathbf{R}$.

$$\mathbf{K} = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{R} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}.$$

Step 11: Verify $\mathbf{R}$ is a genuine rotation.

Rows: $(0, 0, 1)$, $(0, 1, 0)$, $(-1, 0, 0)$ — each of unit length, and each pair orthogonal ($\mathbf{r}_1 \cdot \mathbf{r}_2 = 0$, $\mathbf{r}_1 \cdot \mathbf{r}_3 = 0$, $\mathbf{r}_2 \cdot \mathbf{r}_3 = 0$). ✓

$$\det \mathbf{R} = 0[(1)(0) - (0)(0)] - 0[(0)(0) - (0)(-1)] + 1[(0)(0) - (1)(-1)] = 1. \checkmark$$

Step 12: Verify the product reconstructs $\mathbf{M}$.

$$\mathbf{KR} = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}.$$

Row 1: $(800)(0) + (0)(0) + (320)(-1) = -320$; $(800)(0) + (0)(1) + (320)(0) = 0$; $(800)(1) + 0 + 0 = 800. \Rightarrow (-320, 0, 800) \checkmark$ Row 2: $(0)(0) + (800)(0) + (240)(-1) = -240$; $(0)(0) + (800)(1) + 0 = 800$; $0. \Rightarrow (-240, 800, 0) \checkmark$ Row 3: $(0)(0) + (0)(0) + (1)(-1) = -1$; 0 ; $0. \Rightarrow (-1, 0, 0) \checkmark$

Step 13: Recover $\mathbf{t}$.

$$\mathbf{t} = \mathbf{K}^{-1} \mathbf{p}_4.$$

Because $\mathbf{K}$ is upper-triangular, invert it by back-substitution rather than a general formula. From $\mathbf{Kt} = \mathbf{p}_4$:

$$\text{row 3: } t_z = 10 \quad (24.11)$$

$$\text{row 2: } 800 t_y + 240 t_z = 2400 \implies 800 t_y = 2400 - 2400 = 0 \implies t_y = 0 \quad (24.12)$$

$$\text{row 1: } 800 t_x + 0 t_y + 320 t_z = 3200 \implies 800 t_x = 3200 - 3200 = 0 \implies t_x = 0 \quad (24.13)$$

$$\mathbf{t} = \begin{bmatrix} 0 \\ 0 \\ 10 \end{bmatrix}.$$

Step 14: Recover the camera centre.

$$\mathbf{C} = -\mathbf{R}^T \mathbf{t} = - \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 10 \end{bmatrix} = - \begin{bmatrix} (0)(0) + (0)(0) + (-1)(10) \\ 0 \\ (1)(0) + 0 + 0 \end{bmatrix} = - \begin{bmatrix} -10 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 10 \\ 0 \\ 0 \end{bmatrix}.$$

Step 15: Compare with ground truth.

Quantity	Recovered	Ground truth (Topic 17)
f_x, f_y	800, 800	800, 800 ✓
(o_x, o_y)	(320, 240)	(320, 240) ✓
γ	0	0 ✓
$\mathbf{R}$	$\mathbf{R}_y(90^\circ)$	$\mathbf{R}_y(90^\circ)$ ✓
$\mathbf{t}$	$(0, 0, 10)^\top$	$(0, 0, 10)^\top$ ✓
$\mathbf{C}$	$(10, 0, 0)^\top$	$(10, 0, 0)^\top$ ✓

Every parameter is recovered exactly, because $\mathbf{P}$ here was noise-free. With real measurements the recovered values would carry small errors, $\mathbf{R}$ would be only approximately orthonormal, and the non-linear refinement of the previous section would be required.

Answer.

(a) $12 - 1 = 11$ degrees of freedom, hence 6 points. (b) $\mathbf{K} = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix}$, $\mathbf{R} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$, $\mathbf{t} = (0, 0, 10)^\top$, camera centre $\mathbf{C} = (10, 0, 0)^\top$ — all matching the ground truth.

SOLVED NUMERICAL EXAMPLE: Why $p_{34} = 1$ can fail, worked concretely

Problem. Show that the normalisation $p_{34} = 1$ is invalid when the world origin coincides with the camera centre, and state what to do instead.

Step 1: Recall the structure of the last column.

$$\mathbf{p}_4 = \mathbf{K}\mathbf{t} \implies p_{34} = (\text{row 3 of } \mathbf{K}) \cdot \mathbf{t} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \mathbf{t} = t_z.$$

So p_{34} is precisely the z-component of the translation — the depth of the world origin in camera coordinates.

Step 2: Consider the degenerate configuration.

If the camera centre sits at the world origin, then $\mathbf{C} = \mathbf{0}$, and from $\mathbf{t} = -\mathbf{R}\mathbf{C}$ we get $\mathbf{t} = \mathbf{0}$, hence

$$p_{34} = t_z = 0.$$

Step 3: See what the normalisation does.

Setting $p_{34} = 1$ asserts that the true value is non-zero and rescales the whole matrix by $1/p_{34}$. With $p_{34} = 0$ that is division by zero: the normalisation is undefined. With p_{34} merely *small* — say the world origin is 1 mm in front of the camera while scene points are metres away — the rescaling factor is enormous and amplifies measurement noise correspondingly.

Step 4: Quantify the conditioning problem.

Suppose $t_z = 0.001$ and the measurement noise on $\mathbf{p}$ is ε . Rescaling by $1/t_z = 1000$ multiplies that noise by 1000 as well, so a 0.1% measurement error becomes a 100% parameter error. The estimate is numerically worthless even though nothing has formally divided by zero.

Step 5: State the remedy.

Use the norm constraint $\|\mathbf{p}\| = 1$ instead. It is a constraint on the whole 12-vector, so it is satisfiable for *every* non-zero $\mathbf{P}$ regardless of which individual entries vanish, and it leads to the smallest-eigenvector solution of Topic 18:

$$\mathbf{p} = \arg \min_{\|\mathbf{p}\|=1} \|\mathbf{A}\mathbf{p}\|^2 = \text{last column of } \mathbf{V} \text{ in } \mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top.$$

Answer.

$p_{34} = t_z$, which is zero exactly when the camera centre is at the world origin and small whenever it is near it; the normalisation then becomes undefined or ill-conditioned. Use $\|\mathbf{p}\| = 1$ and solve by SVD.

PART VIII

Appendices

Linear Algebra Refresher

This appendix collects the linear algebra the main text assumes. Nothing here is examinable on its own, but every item is used somewhere above.

A.1 Eigenvalues and eigenvectors

KEY DEFINITION: Eigenvector

A non-zero vector $\mathbf{v}$ is an eigenvector of the square matrix $\mathbf{M}$ if

$$\mathbf{M}\mathbf{v} = \lambda\mathbf{v}$$

for some scalar λ , the corresponding **eigenvalue**. Geometrically: $\mathbf{v}$ is a direction that $\mathbf{M}$ stretches without rotating.

Finding them, for a 2×2 matrix.

$$\mathbf{M} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Step 1. Rewrite $\mathbf{M}\mathbf{v} = \lambda\mathbf{v}$ as $(\mathbf{M} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$.

Step 2. A non-zero solution exists only if $\mathbf{M} - \lambda\mathbf{I}$ is singular, i.e.

$$\det(\mathbf{M} - \lambda\mathbf{I}) = 0.$$

Step 3. Expand for the 2×2 case:

$$(a - \lambda)(d - \lambda) - bc = \lambda^2 - (a + d)\lambda + (ad - bc) = 0,$$

that is

$$\lambda^2 - (\text{trace } \mathbf{M})\lambda + \det \mathbf{M} = 0.$$

Step 4. Solve by the quadratic formula:

$$\lambda = \frac{\text{trace } \mathbf{M} \pm \sqrt{(\text{trace } \mathbf{M})^2 - 4 \det \mathbf{M}}}{2}.$$

Step 5. For each λ , substitute back into $(\mathbf{M} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$ and read off the direction (the two rows will be proportional — that is the check that λ is right).

IMPORTANT FORMULA: Facts used repeatedly in this guide

For any square $\mathbf{M}$ with eigenvalues λ_i :

$$\text{trace } \mathbf{M} = \sum_i \lambda_i \tag{A.1}$$

$$\det \mathbf{M} = \prod_i \lambda_i \tag{A.2}$$

If $\mathbf{M}$ is **real symmetric** (as every scatter and second-moment matrix in this course is), then

all eigenvalues are real, all are non-negative when $\mathbf{M} = \mathbf{X}^\top \mathbf{X}$, and eigenvectors for distinct eigenvalues are orthogonal.

A.2 The singular value decomposition

KEY DEFINITION: SVD

Any real $m \times n$ matrix factors as

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$$

where $\mathbf{U}$ is $m \times m$ orthonormal, $\mathbf{V}$ is $n \times n$ orthonormal, and $\mathbf{\Sigma}$ is $m \times n$ diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$, the **singular values**.

The connection to eigen-decomposition, which is why the SVD appears everywhere above.

$$\mathbf{A}^\top \mathbf{A} = (\mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top)^\top (\mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top) = \mathbf{V} \mathbf{\Sigma}^\top \mathbf{U}^\top \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top = \mathbf{V} (\mathbf{\Sigma}^\top \mathbf{\Sigma}) \mathbf{V}^\top.$$

So the columns of $\mathbf{V}$ are the eigenvectors of $\mathbf{A}^\top \mathbf{A}$, and the eigenvalues are σ_i^2 . Since the σ_i are sorted descending, the *last* column of $\mathbf{V}$ is the eigenvector of the *smallest* eigenvalue.

IMPORTANT FORMULA: The recipe every homogeneous problem uses

$$\arg \min_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\|^2 = \text{last column of } \mathbf{V}, \quad \text{minimum value} = \sigma_{\min}^2.$$

This one line solves total least squares (Topic 20), homography estimation (Topic 23) and camera calibration (Topic 24).

A.3 The pseudo-inverse

For the *inhomogeneous* over-determined system $\mathbf{A}\mathbf{p} = \mathbf{b}$ with $\mathbf{A}$ of full column rank,

$$\mathbf{p} = \mathbf{A}^+ \mathbf{b}, \quad \mathbf{A}^+ = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top.$$

In terms of the SVD, $\mathbf{A}^+ = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^\top$ where $\mathbf{\Sigma}^+$ inverts the non-zero singular values and transposes. The SVD form is preferred numerically because it never forms $\mathbf{A}^\top \mathbf{A}$, whose condition number is the square of $\mathbf{A}$'s.

A.4 Matrix calculus identities

Expression	Derivative with respect to $\mathbf{x}$
$\mathbf{a}^\top \mathbf{x}$	$\mathbf{a}$
$\mathbf{x}^\top \mathbf{x}$	$2\mathbf{x}$
$\mathbf{x}^\top \mathbf{M} \mathbf{x}$ (with $\mathbf{M}$ symmetric)	$2\mathbf{M}\mathbf{x}$
$\ \mathbf{A}\mathbf{x} - \mathbf{b}\ ^2$	$2\mathbf{A}^\top (\mathbf{A}\mathbf{x} - \mathbf{b})$

A.5 Quick 2×2 and 3×3 formulas

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$\det \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} = a(ei - fh) - b(di - fg) + c(dh - eg)$$

For an orthonormal $\mathbf{R}$: $\mathbf{R}^{-1} = \mathbf{R}^\top$, so never invert a rotation numerically — transpose it.

Master Formula Sheet

Everything worth memorising, in syllabus order.

B.1 Image processing

IMPORTANT FORMULA: Point processing and sampling

$$\text{Contrast stretch: } g = 255 \frac{f - f_{\min}}{f_{\max} - f_{\min}} \quad (\text{B.1})$$

$$\text{Gamma: } g = 255 \left(\frac{f}{255} \right)^\gamma \quad (\text{B.2})$$

$$\text{Nyquist: } \omega_s > 2 \omega_{\max} \quad (\text{B.3})$$

$$\text{Alias frequency: } \omega_{\text{alias}} = \min_k |\omega_0 - k\omega_s| \quad (\text{B.4})$$

IMPORTANT FORMULA: Interpolation and filtering

$$\text{Bilinear: } f = (1 - a)(1 - b)Q_{11} + a(1 - b)Q_{21} + (1 - a)bQ_{12} + abQ_{22} \quad (\text{B.5})$$

$$\text{Convolution: } g[i, j] = \sum_{u, v} h[u, v] f[i - u, j - v] \quad (\text{B.6})$$

$$\text{Correlation: } g[i, j] = \sum_{u, v} h[u, v] f[i + u, j + v] \quad (\text{B.7})$$

$$\text{Gaussian: } h_\sigma(u, v) = \frac{1}{2\pi\sigma^2} e^{-(u^2 + v^2)/2\sigma^2} \quad (\text{B.8})$$

$$\text{Cascade: } \sigma_{\text{total}} = \sqrt{\sigma_1^2 + \sigma_2^2} \quad (\text{B.9})$$

$$\text{Kernel size: } k = 2\lceil 3\sigma \rceil + 1 \quad (\text{B.10})$$

$$\text{Unsharp mask: } h = (1 + \alpha)\delta - \alpha h_{\text{blur}} \quad (\text{B.11})$$

IMPORTANT FORMULA: Edges and corners

$$\text{Gradient magnitude: } \|\nabla f\| = \sqrt{f_x^2 + f_y^2} \quad (\text{B.12})$$

$$\text{Gradient direction: } \theta = \text{atan2}(f_y, f_x) \quad (\text{edge runs at } \theta + 90^\circ) \quad (\text{B.13})$$

$$\text{Sobel: } \mathbf{S}_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad \mathbf{S}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (\text{B.14})$$

$$\text{DoG: } \frac{\partial}{\partial x}(g_\sigma * f) = \left(\frac{\partial g_\sigma}{\partial x}\right) * f \quad (\text{B.15})$$

$$\text{Second moment: } \mathbf{M} = \sum w \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix} \quad (\text{B.16})$$

$$\text{Harris response: } R = \det \mathbf{M} - k(\text{trace } \mathbf{M})^2, \quad k \in [0.04, 0.06] \quad (\text{B.17})$$

B.2 Geometric transformations

IMPORTANT FORMULA: The hierarchy

Transformation	DoF	Min. correspondences
Translation	2	1
Rigid	3	2
Similarity	4	2
Affine	6	3
Projective (homography)	8	4
Camera matrix $\mathbf{P}$	11	6

$$\text{Rotation: } \mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad (\text{B.18})$$

$$\text{Translation (homog.): } \mathbf{T} = \begin{bmatrix} 1 & 0 & b_x \\ 0 & 1 & b_y \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{B.19})$$

$$\text{Rotate about } (c_x, c_y): \quad \mathbf{T}(c) \mathbf{R}(\theta) \mathbf{T}(-c) \quad (\text{B.20})$$

$$\text{Homography: } x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}} \quad (\text{B.21})$$

$$\text{Inverse warp: } g(x') = f(T^{-1}(x')) \quad (\text{B.22})$$

B.3 Camera geometry

IMPORTANT FORMULA: Projection and calibration

$$\text{Perspective: } x = f \frac{X}{Z}, \quad y = f \frac{Y}{Z} \quad (\text{B.23})$$

$$\text{Magnification: } m = \frac{f}{Z} \quad (\text{B.24})$$

$$\text{Vanishing point: } \mathbf{v} = \left(f \frac{d_x}{d_z}, f \frac{d_y}{d_z} \right) \quad (\text{B.25})$$

$$\text{Intrinsics: } \mathbf{K} = \begin{bmatrix} f_x & \gamma & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{B.26})$$

$$\text{Camera model: } s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}] \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (\text{B.27})$$

$$\text{Camera centre: } \mathbf{C} = -\mathbf{R}^\top \mathbf{t} = -\mathbf{M}^{-1} \mathbf{p}_4 \quad (\text{B.28})$$

$$\text{Radial distortion: } x_c = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \quad (\text{B.29})$$

B.4 Fitting and robustness

IMPORTANT FORMULA: Estimation

$$\text{LS line: } m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad b = \bar{y} - m\bar{x} \quad (\text{B.30})$$

$$\text{Normal equations: } \mathbf{p} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b} \quad (\text{B.31})$$

$$\text{TLS: } \mathbf{n} = \text{eigenvector of } \mathbf{X}^\top \mathbf{X} \text{ for } \lambda_{\min}, \quad d = a\bar{x} + b\bar{y} \quad (\text{B.32})$$

$$\text{Homogeneous solve: } \mathbf{x} = \text{last column of } \mathbf{V}, \quad \mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \quad (\text{B.33})$$

$$\text{Lagrange: } \nabla f = \lambda \nabla g \quad (\text{B.34})$$

$$\text{RANSAC iterations: } N = \left\lceil \frac{\log(1-p)}{\log(1-w^s)} \right\rceil \quad (\text{B.35})$$

$$\text{Hough vote: } \rho = x \cos \theta + y \sin \theta \quad (\text{B.36})$$

$$\text{Ratio test: } d_1/d_2 < 0.8 \quad (\text{B.37})$$

B.5 The one decision tree you will need

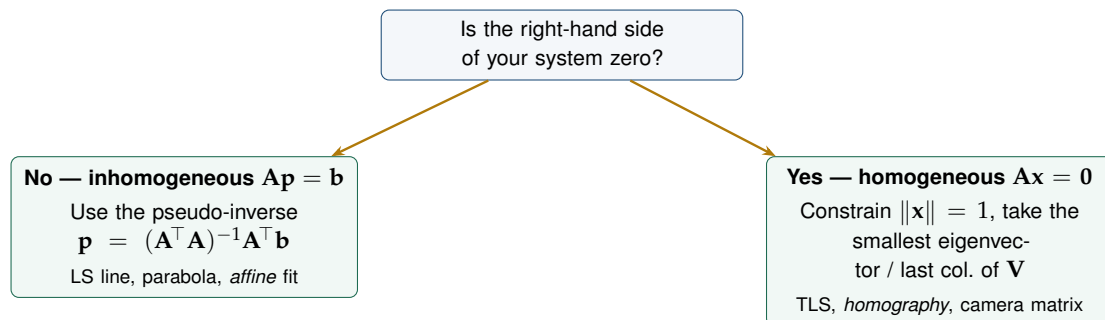


Figure B.1: Nine-tenths of the estimation questions in this syllabus are answered by deciding which branch you are on.

All Exam Pitfalls, Collected

Every red box from the main text, in one place, for the final hour before the paper.

Foundations

1. **Row-column versus x - y .** $I[i, j]$ is row i , column j ; geometry uses (x, y) with x horizontal. State your convention before writing any matrix.
2. **Clipping is not optional and not invertible.** Always apply $\min(255, \max(0, g))$, and remember that once a value clips the original is unrecoverable.
3. **Nyquist concerns the *maximum* frequency**, not the dominant one. Square-wave harmonics and edges are what actually alias. The condition is $\omega_s > 2\omega_{\max}$, with the factor of two.
4. **Bilinear cannot overshoot; bicubic can.** Bilinear weights are non-negative and sum to one, so the output always lies within the corner values' range.

Filtering

5. **The 180° flip matters for asymmetric kernels.** Convolution and correlation with Sobel differ in *sign*. Say which one you are computing.
6. **Sharpening amplifies noise** and creates halos — that is the expected answer to “why does my sharpened image look gritty?”.
7. **Variances add, not standard deviations:** $\sigma_{\text{total}} = \sqrt{\sigma_1^2 + \sigma_2^2}$. Two $\sigma = 3$ blurs give 4.24, not 6.
8. **Choose σ and kernel size together.** $\sigma = 5$ in a 3×3 window is effectively a box filter.
9. **Match the filter to the noise.** Gaussian noise $\rightarrow$ averaging; salt-and-pepper $\rightarrow$ median; edges plus noise $\rightarrow$ bilateral, but *after* impulses are removed. The bilateral filter *preserves* impulse noise, treating it as a sharp edge.
10. **The median filter is not a convolution**, so linearity, separability and associativity do not apply to it.

Edges and corners

11. **Gradient direction $\neq$ edge direction.** They differ by 90° .
12. **NMS compares exactly two neighbours**, interpolated along the gradient direction — not the four or eight neighbours generally.
13. **Harris is not scale-invariant.** It *is* invariant to translation, in-plane rotation and additive intensity change; it is *not* invariant to zoom, and only partially to intensity scaling ($R \rightarrow a^4 R$).
14. **Eigenvalue ratio alone cannot separate corner from flat.** A flat noisy patch can have the same ratio as a corner; it is the *magnitude* that distinguishes them, which is why R combines det and trace.

Transformations

15. **A 2×2 matrix cannot translate**, because $T\mathbf{0} = \mathbf{0}$ always. This is the entire motivation for homogeneous coordinates.
16. **Divide by $\tilde{w}$** . You get away with skipping it for affine maps ($\tilde{w} = 1$) and then silently get a wrong answer for a homography.
17. **Matrices compose right to left**. $ABCx$ applies C first.
18. **Count free parameters, not matrix entries**. Rigid has 3 DoF despite four entries in its rotation block; a homography has 8 DoF despite nine entries.
19. **A homography does not preserve parallelism** or ratios of lengths — only straightness and the cross-ratio.
20. $h_{33} = 1$ is a **normalisation you chose**, not a property of the matrix.
21. **Inverse warping applies T^{-1}** . To produce an output rotated by $+\theta$, sample the input at $R(-\theta)x'$.

Camera geometry

22. **Convert units before substituting**. Focal length in mm, distances in m, answers in pixels: pick one unit, and convert to pixels only at the end via the pixel pitch.
23. f in millimetres $\neq f_x$ in pixels. $f_{px} = f_{mm}/\text{pitch}$.
24. **$\mathbf{t}$ is not the camera position**. The camera centre is $\mathbf{C} = -\mathbf{R}^T \mathbf{t}$.
25. **A planar calibration object is degenerate** for single-image DLT. Use a 3D target or several views of a plane.
26. $p_{34} = 1$ **fails when $t_z \approx 0$** . Use $\|\mathbf{p}\| = 1$ and SVD.
27. **The DLT does not enforce $\mathbf{R}^T \mathbf{R} = \mathbf{I}$** . Project onto the nearest rotation, or refine non-linearly.

Fitting

28. **Vertical least squares cannot fit a vertical line** and is unstable near one. That is a defect of the model, not of the code.
29. **Take the *smallest* eigenvector in TLS**. The largest gives the direction *along* the line; you want the normal.
30. **Affine uses the pseudo-inverse; homography uses the smallest eigenvector**. Inhomogeneous versus homogeneous — decide this first.
31. **Normalise coordinates before the DLT** (Hartley normalisation), or the system is badly conditioned.
32. **RANSAC's N does not depend on the data size** but grows exponentially in the sample size s . Keep s minimal.
33. **Do not forget the RANSAC refit**. The final model should be least-squares fitted to all inliers, not left as the minimal-sample hypothesis.
34. **Choosing the scale is the hard part** — for RANSAC's t , for Huber's δ , and for the Hough bin size. Too large loses robustness; too small flattens the objective and scatters the votes.
35. **Do not drop a Lagrange branch**. When a factorisation gives $2x(y - \lambda) = 0$, evaluate both $x = 0$ and $y = \lambda$ before discarding either, and check maximum versus minimum at the end.

KEY DEFINITION: The three questions to ask of any estimation problem

1. **How many degrees of freedom does my model have**, and therefore how many correspondences do I need?
2. **Is my system homogeneous or inhomogeneous**, and therefore do I use the pseudo-inverse or the smallest eigenvector?
3. **Do I expect outliers**, and therefore do I need RANSAC around the whole thing?

Answer these three and the rest is arithmetic.